

MEMORANDUM
RM-3731-CC
JUNE 1963

**THE LOGIC THEORY MACHINE:
A MODEL HEURISTIC PROGRAM**

Einar Stefferud

PREPARED FOR:
CARNEGIE CORPORATION

The **RAND** *Corporation*
SANTA MONICA • CALIFORNIA

MEMORANDUM
RM-3731-CC
JUNE 1963

**THE LOGIC THEORY MACHINE:
A MODEL HEURISTIC PROGRAM**

Einar Stefferud

This research is sponsored by the Carnegie Corporation. Views or conclusions contained in this Memorandum should not be interpreted as representing the official opinion or policy of the Carnegie Corporation.

The **RAND** *Corporation*
SANTA MONICA • CALIFORNIA -

PREFACE

This Memorandum has been prepared to fill the need for a model program for use in teaching Information Processing Language-V (IPL-V). Experience in teaching IPL programming has shown that class discussion of a good program, developed as a pedagogical tool, is an essential ingredient for comprehending IPL applications and potentialities.

The IPL computer programming language was originally developed at The RAND Corporation, under U. S. Air Force Project RAND, and at Carnegie Institute of Technology, for expressing complex computer programs. IPL is being used to great advantage in dealing with problems requiring flexible memory structures and hierarchies of subroutines and data. IPL is also being applied in the field of artificial intelligence in studies of complex information processing, and by psychologists who are using computer simulation of human cognitive processes.

A number of colleges and universities now offer courses in IPL-V coding, and more such courses appear imminent. IPL coding was taught at the 1962 Summer Heuristic Programming Institute, held at The RAND Corporation and sponsored by the Carnegie Corporation. At that time, several interesting IPL programs were available, but none could serve as a pedagogical model. The Logic Theory Machine (LT) was determined to be the best candidate for such a model, and a new version, documented in this Memorandum, was developed. Bert F. Green and Fred M. Tonge consulted on the development of the program and preparation of the Memorandum.

Use of LT presumes familiarity with the Information Processing Language-V Manual⁽¹⁾ and particularly with Part Two, "Programmers' Reference Manual." In most cases, LT will be used in conjunction with a study of Part One of the Manual, "The Elements of IPL Programming."[†]

This Memorandum was made possible by funds granted by Carnegie Corporation of New York. The statements made and views expressed are solely the responsibility of the author.

[†] LT may be used by advanced students who have completed Part One of the Manual, or may be used simultaneously with Part One to illustrate the implementation in a complete program of the concepts being developed.

SUMMARY

This Memorandum contains a highly detailed program listing for the Logic Theory Machine (LT), a computer program written in Information Processing Language-V (IPL-V), and developed especially for use as a pedagogical model. The text portions of the Memorandum expand upon the documentation in the listing, tracing program flow, analyzing routines utilized, and providing insight into the structure and the development of the program.

LT was originally programmed in an early version of IPL by Newell, Shaw, and Simon⁽²⁻⁴⁾ to derive proofs of logic expressions in the sentential calculus of Whitehead and Russell.⁽⁵⁾ In rewriting the program for use as a teaching aid, a new method of replacement on subexpressions has been included, and many minor changes effecting improvements in clarity have been incorporated. Features of the code that were unjustifiably hard to explain have been simplified.

The Memorandum defines LT's activity in terms of problem solving, and then a representation of the defined problem is given in terms of IPL-V. Finally, what LT does is discussed in terms of process hierarchies which operate on the list structure representations of logic expressions.

LT can be implemented on any computer for which an IPL-V processor is available.[†]

[†]These include the IBM 650, 704, 709, 7090, 7094, Philco 2000, Bendix G-20, CDC 1604, UNIVAC 1105, and the AN/FSQ-32. A system for the Burroughs 220 is under development. LT, or any IPL-V program written in accordance with the IPL-V Manual, can be executed on any of these machines. In order to facilitate student modification of LT, information on obtaining the program deck can be obtained by writing The RAND Corporation.

CONTENTS

PREFACE	iii
SUMMARY	v
Section	
I. INTRODUCTION	1
II. WHAT LT DOES	3
The Theorem-Proving Problem	3
Proof Examples	4
III. REPRESENTATION OF "THE PROBLEM" IN THE IPL-V MACHINE	5
IV. HOW LT WORKS	8
V. SUBSTITUTION AND MATCHING	16
VI. THE OTHER METHODS	24
VII. UTILITY MEASURES	33
VIII. INFORMATION STORAGE AND RETRIEVAL	38
IX. LOWER-LEVEL ROUTINES	50
X. INPUT-OUTPUT ROUTINES	62
XI. RUN EXECUTIVES AND DEBUGGING ROUTINES	67
XII. A SAMPLE RUN	70
Input Deck	70
Output	77
XIII. LEVELS OF VOCABULARY AND USE OF SYMBOLS ..	119
XIV. COMPLETE VOCABULARY LISTING	122
XV. COMPLETE PROGRAM LISTING	129
REFERENCES	185

LIST OF FIGURES

Figure

1.	The Single-Problem Executive	12
2.	Sample Subproblem Tree	13
3.	The Substitution Method	17
4.	Test for Match with Substitution	18
5.	The Match Process	18
6.	The Match Subprocess	19
7.	Match with Substitution and Output Expanded Substitution List	21
8.	Expand Substitution List	21
9.	Delineate Expression	22
10.	Delineate Free Variable	23
11.	Replace Free Variable by Delineated Substitutor	23
12.	Detachment Method	25
13.	Detachment Method Subprocess.....	26
14.	Sublevel Replacement Method I	27
15.	Try to Replace Located Segment	28
16.	Try to Replace Segment by Matching Left (Right) Sides	29
17.	Finish Building New Subproblem	31
18.	Measure Utility	34
19.	Add to Found List	35
20.	Test Match of Total Expressions Without Substitution	36
21.	Test Match of Segments Without Substitution	37
22.	Axiom Map	41
23.	Axiom Map in Key punched Form	42-43
24.	Add Expression to Map	44
25.	Create List of Feasible Expressions	46
26.	Axiom Map with Overlaid Problem	47
27.	Find Number of Levels	52
28.	Count Sublevels Recursively	53
29.	Find Number of Distinct Variables	54
30.	Generate Locations of Bound Variables	55
31.	Generate Segment Locations from Given Level of Expression	56
32.	Generate Subsegment Locations from Segment	57
33.	Create Main Segment from List Expression ..	59
34.	Create Next Segment from List Expression ..	60
35.	Read Next Logic Expression	64-65

I. INTRODUCTION

Experience in teaching IPL (Information Processing Language)[†] programming has shown that class discussion of a good model program is an essential ingredient for comprehension of applications and potentialities of the IPL concepts. This documentation of the "Logic Theory Machine,"⁽²⁾ otherwise known as LT, is specifically aimed at filling the need for such a program.

LT was the best candidate for development as a pedagogical model because 1) it is based in the readily understood context of theorem proving in sentential calculus; 2) literature on theorem proving is readily available;^(3,4) and 3) the original LT, written in an early version of IPL by Newell, Shaw, and Simon, established the field of heuristic programming. Because it is a valuable example for students to examine and modify, LT has survived beyond its usefulness as a research tool.

The original version of LT was converted to IPL-V from IPL-II by Fred Tonge, and later converted into the present pedagogical model.[‡] The questions originally raised by LT are still valid and interesting, and although conversion has introduced some changes in LT's procedures, its structure remains essentially unchanged and its operating performance is nearly identical to that of the original.

LT does not represent an effort to obtain high machine efficiency. It is rather an effort to take ad-

[†]A discussion of the development of the IPL's can be found in the introduction to the IPL-V Manual. (1)

[‡]The latter conversion was made by the author.

vantage of the powers of symbolization via heuristic symbol manipulation techniques. It is not very successful in comparison with people or even with some computer programs.^(6,7) (For an excellent treatment of this topic, see Minsky's discussion of problem solving.)⁽⁸⁾

Since this Manual is designed for class use by students of list processing languages, the discussions and descriptions of LT assume familiarity with IPL-V. Several viewpoints are adopted for discussion purposes: first, what LT does is defined in terms of problem solving; then, representation of the defined problem is described in terms of IPL-V; and finally, what LT does is discussed in terms of process hierarchies which operate on the list structure representations of logic expressions.

The text describing LT is not intended to completely describe the program. Sections XIII through XV contain listings of the entire program along with supporting vocabulary listings. The text and flow diagrams are only intended to introduce the student to the program listing, which provides sufficient annotation to allow him to dissect LT's processes and learn how IPL-V can be used for complex information processing.

To facilitate detailed inspection, the entire program has been carefully and extensively documented in the comment fields. Outlines and flow diagrams have been supplied for some key routines, but others have been left for the student to construct, since learning to write down routine specifications is one of the most important facets of learning to use IPL-V.

II. WHAT LT DOES

THE THEOREM-PROVING PROBLEM

LT derives proofs of logic expressions in the sentential calculus of Whitehead and Russell.⁽⁵⁾ To do this, it uses the following entities:

- 1) Expressions, compounded from:
 - a) free variables:[†]
A,B,C,D,E,F,G
 - b) bound variables:
P,Q,R,S,T
 - c) connectives:
 - intuitive meaning: NOT
 - V intuitive meaning: OR
 - I intuitive meaning: IMPLIES
 - * intuitive meaning: AND
 - = intuitive meaning: EQUIVALENT TO
 - .=. intuitive meaning: EQUIVALENT TO
BY DEFINITION
- 2) Axioms (expressions given as true):
 - *1.2 [AVA]IA
 - *1.3 AI[BVA]
 - *1.4 [AVB]I[BVA]
 - *1.5 [AV[BVC]]I[BV[AVC]]
 - *1.6 [AIB]I[[CVA]I[CVB]]
- 3) Definitions (expressions defining connectives):
 - *1.01 [AIB].=[-AVB]DEF.
 - *3.01 [A*B].=[-AV-B]DEF.
 - *4.01 [A=B].=[[AIB]*[BIA]]DEF.

[†] The distinction between bound and free variables is needed to distinguish variables of true expressions from variables of unproved expressions.

4) Problems (expressions proposed as provable theorems):

*2.08 PIP
 *2.14 --PIP
 *2.45 -(PVQ)I-P
 *3.22 (P*Q)I(Q*P)
 3.24 -(P-P)
 *4.20 P=P
 etc.

5) Methods based on rules of inference:[†]

Substitution (for free variables)
 Replacement (through definitions)
 Detachment (A and AIB⇒B)
 Chaining (AIB and BIC⇒AIC)

PROOF EXAMPLES

TO PROVE:	*2.08	PIP	
	*1.3	AI[BVA]	Given
		PI[PVP]	Substitution
	*1.2	[AVA]IA	Given
		[PVP]IP	Substitution
	*2.08	PIP	Chaining
		Q.E.D.	

Chaining uses the two expressions PI[PVP] and [PVP]IP to yield PIP.

TO PROVE:	*2.14	--PIP	
	*2.13	AV---A	Given
		PV---P	Substitution
	*1.4	[AVB]I[BVA]	Given
		[PV---P]I[---PVP]	Substitution
		---PVP	Detachment
	*1.01	[AIB].=[-AVB]	Given
		[--PIP].=[---PVP]	Substitution
	*2.14	--PIP	Replacement
		Q.E.D.	

This proof requires that *2.13 be previously proved or given as true.

[†] Chaining is not given as a rule in Whitehead and Russell, but is provable from the axioms. Other such methods might be developed, but chaining is the only one included in LT.

III. REPRESENTATION OF "THE PROBLEM" IN THE IPL-V MACHINE

Expressions, variables, and connectives are represented by IPL-V data list structures. An expression is recursively defined as a single variable or a list of subexpressions with a connective in its head. For example, Axiom *1.5 [AV[BVC]]I[BV[AVC]], looks like this:

Total Expression	{	*15	9-0	Description list
			9-1	0 Main expression
	{	9-0	0	
			Q15	Attr. "tree form"
Description list	{		Q15	
			Q7	Attr. "external name"
			9-2	0 Value is data term
		9-2	21*1.5	containing text
	{	9-1	I0	Main connective
Main Expression	{		9-3	Left subexpression
			9-4	0 Right subexpression
	{	9-3	V0	Connective OR
Left Expression	{		A0	Variable A
			9-5	0
	{	9-4	V0	
Right Expression	{		B0	Variable B
			9-6	0
	{	9-5	V0	
Right of left	{		B0	
			C0	0
	{	9-6	V0	
Right of Right	{		A0	
			C0	0 Variable C

Variables and connectives look like this:

Connective	I0	9-0	0
IMPLIES	9-0	0	
Attr. "type"		Q14	
Value "non-unary"		J4	
		Q7	
		9-1	0
	9-1	21I	
Connective OR	V0	9-0	0
	9-0	0	
		Q14	
		J4	
		Q7	
		9-1	0
	9-1	21V	
Free Variable "A"	A0	9-0	0
	9-0	0	
Attr. "Variable"		Q5	
		Q5	
Attr. "Free Var."		Q6	
		Q6	
		Q7	
		9-1	0
	9-1	21A	
Free Variable "B"	B0	9-0	0
	9-0	0	
		Q5	
		Q5	
		Q6	
		Q6	
		Q7	
		9-1	0
	9-1	21B	
Free Variable "C"	C0	9-0	0
	9-0	Q5	
		Q5	
		Q6	
		Q6	
		Q7	
		9-1	0
	9-1	21C	

The rules of inference are embodied in routines called methods:

M11	Detachment
M12	Substitution
M13	Replacement
M14	Forward Chaining
M15	Backward Chaining
M16	Subexpression Replacement I
M17	Subexpression Replacement II

Methods are applied to problems (unproved expressions) by the executive routines:

M1	Single-Problem Executive
M2	Multiple-Problem Executive
M7	Apply Methods (1) to Problem (0)
M8	Create a List of Methods for (0)

Application of methods to a problem results in symbol manipulation on the data list structures representing the problem and will, hopefully, result in finding a proof.

IV. HOW LT WORKS

In LT, the single problem executive uses the methods to find proofs for given problems. The methods are based on rules of inference set forth in Principia Mathematica.⁽⁵⁾

The substitution method tries to directly prove a given problem expression by matching it to an axiom expression or a previously proved theorem expression. The matching procedure tries to effect identity between the two expressions with an appropriate series of substitutions for free variables. If the match succeeds, a proof has been found.

The other methods do not try to find proofs directly. Instead, they try to construct subproblem expressions to serve as surrogates for the given problem. By construction, each new surrogate subproblem, when proved, will imply proof of the given problem from which it was developed. Substitution is immediately tried on each new subproblem in the hope that a proof is at hand.

The replacement methods try to develop subproblem expressions by replacing logical connectives as specified by the definitions. For example:

"Definition [AIB].=[-AVB] and problem
-PVP yield subproblem PIP."

PIP and -PVP are the same assertion in alternate forms as specified by definition *1.01.

The method of detachment is based on the rule of detachment (*1.11), and as in replacement, it tries to develop a surrogate for the problem expression. The rule of detachment:

"True expressions AIB and A yield new
true expression B"

is used in a backward sense:

"True expression AIB and problem B yield
subproblem A"

so that proof of the subproblem will imply proof of the
problem from which it was derived. The substitution
method is applied to each new subproblem immediately
after it is developed.

Chaining is not set forth as a rule of inference,
but its legitimacy as a method is provable from Axiom *1.5
or Theorems *2.05 and *2.06 by detachment. To be
strictly legal, chaining should not be used unless *2.05
and *2.06 are in the set of true expressions. Appropriately,
LT can prove all theorems through *2.06 without use of
chaining.

The methods of chaining also produce subproblem
expressions by working backward. For example:

"Problem AIC and theorem AIB yield sub-
problem BIC."

In this example, forward chaining works backward to obtain
a new subproblem which, if proved, implies proof of the
given problem. Backward chaining works backward in a
similar manner:

"Problem AIC and theorem BIC yield sub-
problem AIB."

The main heuristic[†] in LT is this procedure of working backward. It is easy to see that the methods could work forward using only true expressions to develop more true expressions, testing each new one to see if it proves the given problem. This procedure would make LT's behavior independent of the given problem, up to the time of proof completion, and it would not perform any better than the British Museum Algorithm.⁽³⁾

Working backward gives LT some vital sense of direction by taking advantage of the "heuristic connection"[‡] of its problem space. If LT is viewed as a trial solution generator with a solution tester, it is easy to see that the generator should have some sense of how to produce good trial solutions. LT's methods get this necessary sense of direction by working backward.

Substitution is the only method that finds proofs directly; thus, it serves as LT's trial solution tester. It is immediately applied to given problems and is applied as a subprocess to new subproblems as they are developed by the other methods.

The other methods serve as trial solution generators. New subproblems that do not lead directly to a proof are set aside in the untried subproblem list to be selected later for additional application of the other methods.

[†] By heuristic, we mean, "Any principle or device that contributes to the reduction in the average search to solution." (9)

[‡] For an excellent discussion of "heuristic connection," see Marvin Minsky on, "The Problem of Search." (10)

Selection of problems and application of methods is controlled by the single-problem executive routine, M1 (Fig. 1). This routine applies substitution first to avoid wasting effort on a directly provable problem. If substitution fails, the other methods are used to build trial proof sequences by developing subproblems. The method of derivation, the true expression used, and the problem from whence it came, are associated with each new subproblem.

The collection of subproblems develops into a tree of hypothesized proof sequences in which any proved subproblem constitutes proof of the given problem and of all intervening subproblems. Since all new subproblems have been through the substitution method, only subproblems at the outer reaches of the tree are candidates for further effort. These problems are kept on the untried problem list. M1 takes subproblems from the untried problems list, while the methods add new subproblems to it. Figure 2 shows an example of a subproblem tree.

It is interesting to note that while M1 works iteratively, the proof sequence tree grows recursively. This occurs because the context (derivation information) of each subproblem is directly associated with the subproblem itself, while the names of worthwhile candidates for additional effort are kept separately on the untried problems list. This arrangement allows the problem executive to work on whatever part of the tree it decides is most profitable looking.

If M1 worked recursively, LT would attack subproblems in the order of their development, which would involve a

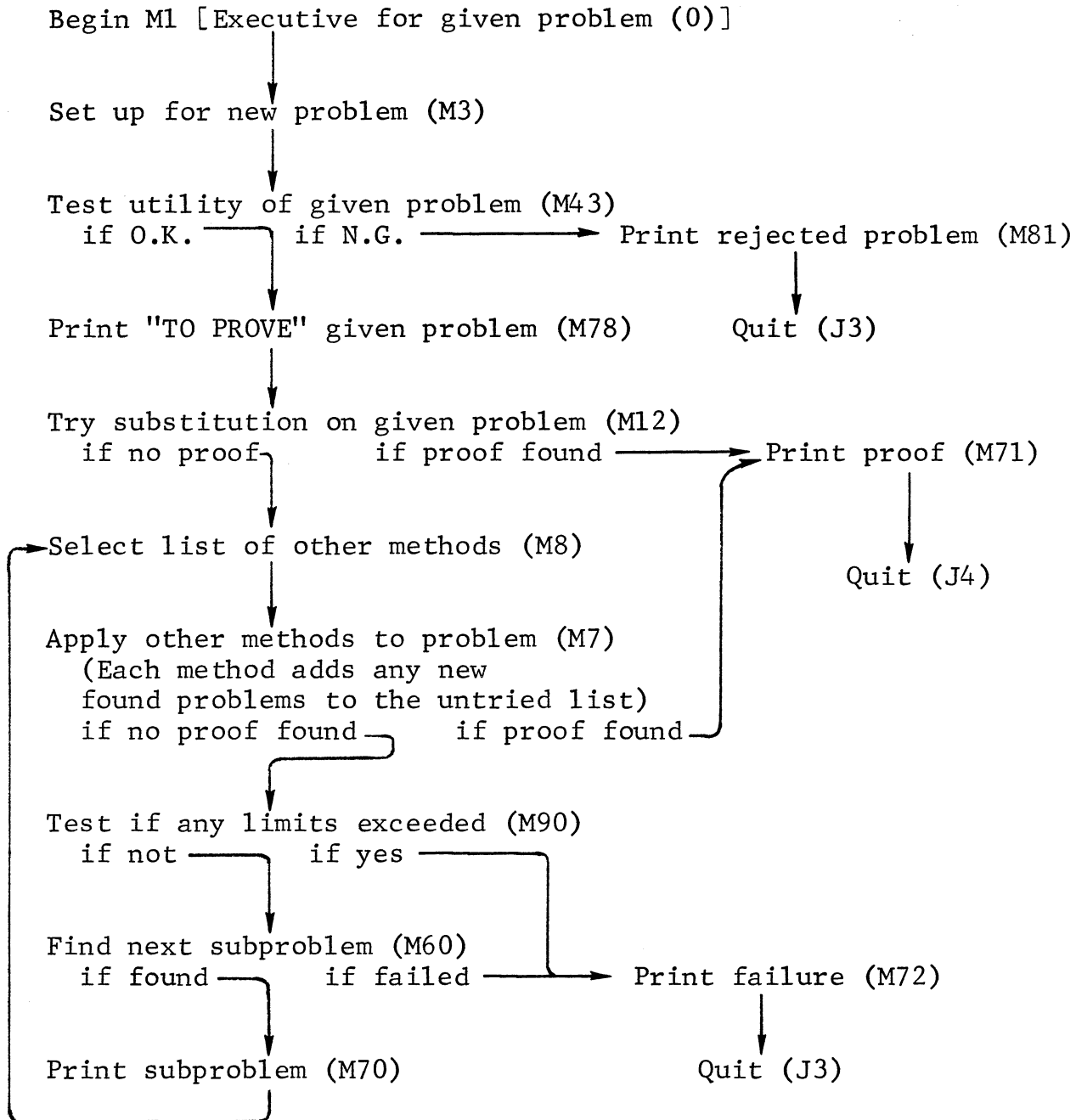


Fig. 1--The Single-Problem Executive

TO PROVE:

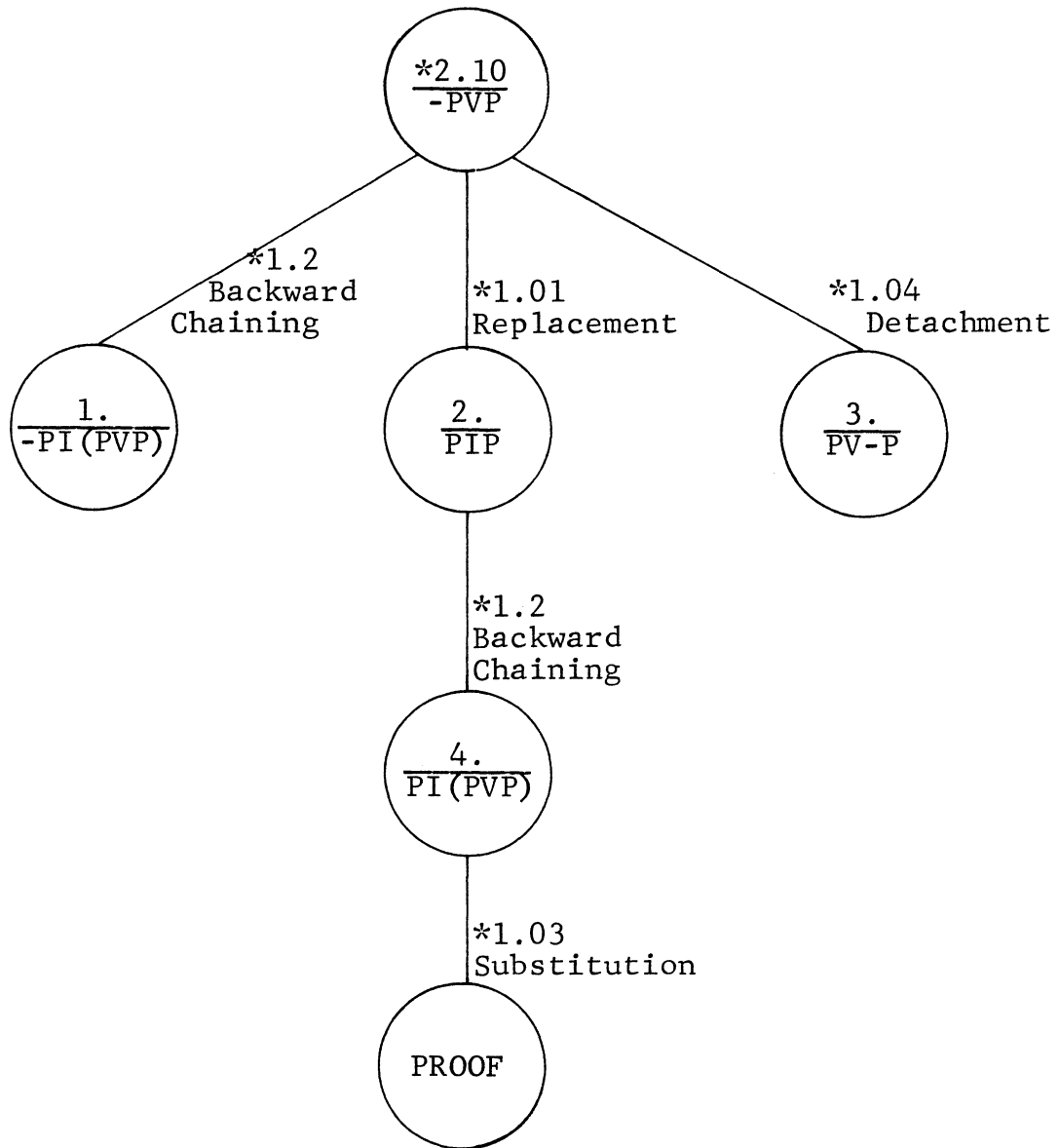


Fig. 2--Sample Subproblem Tree

depth-first attack on the problem, and make it difficult, if not impossible, to temporarily shift attention to more interesting parts of the tree. The subproblem tree in some sense represents what is known about the problem and it seems reasonable that good problem-solving procedures will require some ability to take an overall look at "what can be done" in order to plan future activity.[†]

The procedures embodied in M1 raise a number of questions for which good answers are hard to find. For example, it is unreasonable for each method to try all available theorems, since most of them won't yield any progress. It is easy to answer that a good selection process is needed, but this is not very precise.

This problem is closely connected to the relevancy question in information retrieval research. A more precisely stated answer might be, "Design a theorem storage and retrieval system that will deliver the names of theorems, appropriate parts of which are feasible matches for a given expression." The word "appropriate" takes on a specific meaning for each method using the system.

There are many ways to implement such a system. An interesting one is implemented around routines M54 and M63 of LT. It is discussed in Sec. IX.

There are other interesting questions raised by M1, as enumerated in the following list:

- 1) What is meant by utility and how can it be measured?
- 2) How should substitution be implemented?

[†] See Refs. 11 and 12 for recent discussions of the implications of questions raised here.

- 3) How should the other methods be implemented?
- 4) How are methods to be selected?
- 5) What theorems should each method try?
- 6) How many theorems should each method try?
- 7) What kind of effort limits are meaningful?
Useful?
- 8) How should the next subproblem be selected?

The remaining sections describe ways in which these questions have been resolved in the current version of LT. They are not the only or the best ways but they do enable LT to do a passable job of theorem proving.

V. SUBSTITUTION AND MATCHING

Matching is the heart of LT. The substitution method, M12 (Fig. 3), uses test for match routine M114 (Fig. 4) to try to effect identity between a problem and a theorem. M114 uses substitution for free variables as appropriate.

It may seem confusing, but substitution method M12 does not actually do any substitution. It only acts as an executive for matching a given problem with a sequence of appropriate theorems until a match is found or the theorem supply is exhausted. Determination of required substitutions is made by the match process.

Test for match routine, M114, is also an executive. It "puts off the work" to match process M111 (Fig. 5). M114's only reason for existence is to discard the output of M111 if a match is successful. Remember, TEST routines should leave no outputs in H0.

M111 does its matching with subprocess 9-100 (Fig. 6) after setting up an empty substitutions list. Since an expression is recursively defined in LT, the match subprocess is recursive. It tests for identity of variables and connectives between expressions, arranges effective substitution for free variables as required, and recursively matches corresponding subexpressions.

Required substitutions are effected through use of the substitution list. The match process adds new required substitutions to the list as they are discovered and looks up previously assigned substitutions when free variables are encountered in expressions. The substitution

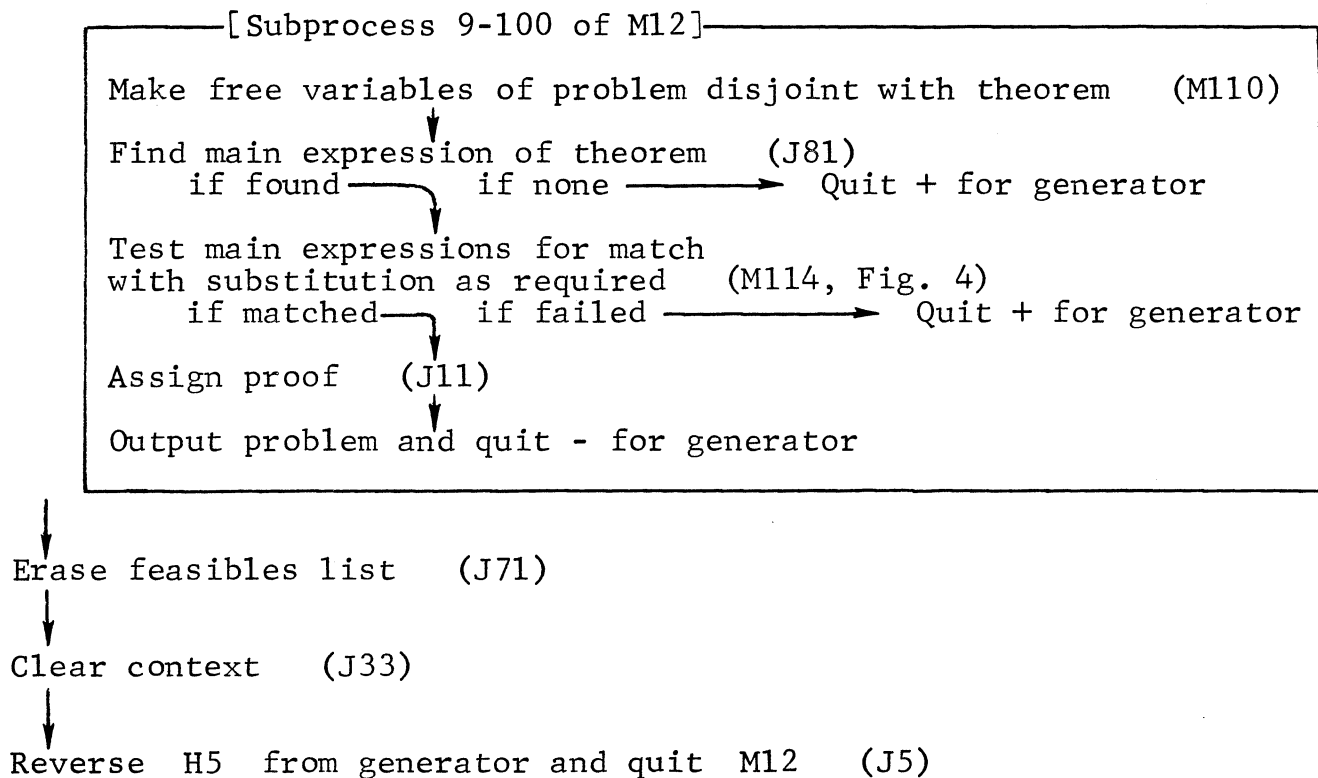
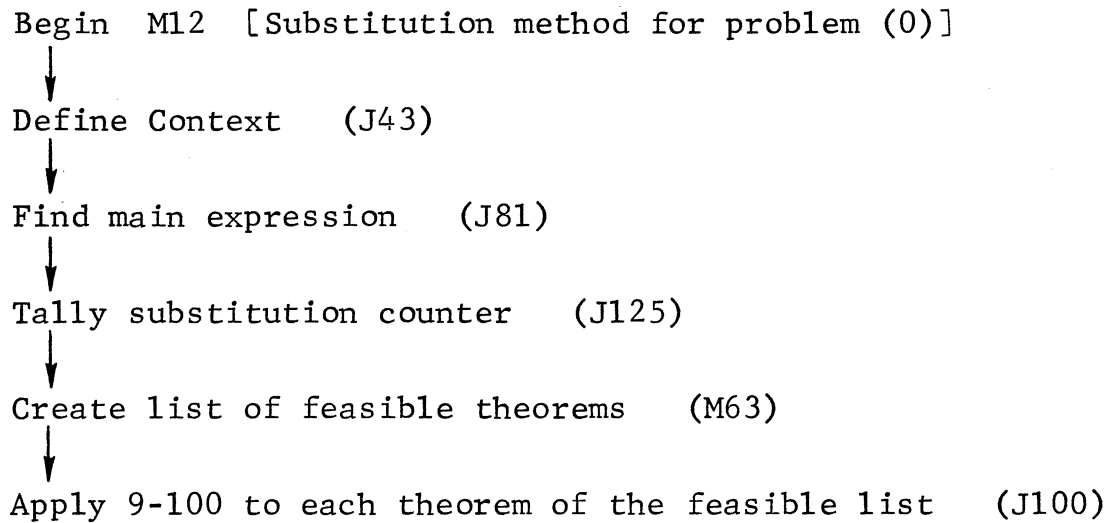


Fig. 3--The Substitution Method

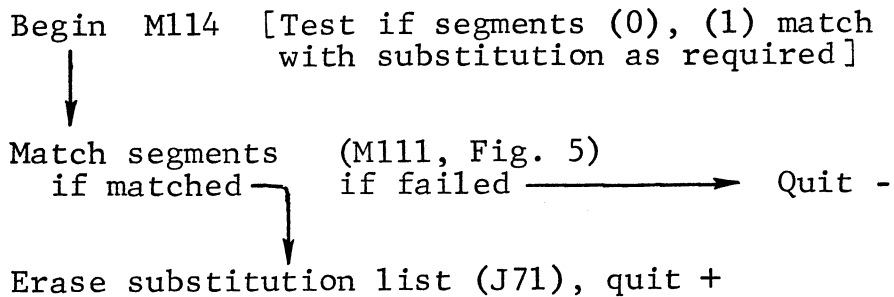


Fig. 4--Test for Match with Substitution

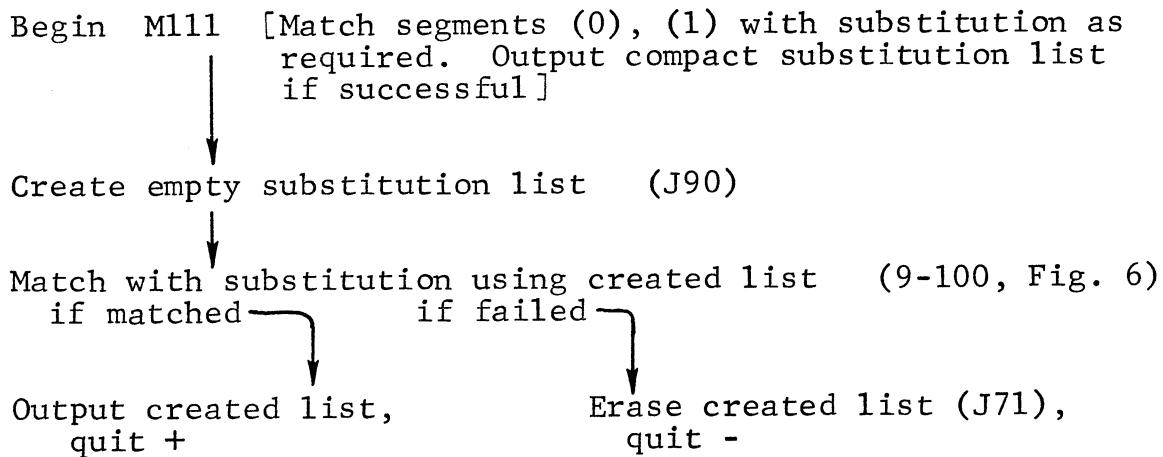


Fig. 5--The Match Process

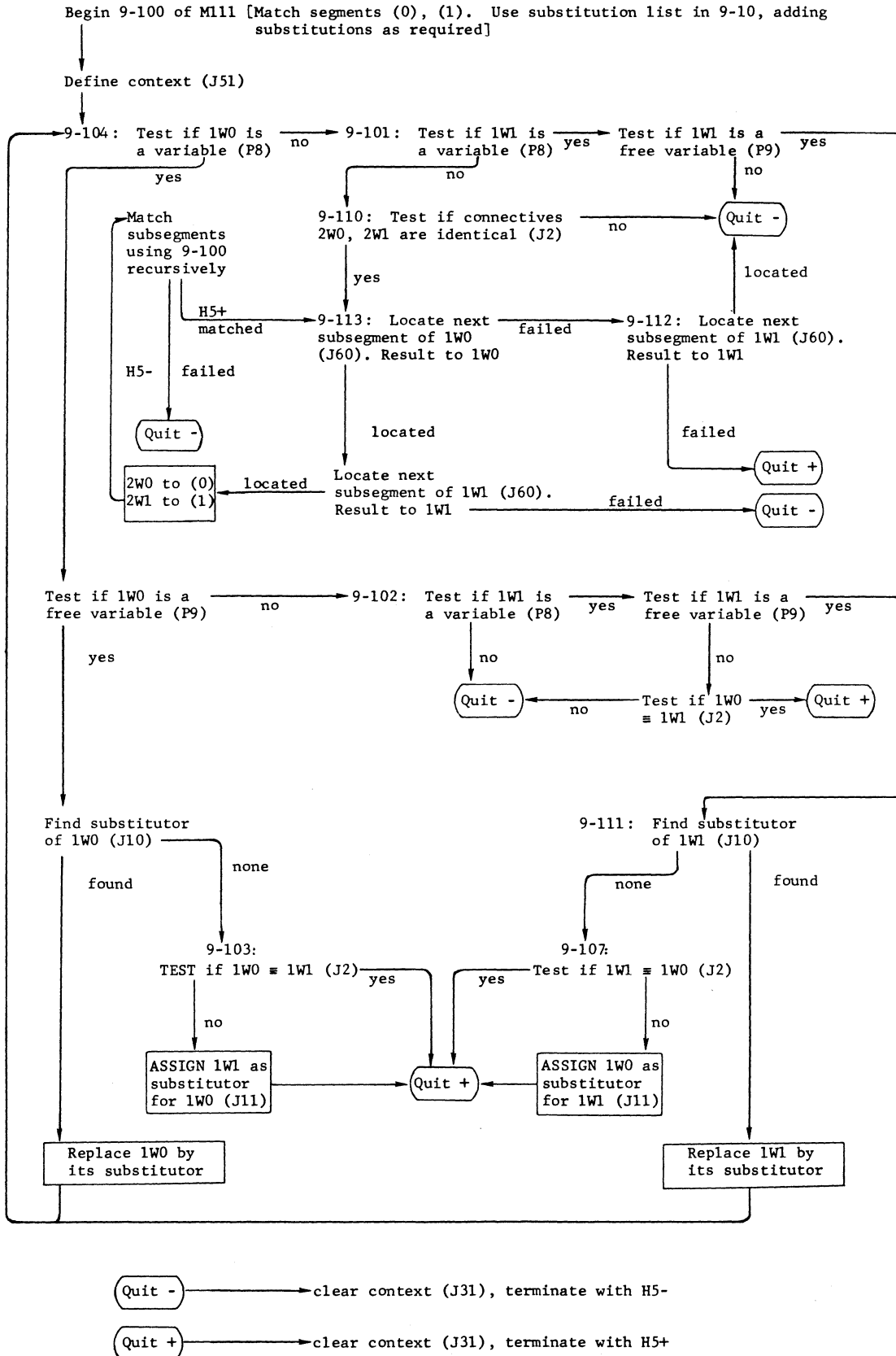


Fig. 6--The Match Subprocess

list is a description list with free variables for attributes and assigned substitutions as values. Because the values are often subsegments of the actual expressions being matched and because they, in turn, may require substitutions within themselves, the substitution list output by M111 is called a compact substitution list.

Figure 6 shows how M111 uses the compact substitution list to effect identity between expressions. The IPL-V symbols in Fig. 6 are taken from the code listed in Sec. XV.

A compact substitution list is not suitable for actual substitution in newly constructed expressions as required by other users of the match process. An expanded form can be obtained by matching with M113 (Fig. 7) which applies M112 (Figs. 8-11) to expand the compact list from M111. The expanded list contains completely substituted locally named copies of the parts and pieces of original expressions that made up the original compact list.

M112 expands the list in one pass by replacing each assigned substitution by its properly substituted locally named copy, using the substitution list itself to look up required substitutions as it goes. The subprocess that constructs properly substituted copies for replacement works recursively and is called delineation (9-100 and 9-200 of M112, Figs. 9, 10). 9-300 of M112 (Fig. 11) is used to replace free variables in expressions.

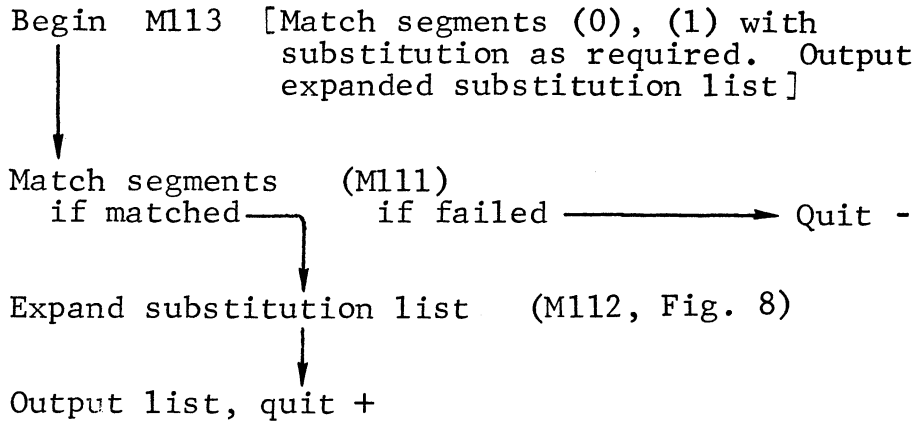


Fig. 7--Match with Substitution and Output Expanded Substitution List

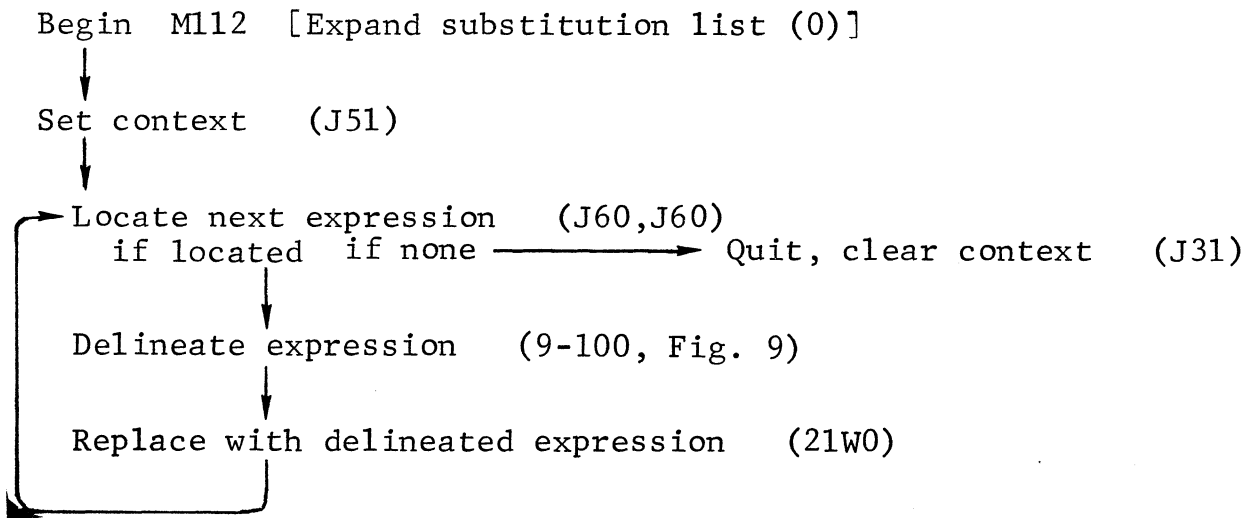


Fig. 8--Expand Substitution List

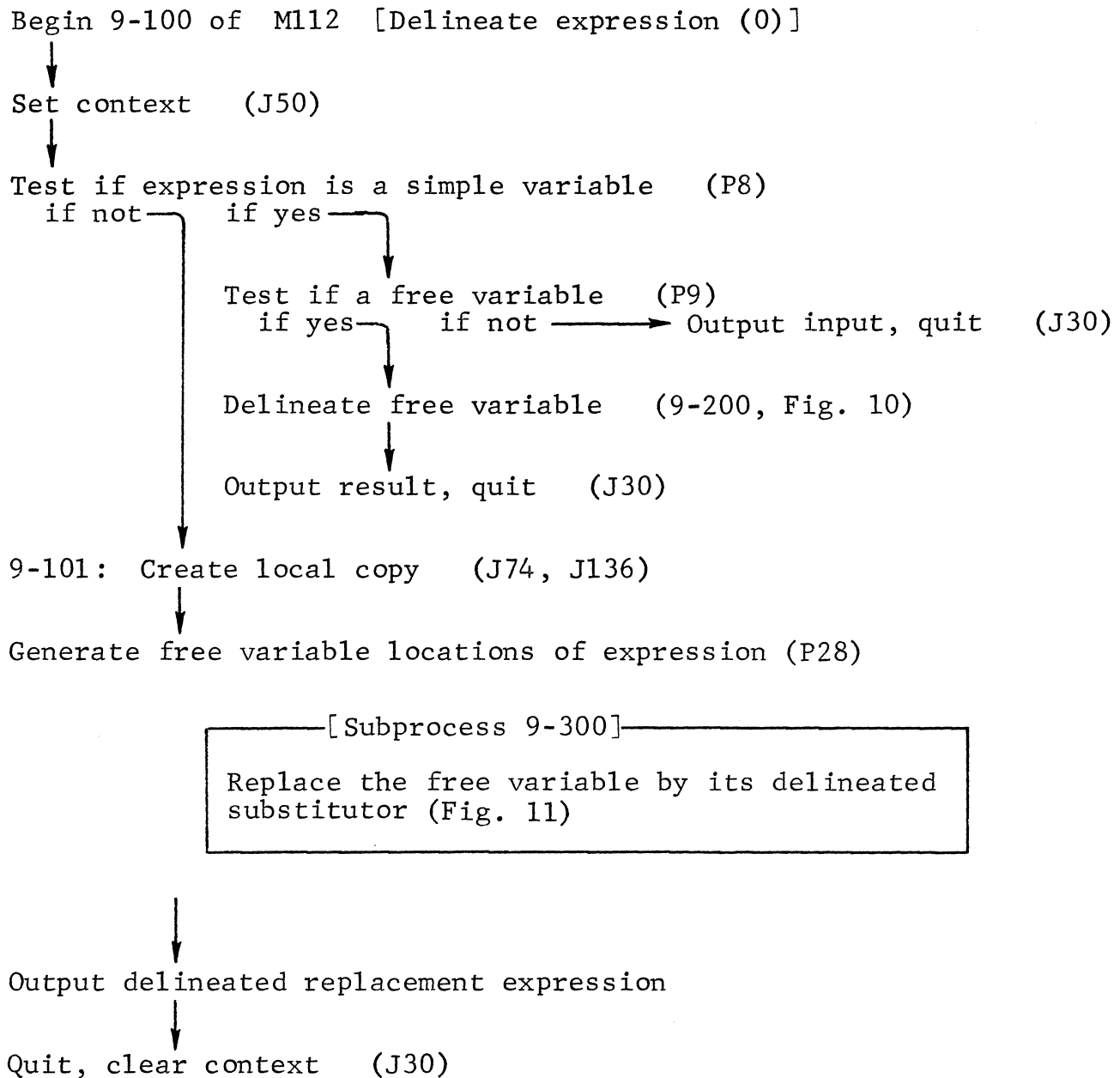


Fig. 9--Delineate Expression

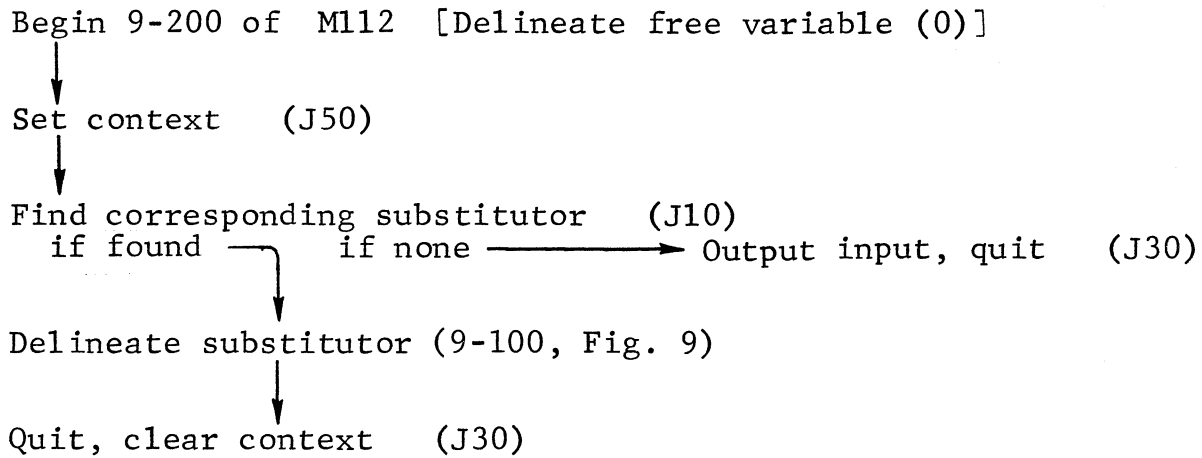


Fig. 10--Delineate Free Variable

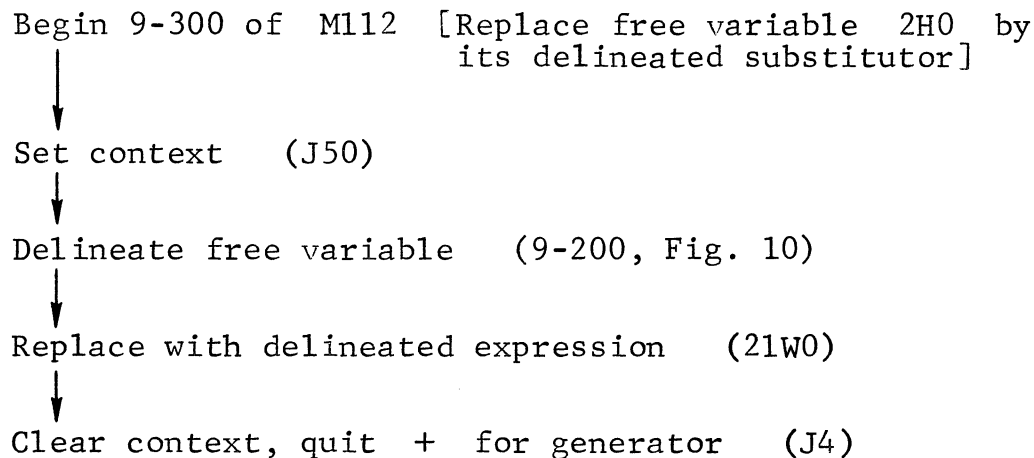


Fig. 11--Replace Free Variable by Delineated Substitutor

VI. THE OTHER METHODS

The methods, other than M12, do not detect proofs directly. It is the purpose of the other methods to develop new subproblems, which, if they can be proved, will imply proof of the given problem.

In the detachment method (M11, Figs. 12, 13) this is done by matching the whole problem to the right sides of a sequence of theorems whose main connectives are IMPLIES. For each successful match, a new subproblem is constructed by copying the left side of the theorem and substituting into the copy from the expanded substitution list obtained from the successful match.

Problem -PVP and axiom $[AVB]I[BVA]$
yield subproblem PV-P.

Replacement method M13 works the same way except that it can match expressions to either side of definitions (expressions with main connective $.=.$).

Problem PIP and definition $[AIB].=[-AVB]$
yield subproblem -PVP.

Problem -PVP and definition $[AIB].=[-AVB]$
yield subproblem PIP.

The sublevel replacement method (M16, Figs. 14-16) matches problem subsegments to definitions, proceeding through a problem expression one level at a time. At each level, each subsegment is tried for replacement. A new subproblem is formed if one or more subsegments are replaced at a given level.

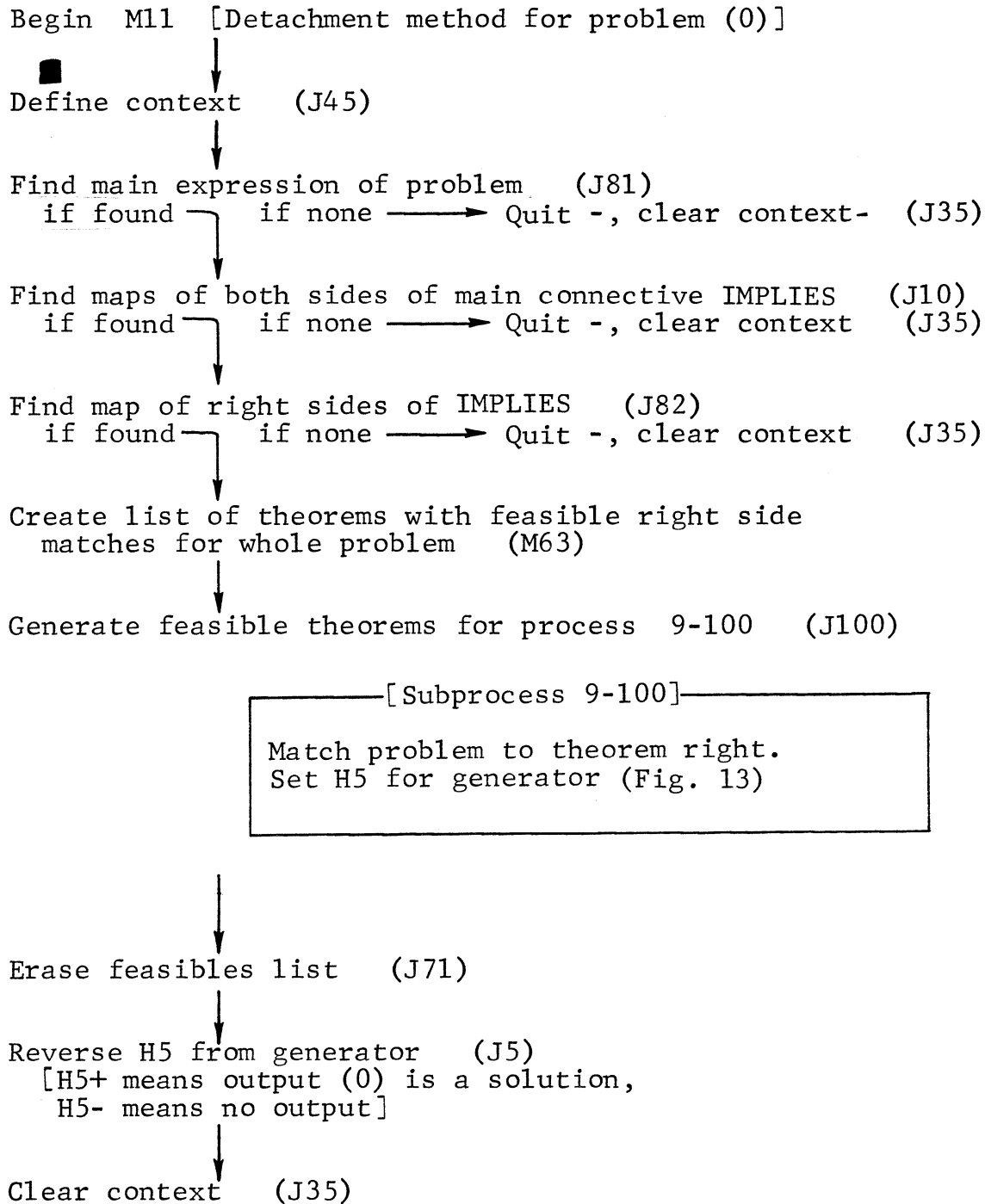


Fig. 12--Detachment Method

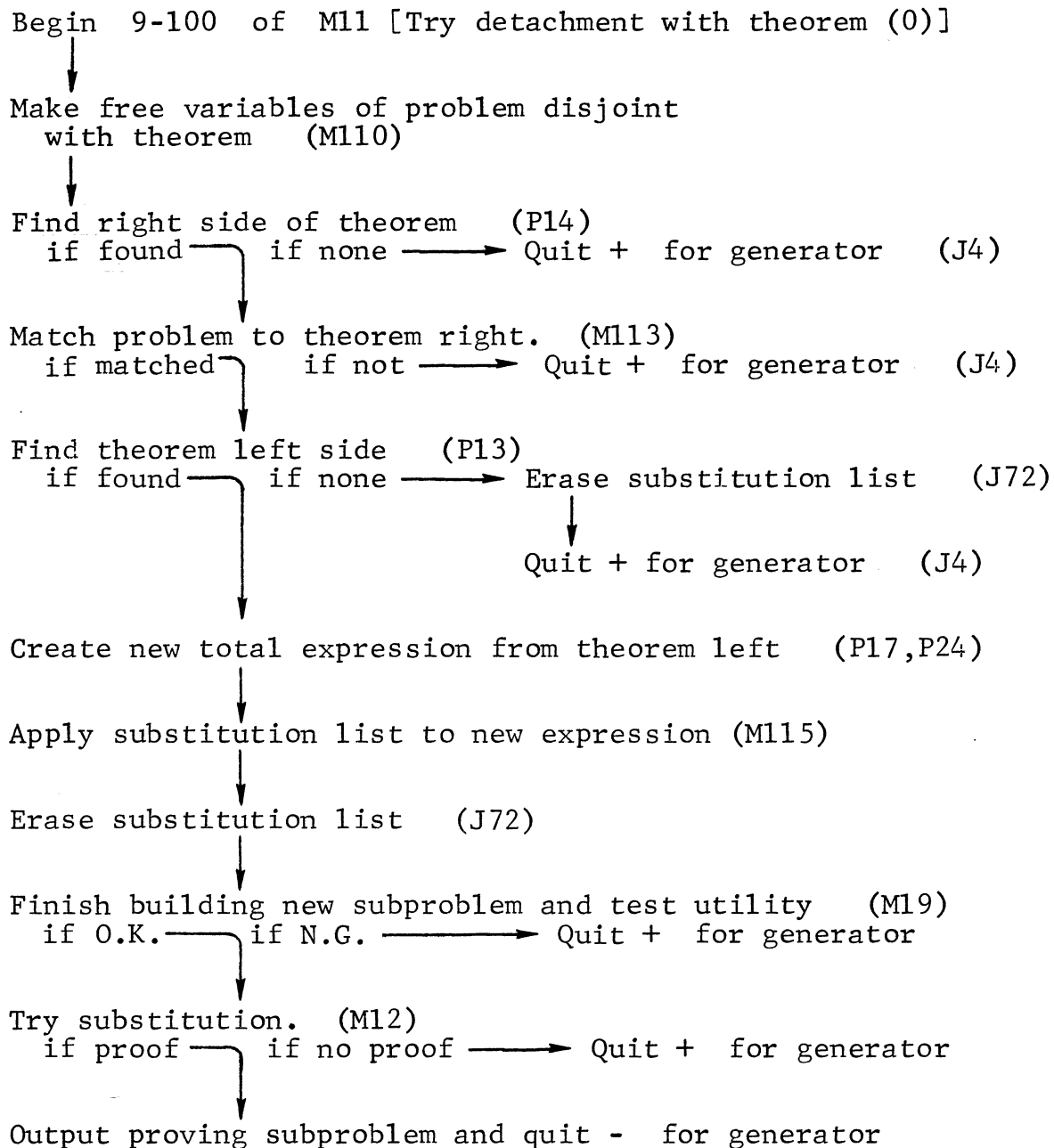


Fig. 13--Detachment Method Subprocess

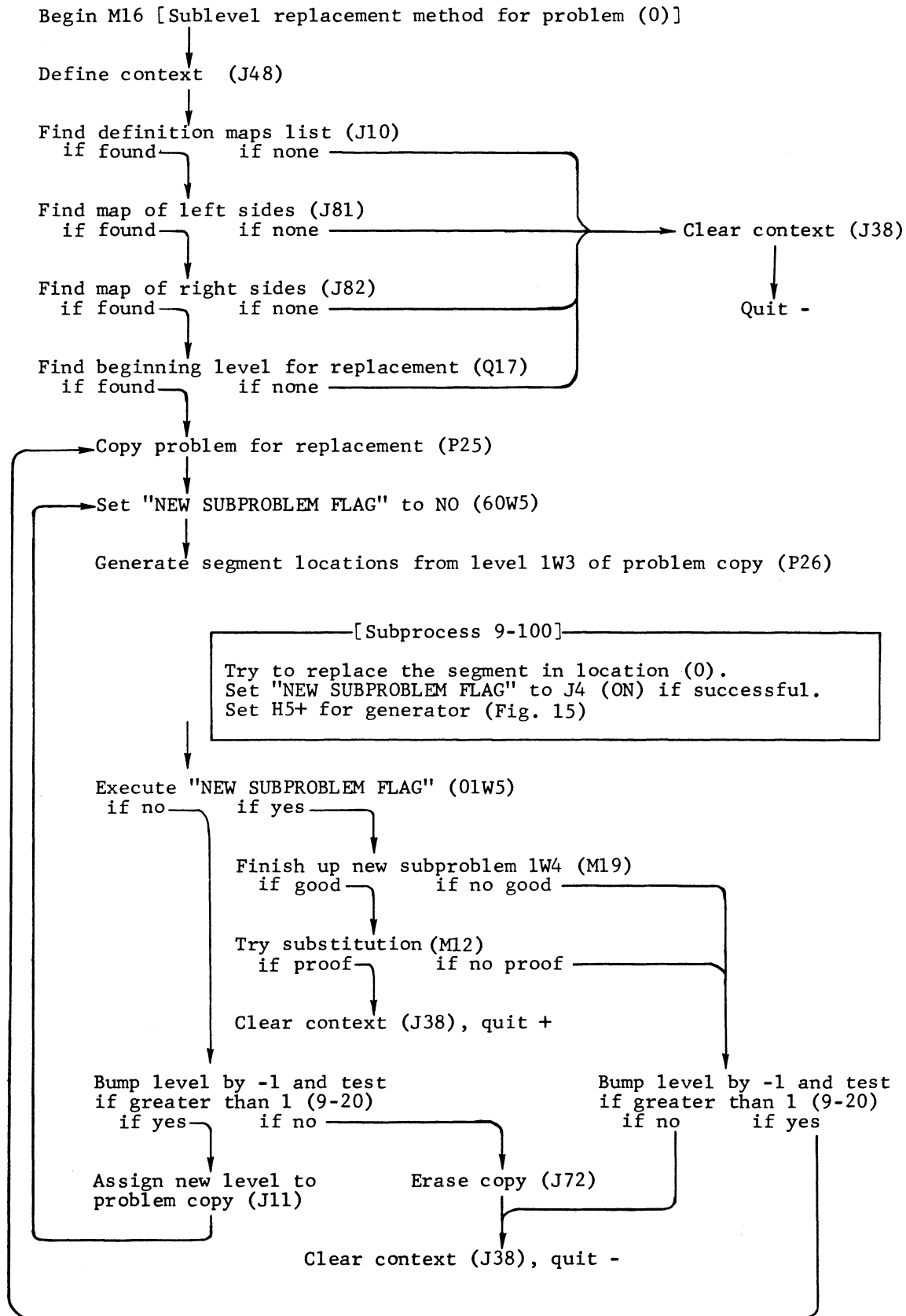


Fig. 14--Sublevel Replacement Method I

Begin subprocess 9-100 of M16
[Try to replace the segment in location (0)]
↓
Create a list of definitions with feasible left side
matches to segment (M62)
↓
Generate feasible definitions (J100)

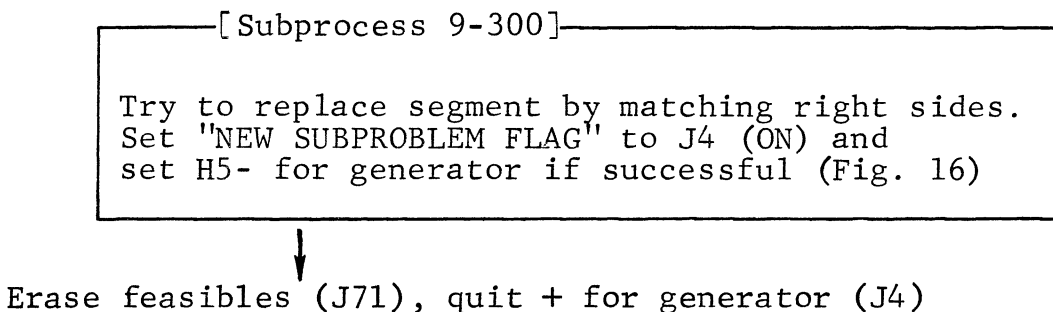
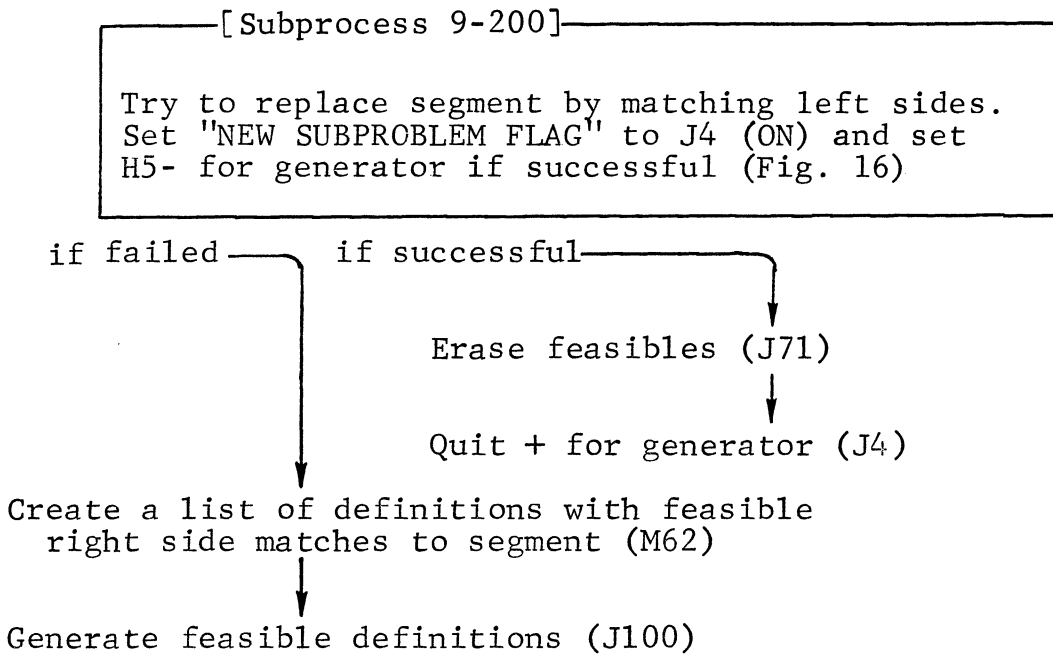


Fig. 15--Try to Replace Located Segment

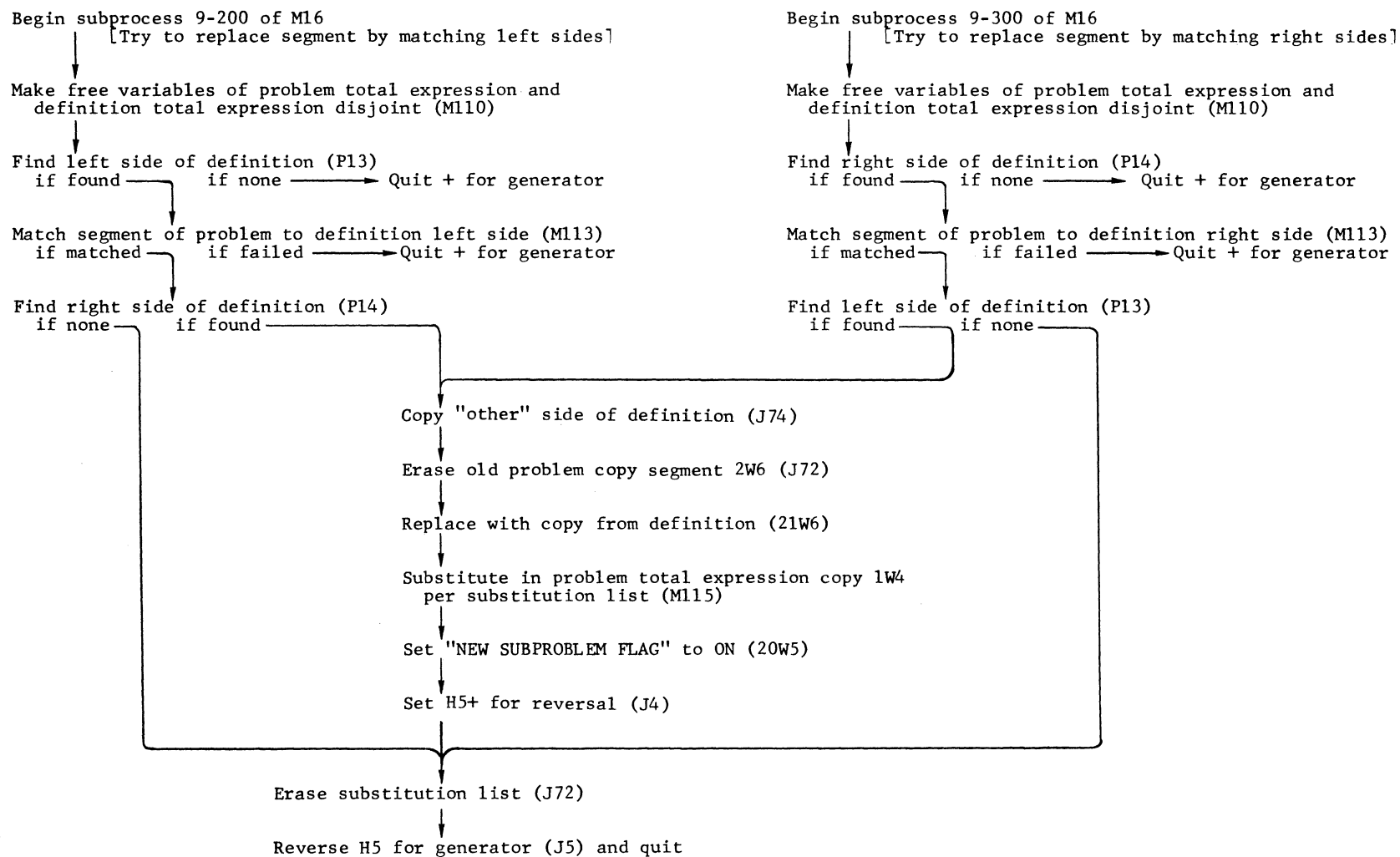


Fig. 16--Try to Replace Segment by Matching Left (Right) Sides

Problem $[PI-P]I-P$ and definition $[AIB].=[-AVB]$
yield subproblem $[-PV-P]I-P$.

Sublevel replacement method M17 is identical to M16 except that it tries to replace all subsegments starting with those at the lowest level of the problem expression.

The forward chaining method matches the left side of the problem with left sides of a sequence of appropriate theorems. In the event of a match, a new problem is constructed with copies of the right side of the problem and the right side of the theorem. The new subproblem is then substituted into according to the substitution list obtained from M113.

Problem $PV---P$ and theorem $AV-A$ yield
new subproblem $-PI---P$.

Backward chaining is the same except that it matches right sides and uses the left sides to construct new subproblems.

Problem $PI[PVQ]$ and theorem $[AVB]I[BVA]$
yield new subproblem $PI[QVP]$.

Each new subproblem produced by a method needs further processing after appropriate substitution for free variables. Since this is the same for all methods, a separate routine, M19 (Fig. 17), was designed to finish off new subproblems.

M19 connects a new problem into the subproblem tree by assigning values to its derivation attributes. Then, after testing utility, it either quits with H5+ or erases the bad problem and quits with H5-. The utility measures used are discussed in the next section.

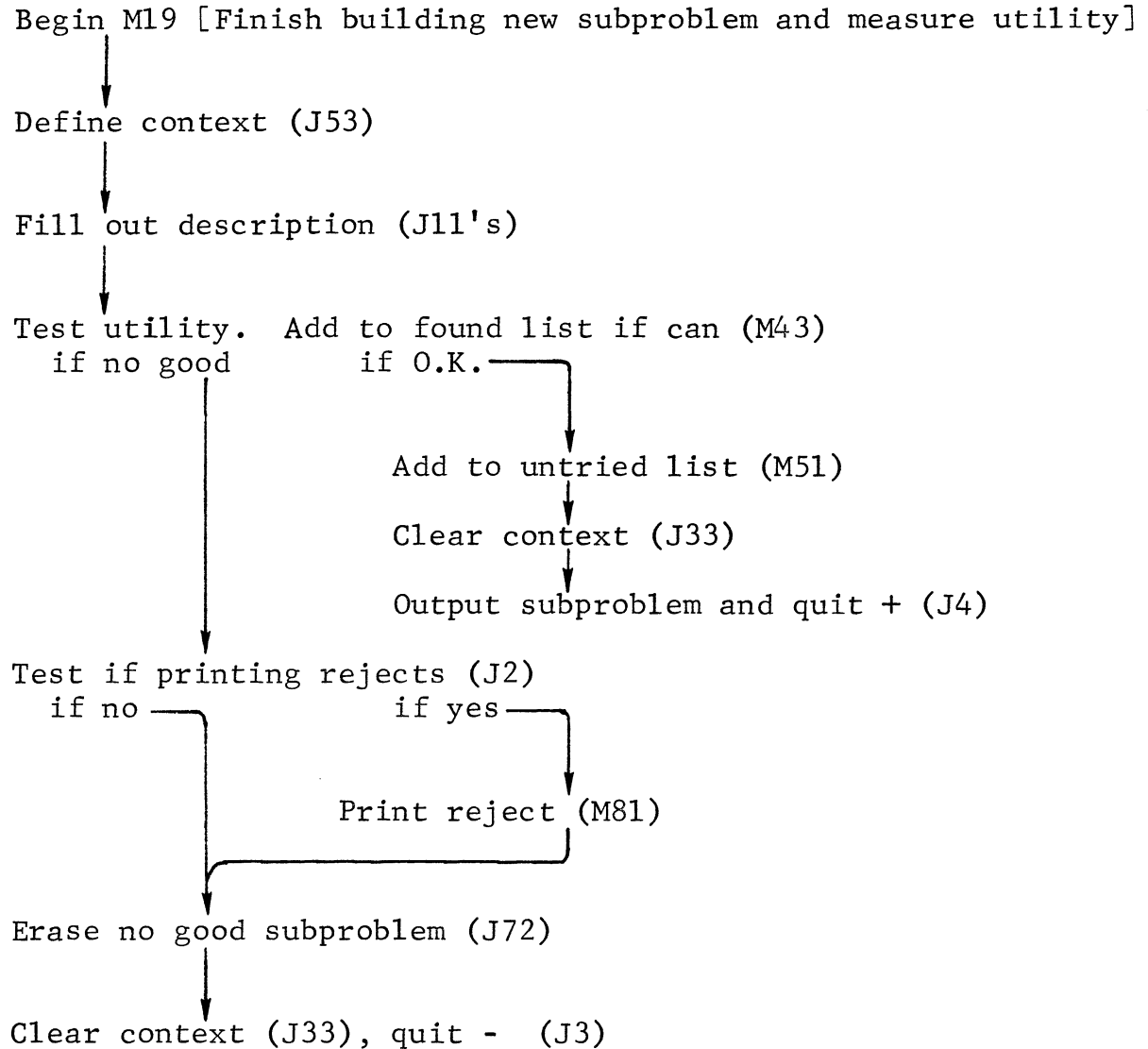


Fig. 17--Finish Building New Subproblem

The last thing each method does to each "good" subproblem is to try substitution in the hope that a proof is at hand.

VII. UTILITY MEASURES

LT considers a problem to have sufficient utility if it has not been previously found and is not clearly unprovable. The newness test adds a new problem to the found problems list if it doesn't match any problems already on the list. The non-provability test rejects single variables (P , $\neg P$), problems with matching sub-segments of main connective OR (PVP , $[PIQ]V[PIQ]$), and problems with faulty structures. Routine M43 (Fig. 18) handles utility measurements.

The test for match across OR uses M114 which uses substitution. The test for newness, M42 (Fig. 19), uses M40 (Fig. 20) which does not match with substitution but does consider any pair of free variables in corresponding positions to be matched.

In order to reduce search time in M42, the found problems list is kept in a structured form. The structure is based on the number of levels ($Q2$), number of distinct variables ($Q3$), and the number of variable places ($Q4$) in the problems stored. Only problems with identical values for $Q2$, $Q3$, and $Q4$ are put through a full match test (M40).

Other more interesting and profitable organization techniques might be developed as student exercises. For example, the found problems might be kept on a map similar to that used for true expressions. This might lead to new ways to organize the untried problems and facilitate new and interesting procedures for the single and multiple problem executives.

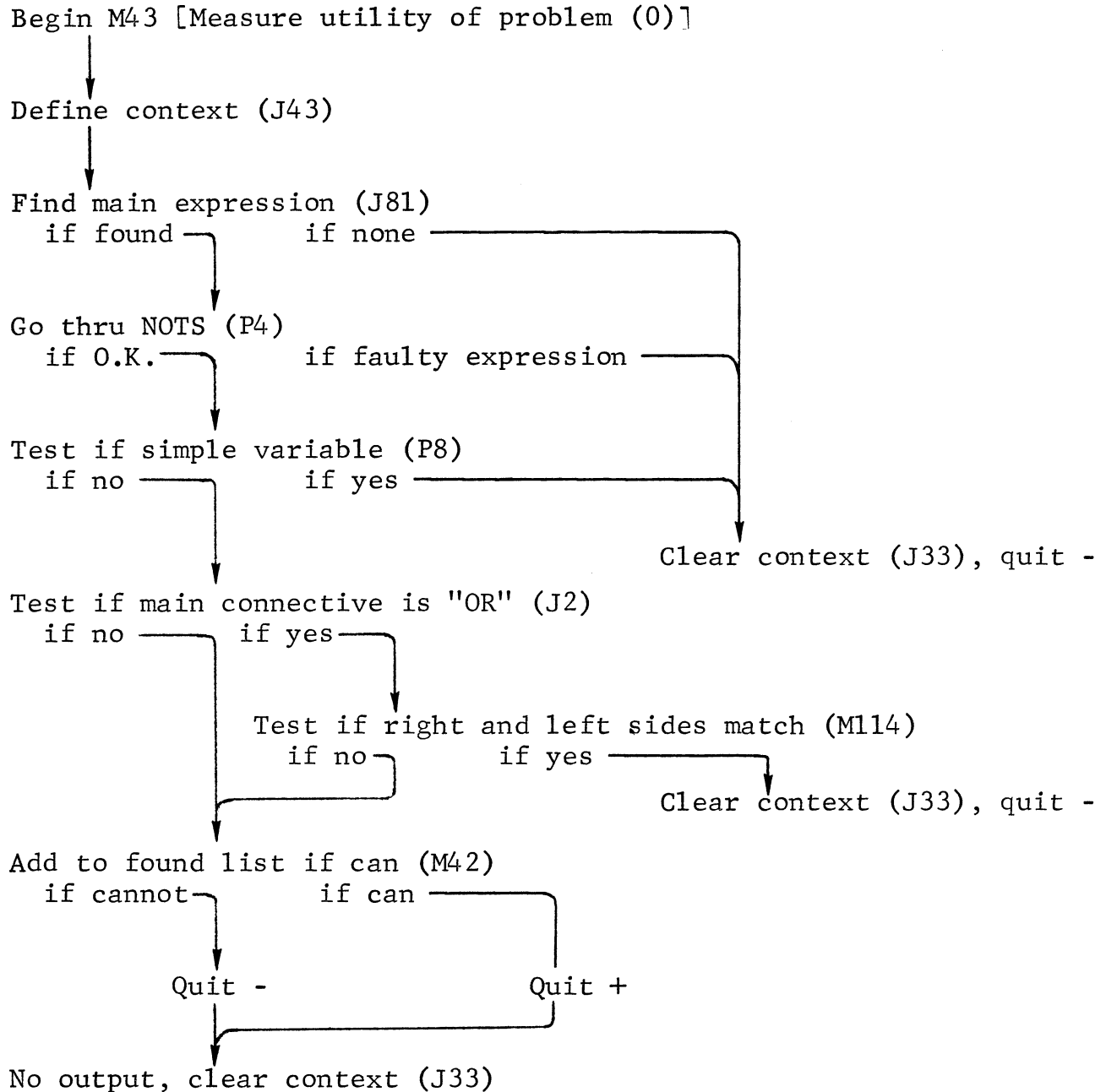


Fig. 18--Measure Utility

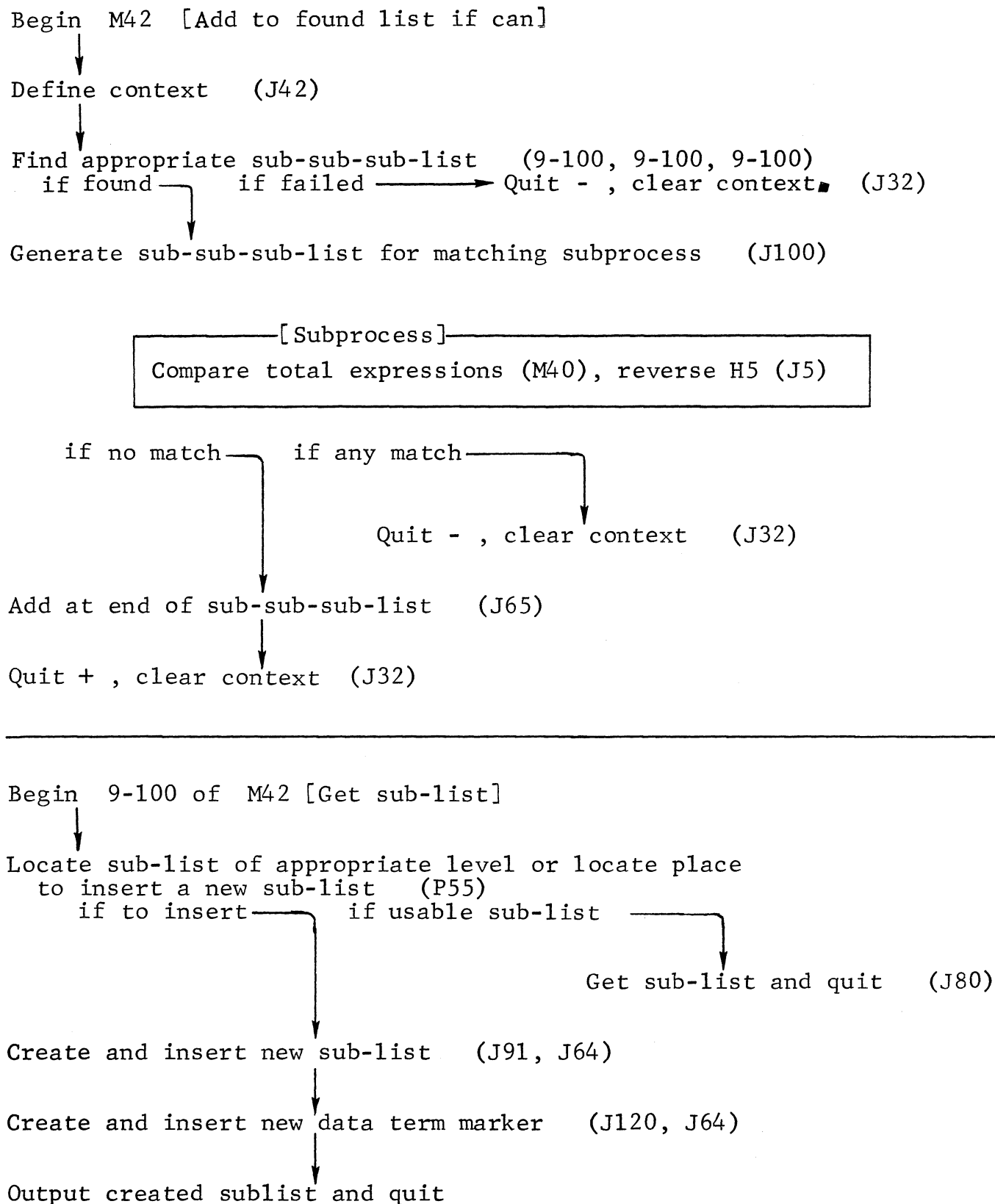


Fig. 19--Add to Found List

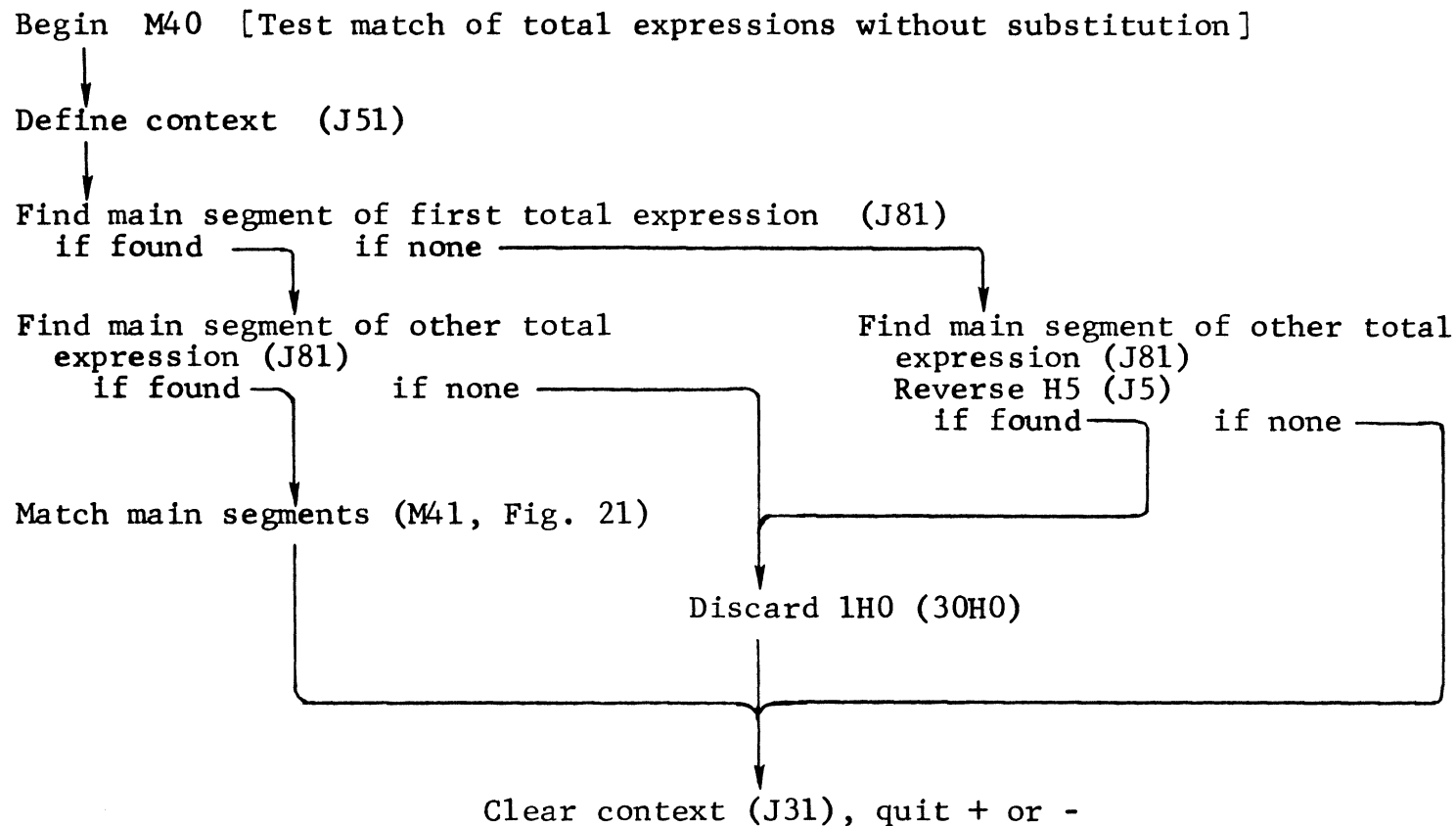


Fig. 20--Test Match of Total Expressions Without Substitution

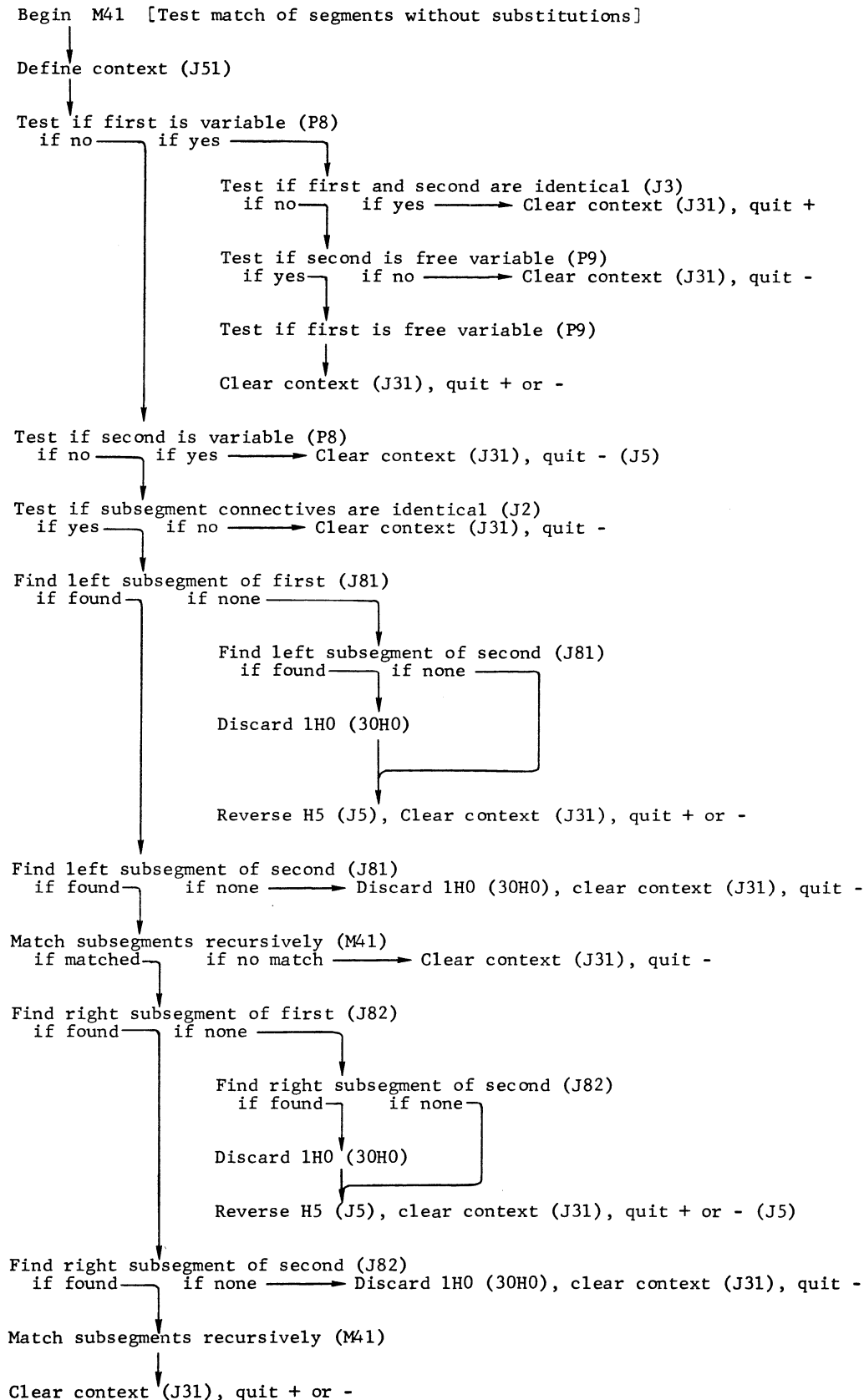


Fig. 21--Test Match of Segments Without Substitution

VIII. INFORMATION STORAGE AND RETRIEVAL

The original version of LT similarity-tested all available axioms and theorems for matching with each problem expression. The similarity test used the number of levels, number of distinct variables, and number of variable places in expressions to measure match feasibility. The purpose of the test was to obtain efficiency by screening out unlikely match candidates.

These measures of match feasibility were not very effective because of the extensive processing required and because of the global nature of the measurements.

Selection of true expressions for matching should take advantage of the fact that successful matching depends on similarity of local structural characteristics. Of course, any cheap elimination of useless true expressions is good, but difficulties arise when relatively scarce useful theorems are eliminated. Use of the original similarity test sometimes made problems unprovable because crucial matches were prevented, thus preventing development of crucial subproblems.

These and other considerations have led to development of better measures of match feasibility. Work involved with extending LT[†] to handle problems involving AND and EQUIVALENCE connectives led to the ideas that an expression is better characterized by its form than by simple counts of its various elements, and that the essence of a form is in its connective structure.

[†] Development of the true expressions map and extension of LT to handle AND and EQUIVALENCE connectives was done by the author as a term project under Fred Tonge, using the Western Data Processing Center's IBM 7090.

New requirements for match feasibility were developed as follows:

A true expression is considered to be a feasible match if it has the same connective structure as the given problem, viewing the problem as contracted at sub-segment places as required.

By "viewing as contracted," we mean viewing a segment as though it were a simple bound variable.[†] This corresponds to the M111 matching procedure of assigning segments as substitutors for free variables as required. For example, under the new feasibility requirement:

[PVQ]I[PV[PVQ]]

is a feasible match for Axioms

*1.2 [AVA]IA
*1.3 AI[BVA]
*1.4 [AVB]I[BVA] ;

but not for Axioms

*1.5 [AV[BVC]]I[BV[AVC]]
*1.6 [AIB]I[[CVA]I[CVB]] .

*1.5 fails because its left side has a segment place corresponding to a bound variable place. *1.6 fails because its subsegments have the wrong connectives.

Implementation of the new requirement was accomplished by mapping all true expressions onto one structure, called "The Map of All True Expressions" (L4). This map serves

[†]In LT, bound variables are treated as though they are segments of a particular but unknown form; thus, they can be substituted for free variables but nothing can be substituted for them.

as an index to the true expressions and facilitates selection of feasible matches without individual testing of each true expression.

The structure of the map is similar to that of expressions in that it forms a tree and has nodes corresponding to variable and segment places. Figure 22 shows the map structure when it contains Axioms *1.2 through *1.6. Main map L4 is an IPL-V description list with connectives as attributes and sublists of submaps as values. Each submap has the same form as the main map and each submap place corresponds to a variable or segment place in at least one true expression. The head of each submap holds a list of theorems, each of which has a variable place corresponding to the submap place. Figure 23 shows how the axiom map looks when keypunched.

By examining the map (Figs. 22, 23) it can be seen that Axiom *1.4 has a variable at the left of OR, which is at the left of main connective IMPLIES, etc. Main map L4 in Fig. 23 has only one attribute because all axioms have the same main connective.

Addition of definitions and newly proved theorems will cause additional attributes to appear in the main map. The head of L4 will always remain empty because no true expression can be without at least one connective. The map is constructed and searched by the routines discussed below.

M54 (Fig. 24) adds expressions to the map structure by mapping each new expression over those previously mapped, extending the structure as needed. Since the structure is a tree, M54 does its work with a recursive subprocess (9-100) which adds subsegments to submaps.

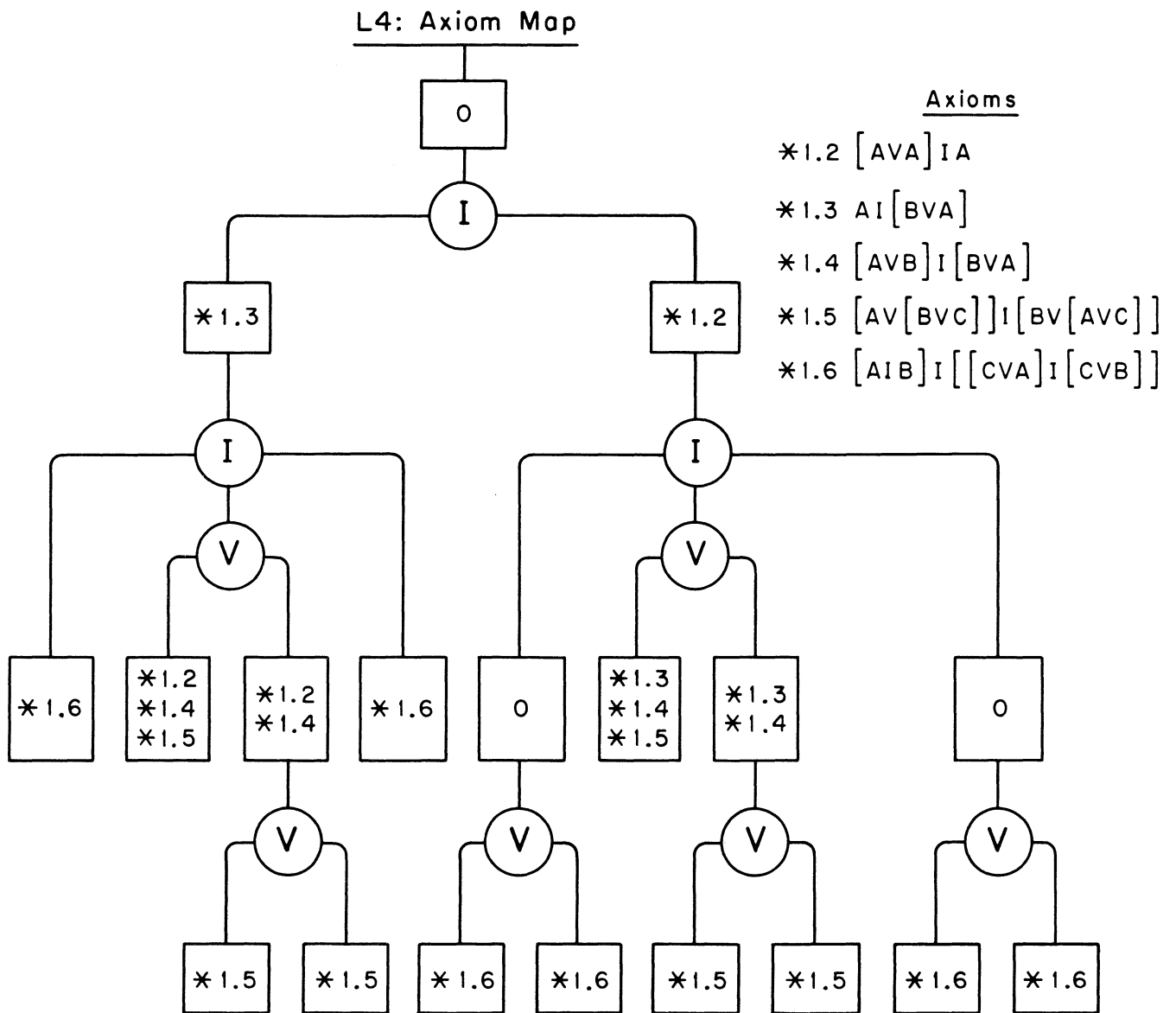


Fig. 22--Axiom Map

TRUE EXPRESSIONS MAP HOLDING AXIOMS	L4	0	
*1.2, *1.3, *1.4, *1.5, *1.6.		I0	
		9-1	0
MAPS LIST FOR MAIN CONNECTIVE I.	9-1	0	
		9-2	
		9-3	0
SUBMAP, LEFT SIDES OF I.	9-2	9-4	
		I0	
		9-5	
		V0	
		9-6	0
SUBMAP, RIGHT SIDES OF I.	9-3	9-7	
		I0	
		9-8	
		V0	
		9-9	0
AXIOM, VARIABLE ON LEFT OF I.	9-4	0	
		*13	0
MAPS LIST FOR I ON LEFT OF I.	9-5	0	
		9-10	
		9-11	0
MAPS LIST FOR V ON LEFT OF I.	9-6	0	
		9-12	
		9-13	0
AXIOM, VARIABLE ON RIGHT OF I.	9-7	0	
		*12	0
MAPS LIST FOR I ON RIGHT OF I.	9-8	0	
		9-14	
		9-15	0
MAPS LIST FOR V ON RIGHT OF I.	9-9	0	
		9-16	
		9-17	0
SUBMAP, LEFT OF I ON LEFT OF I.	9-10	9-18	0
SUBMAP, RIGHT OF I ON LEFT OF I.	9-11	9-19	0
SUBMAP, LEFT OF V ON LEFT OF I.	9-12	9-20	
		V0	
		9-21	0
SUBMAP, RIGHT OF V ON LEFT OF I.	9-13	9-22	0
SUBMAP, LEFT OF I ON RIGHT OF I.	9-14	0	
		V0	
		9-23	0
SUBMAP, RIGHT OF I ON RIGHT OF I.	9-15	0	
		V0	
		9-24	0
SUBMAP, LEFT OF V ON RIGHT OF I.	9-16	9-25	0
SUBMAP, RIGHT OF V ON RIGHT OF I.	9-17	9-26	
		V0	
		9-27	0
AXIOM, VAR. LEFT OF I, LEFT OF I.	9-18	0	
		*16	0
AXIOM, VAR. RIGHT OF I, LEFT OF I.	9-19	0	
		*16	0

Fig. 23--Axiom Map in Key punched Form

AXIOMS, VAR. LEFT OF V, LEFT OF I.	9-20	0	
		*14	
		*12	0
MAPS LIST, V LEFT OF V, LEFT OF I.	9-21	0	
		9-28	
		9-29	0
AXIOMS, VAR. RIGHT OF V, LEFT OF I.	9-22	0	
		*15	
		*14	
		*12	0
MAPS LIST, V LEFT OF V, RIGHT OF I.	9-23	0	
		9-30	
		9-31	0
MAPS LIST, V RT. OF I, RT. OF I.	9-24	0	
		9-32	
		9-33	
AXIOMS, VAR. LT. OF V, RT. OF I.	9-25	0	
		*15	
		*14	
		*13	0
AXIOMS, VAR. RT. OF V, RT. OF I.	9-26	0	
		*14	
		*13	0
MAPS LIST, V RT. OF V, RT. OF I.	9-27	0	
		9-34	
		9-35	0
MAP, LT. OF V, LT. OF V, LT. OF I.	9-28	9-36	0
MAP, RT. OF V, LT. OF V, LT. OF I.	9-29	9-37	0
MAP, LT. OF V, LT. OF V, RT. OF I.	9-30	9-38	0
MAP, RT. OF V, LT. OF V, RT. OF I.	9-31	9-39	0
MAP, LT. OF V, RT. OF I, RT. OF I.	9-32	9-40	0
MAP, RT. OF V, RT. OF I, RT. OF I.	9-33	9-41	0
MAP, LT. OF V, RT. OF V, RT. OF I.	9-34	9-42	0
MAP, RT. OF V, RT. OF V, RT. OF I.	9-35	9-43	0
AXIOM, VAR. L OF V, L OF V, L OF I.	9-36	0	
		*15	0
AXIOM, VAR. R OF V, L OF V, L OF I.	9-37	0	
		*15	0
AXIOM, VAR. L OF V, L OF V, R OF I.	9-38	0	
		*16	0
AXIOM, VAR. R OF V, L OF V, R OF I.	9-39	0	
		*16	0
AXIOM, VAR. L OF V, R OF I, R OF I.	9-40	0	
		*16	0
AXIOM, VAR. R OF V, R OF I, R OF I.	9-41	0	
		*16	0
AXIOM, VAR. L OF V, R OF V, R OF I.	9-42	0	
		*15	0
AXIOM, VAR. L OF V, R OF V, R OF I.	9-43	0	
		*15	0

Fig. 23--(Continued)

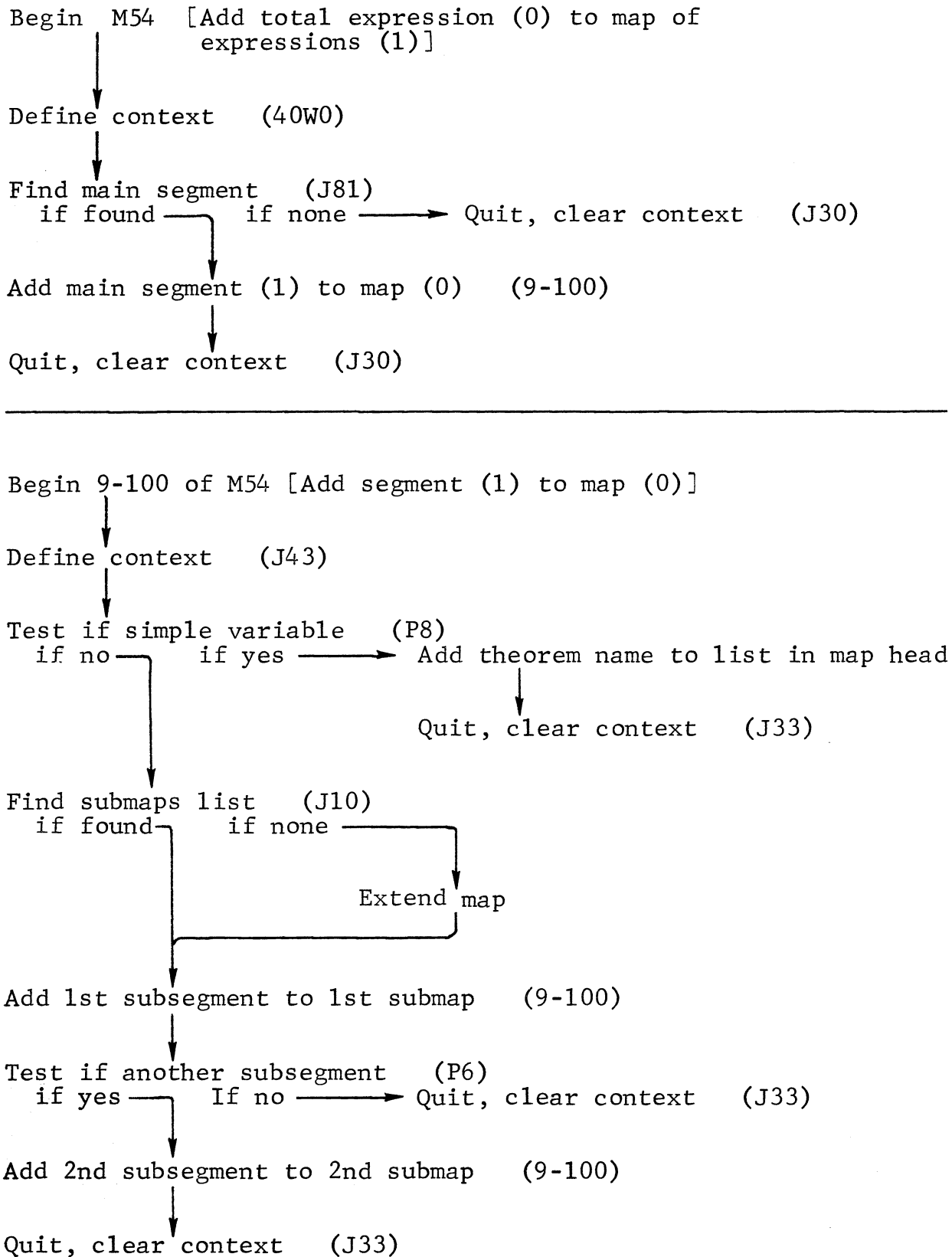


Fig. 24--Add Expression to Map

M63 (Fig. 25) extracts names of feasible match expressions from the map by using a given problem expression as a guide. This is done by laying the problem expression over the map structure and collecting true expression names from overlaid map heads. Figure 26 shows a problem expression $[PVQ]I[PV[PVQ]]$, laid over the map so that Axioms *1.2, *1.3, *1.4, and *1.5 appear in overlaid map heads. Note that *1.5 is not a feasible match for the given problem and that *1.6 is never encountered.

M63 uses recursive processes M62 (Fig. 25) to extract a list of feasible match expressions from the map structure. It uses the following procedure to gather expression names from overlaid map heads and prevent unwanted expressions, such as *1.5 of Fig. 26, from appearing in the output list.

For each map (submap) and corresponding segment (subsegment):

- 1) Recursively find all feasible matches for all subsegments from corresponding submaps; then
- 2) If the segment (subsegment) connective is unary (such as NOT), skip to step 3) below, otherwise find the intersection of matches obtained from all submaps; then
- 3) Append the results of step 2) to a copy of the list of expressions from the head of this map (submap); and
- 4) Output the result as a list of feasible from this map (submap).

This procedure meets all requirements set forth previously. When applied to Fig. 26, it shows that Axiom *1.5

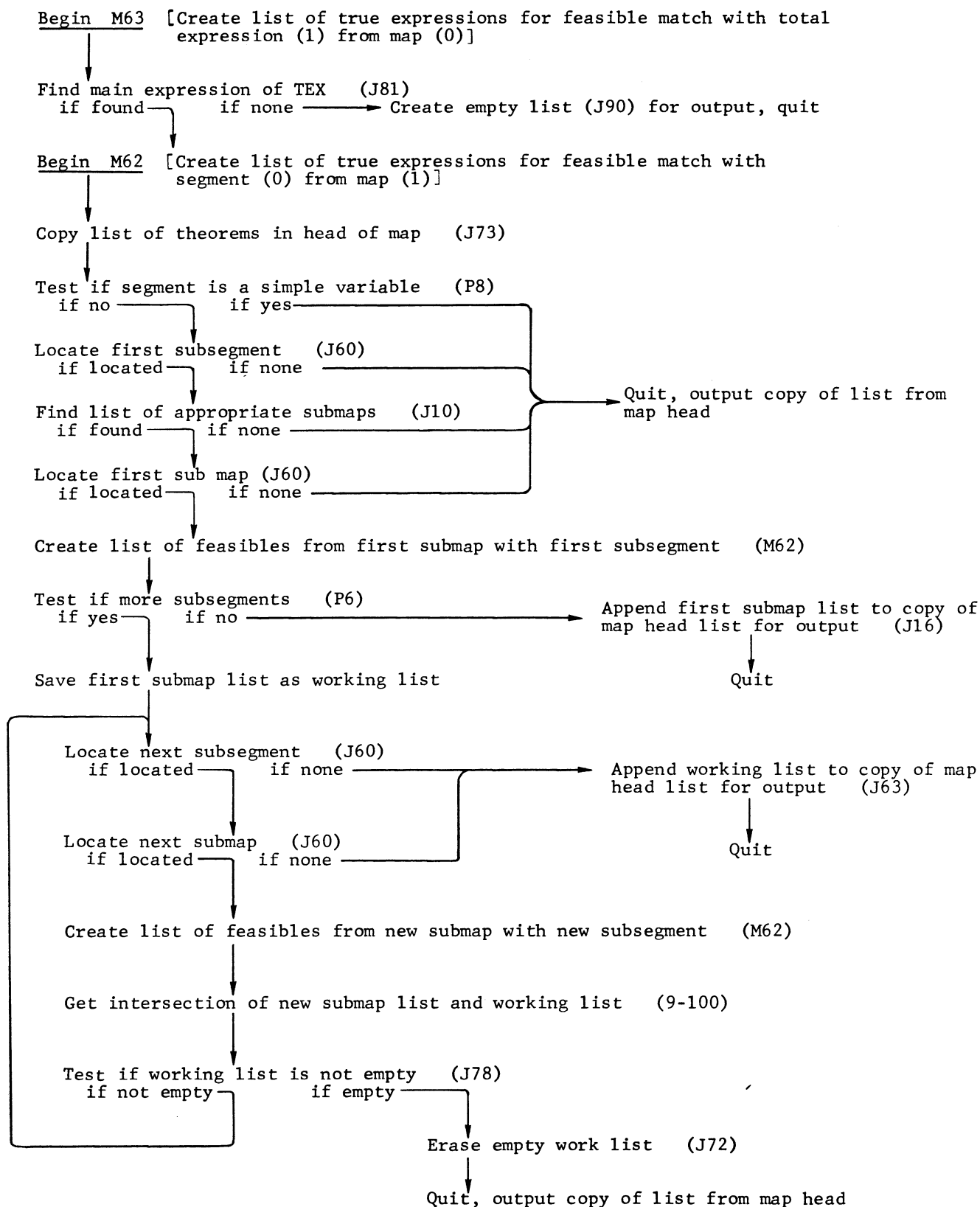


Fig. 25--Create List of Feasible Expressions

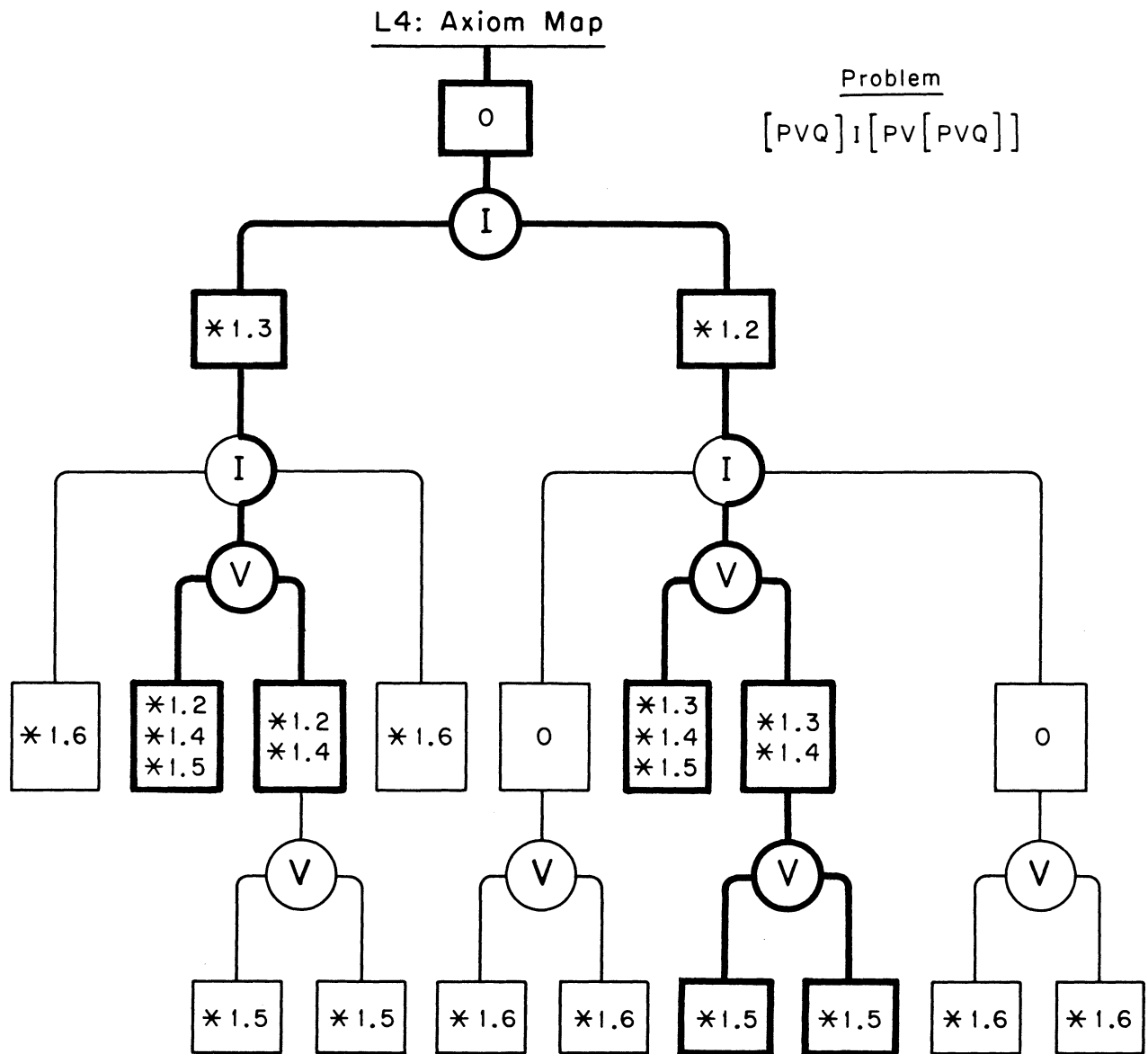


Fig. 26--Axiom Map with Overlaid Problem

is eliminated by step 2) because its name doesn't appear in the map head on the right of OR on the left of main connective IMPLIES. Step 3) provides viewing as contracted at every segment and subsegment place. The procedure completely avoids consideration of *1.6 because *1.6 doesn't have the right subsegment connectives.

The ability to completely avoid consideration of certain types of unlikely match candidates is one of the most important features of this retrieval system because each method (M11-M16) deals with a different part of the true expressions it uses. M12 matches whole expressions, so it uses the whole map. M11 matches whole problem expressions to the right sides of true expressions that have main connective IMPLIES, so it uses only the submap of right sides of main connective IMPLIES. M13, M16, and M17 use only the submaps of DEFINITIONAL EQUIVALENCE. M14 and M15 use whatever section of the map is appropriate for the problem at hand.

M14, M15, M16, and M17 use problem subsegments to search appropriate submaps, thus taking full advantage of the ability to ignore all irrelevant parts of the map. This ability becomes more valuable as the number of true expressions becomes large. Thus, this new theorem storage and retrieval system at least partially solves the old problem of what to do with too much information. Obviously irrelevant true expressions no longer get in the way.

Although the expression map technique can be used elsewhere, it is used in this version of LT only to organize true expressions. In particular, it can be applied to the found problems list which is currently

organized in terms of number of levels, number of variables, and number of variable places. However, the payoff will be smaller because the found problems match process (M40) doesn't use substitution, so there is no need for viewing as contracted at any level. In fact, a map search routine for a found problems map should specifically avoid viewing as contracted.

Reorganization of the untried problems with a map structure would be more interesting. Some modification of the structure would be required because the viewing as contracted problem is inverted. Once this is done, routines could be devised to search both the true expressions map and the untried problems map at the same time. The result of such a development might enable LT to "see" a larger part of its problem at a time by giving it some ability to scan the problem and its context as a unit.

This sort of thing should lead to more sophisticated ways to select and apply methods to problem theorem pairs. The problem executives (M1, M2) would need expansion to explore problems and plan attacks. The lower-level routines would not need any significant modification.

IX. LOWER-LEVEL ROUTINES

The lower-level routines operate on expression structures and terms. The objective of segregating these processes from the higher-level routines is to generalize the program. In theory, the expression structures that LT handles can be modified by changing only the lower-level routines. In practice, a few changes would also be required in some of the higher-level routines.

Some lower-level routines have interesting characteristics. First among these are the "Q" routines. Each of the IPL-V symbols, Q1-Q19, can take on more than one meaning, depending on the context of its usage. For example, Q7 is the name of a routine (FIND EXTERNAL NAME) and is also the name of an attribute (EXTERNAL NAME) used on the description lists of total expressions, variable terms, and character symbols.

<u>COMMENTS</u>	<u>TYPE</u>	<u>NAME</u>	<u>PQ</u>	<u>SYMB</u>	<u>LINK</u>
ROUTINE HEADER	5		00		
FIND EXTERNAL NAME		Q7	10	Q7	J10
DATA HEADER	5		01		
SYMBOL FOR CHARACTER X		X0		9-1	0
		9-1		0	
ATTRIBUTE-EXTERNAL NAME				Q7	
				9-2	0
VALUE-ALPHANUMERIC DATA TERM		9-2	21	X	

There is no conflict of usage because the description list processes never operate on the contents of cells whose names are used as attributes. If, by private convention, nothing is ever done with the value of an attribute, the attribute symbol may also serve as its own dummy value,

as well as for the name of the routine used to find it. Q5, Q6, and Q7 are examples of this usage. (See Sec. XV for code listings.)

There is no reason for "find" routines to be short and simple. Q2, Q3, and Q4 are examples of what is sometimes called an active attribute. Routine Q2 (Figs. 27, 28), when applied to a total expression, tries to find the number of levels in the expression as the value of attribute Q2 on the description list. If it doesn't find a value there, the routine counts levels, assigns the count as the value of attribute Q2, and outputs the value. H5 is set minus if a value cannot be found. This can only happen if the expression structure is faulty so that the number of levels is undefined. Q3 (Fig. 29) and Q4 work in a similar fashion to find the number of distinct variables and number of variable places.

Some interesting generators can be found among the "P" routines. P29 (Fig. 30) is interesting because it uses itself recursively with J18 as a subprocess to search the lower levels of expressions. P29 generates bound variable locations from expressions. P28 is of the same form, but generates free variable locations.

P26 (Figs. 31, 32) has a more difficult task. It is used to generate subsegment locations from a given level of a given expression. To do this, it uses local subgenerator 9-200 (Fig. 32), which counts expression sublevels as it descends into the expression to generate subsegment locations. The most interesting feature of P26 is the way subgenerator 9-200 is executed by subprocesses 9-100, with 9-100 as the subprocess.

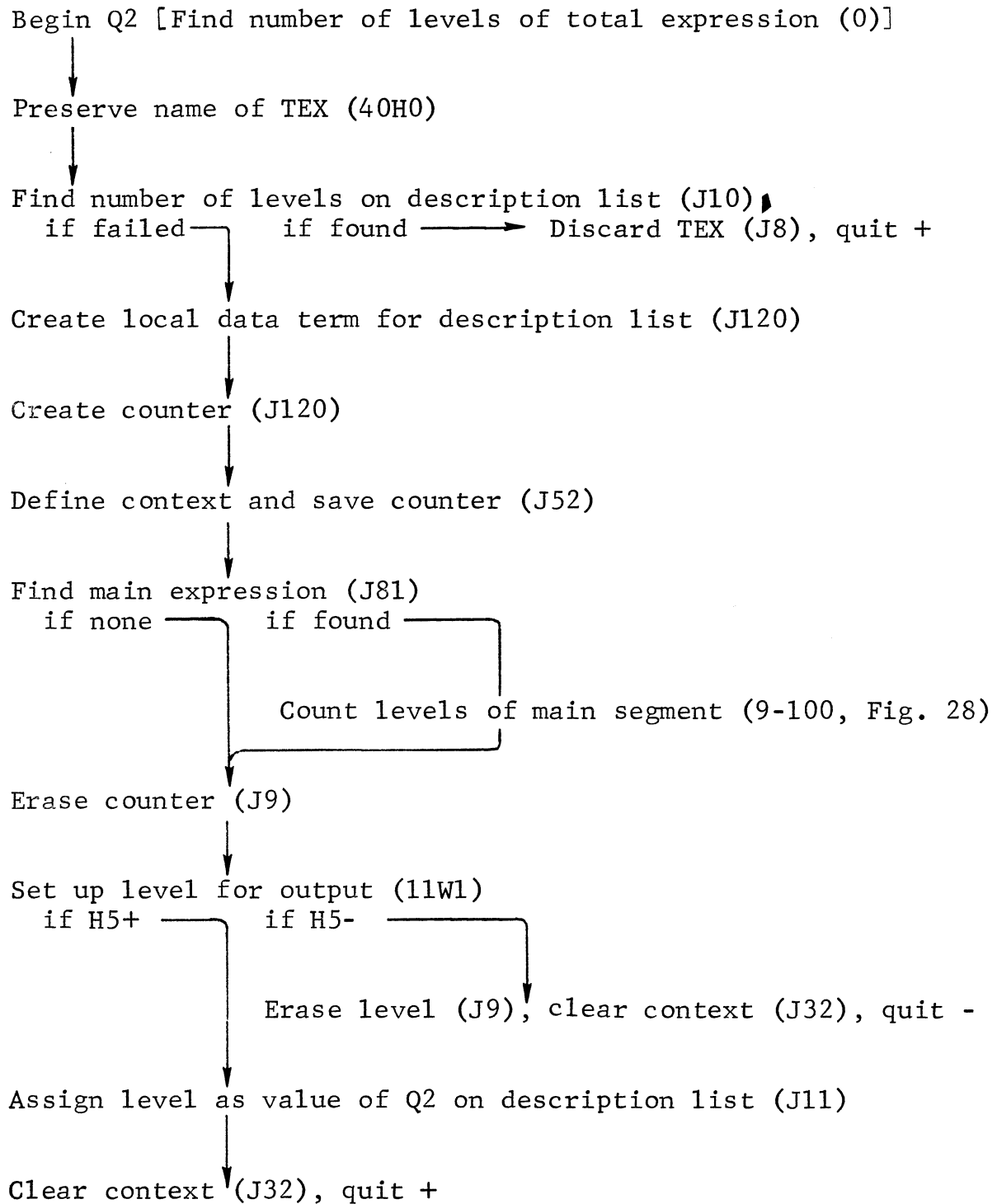


Fig. 27--Find Number of Levels

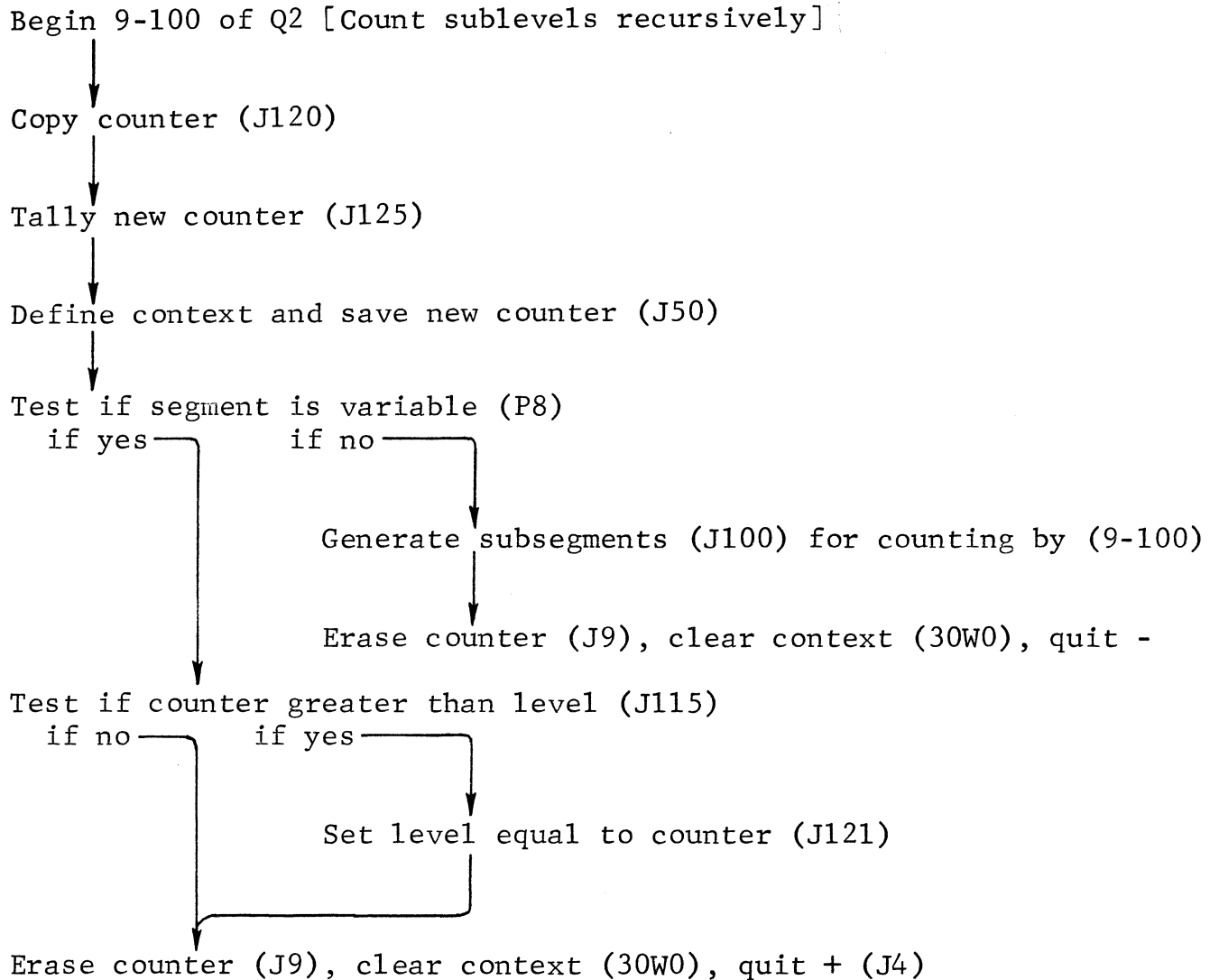


Fig. 28--Count Sublevels Recursively

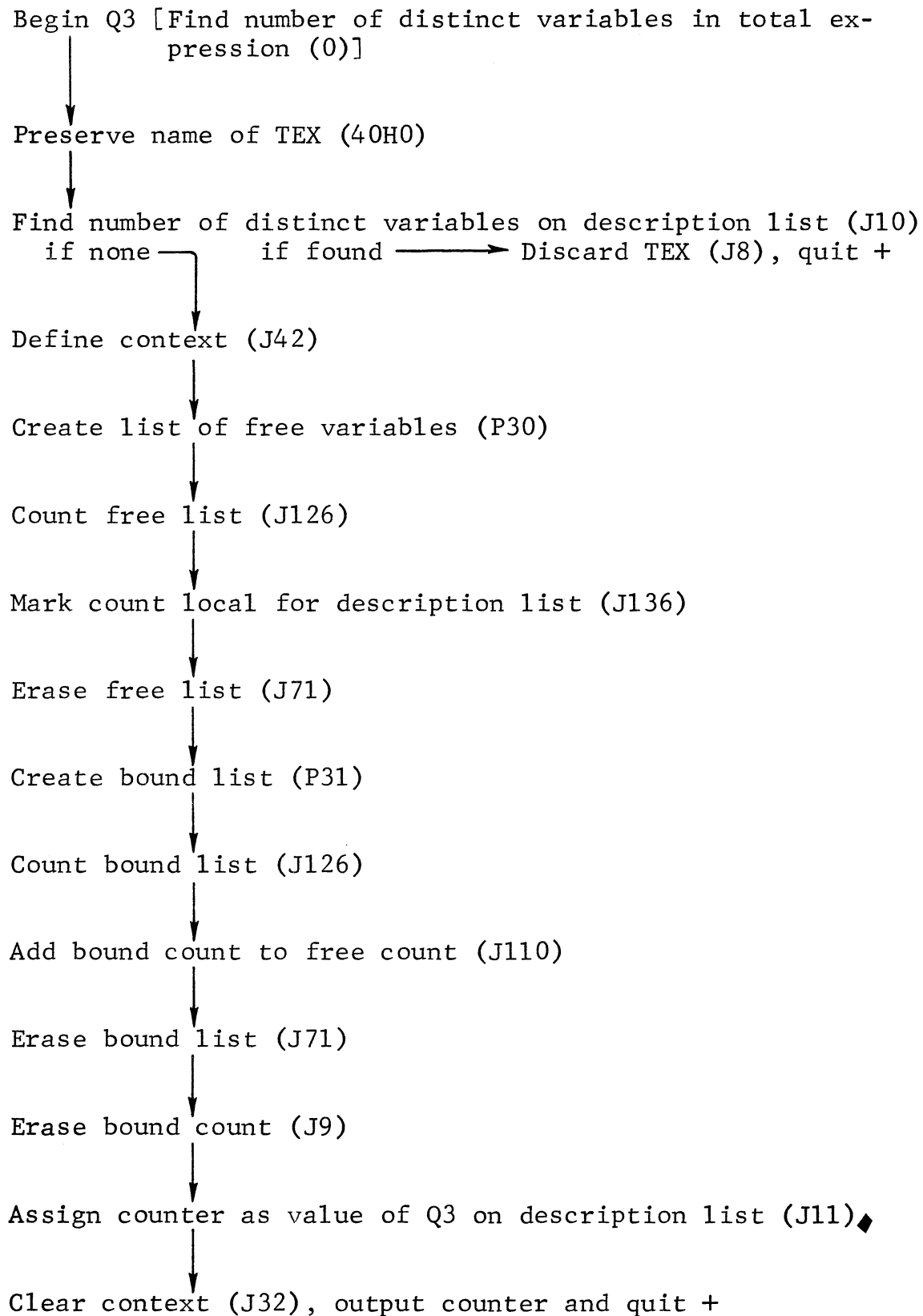


Fig. 29--Find Number of Distinct Variables

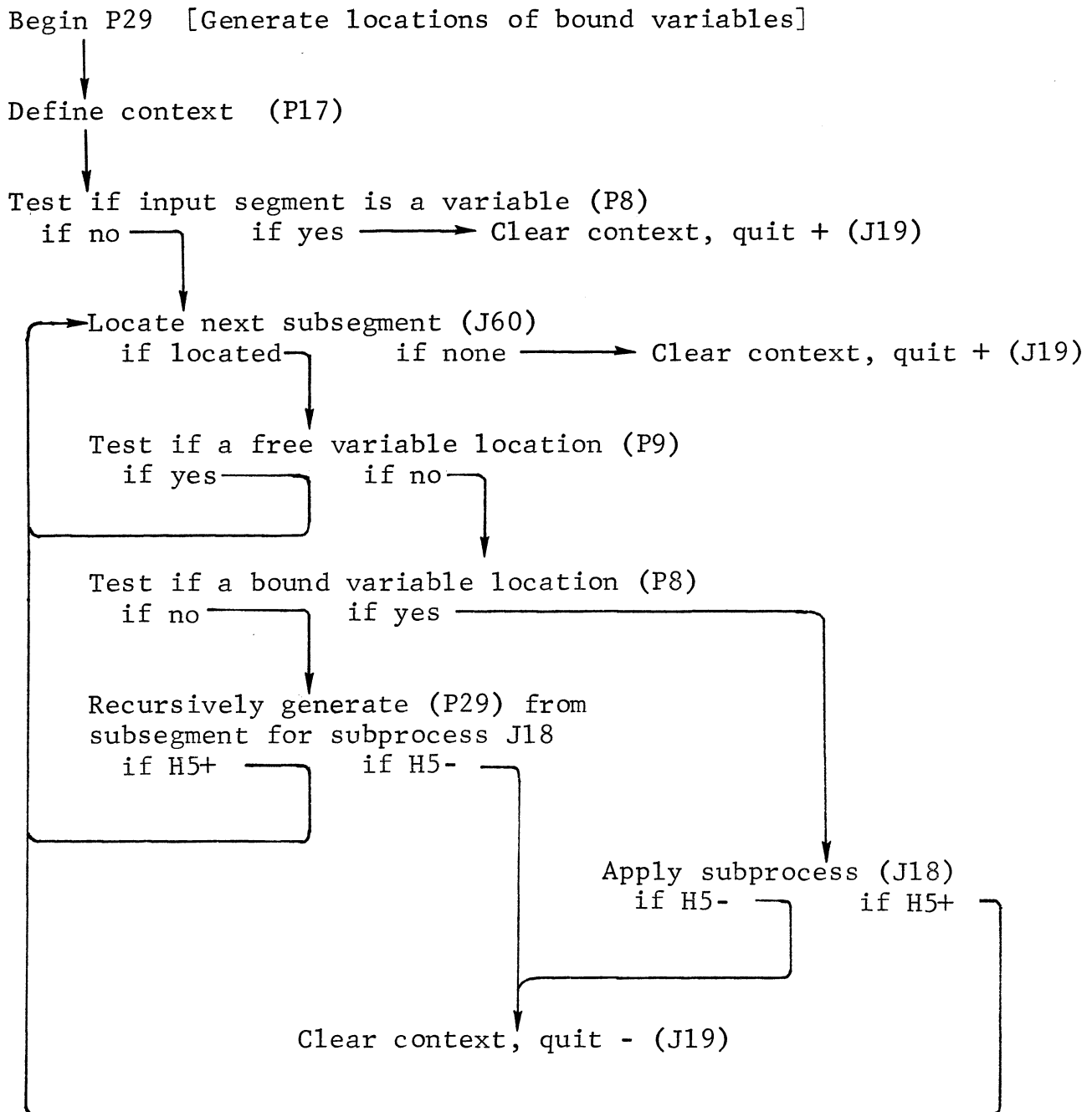


Fig. 30--Generate Locations of Bound Variables

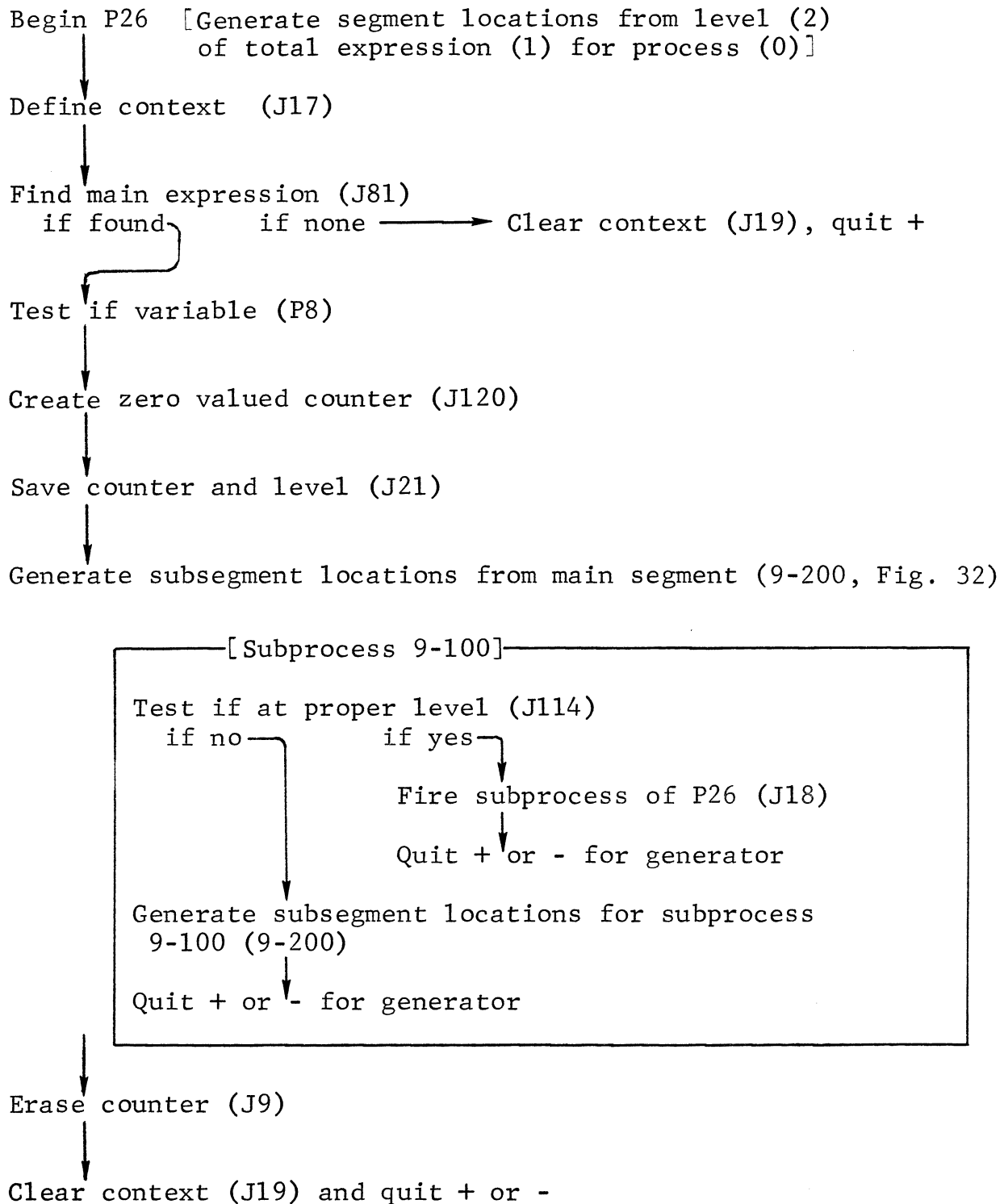


Fig. 31--Generate Segment Locations from
Given Level of Expression

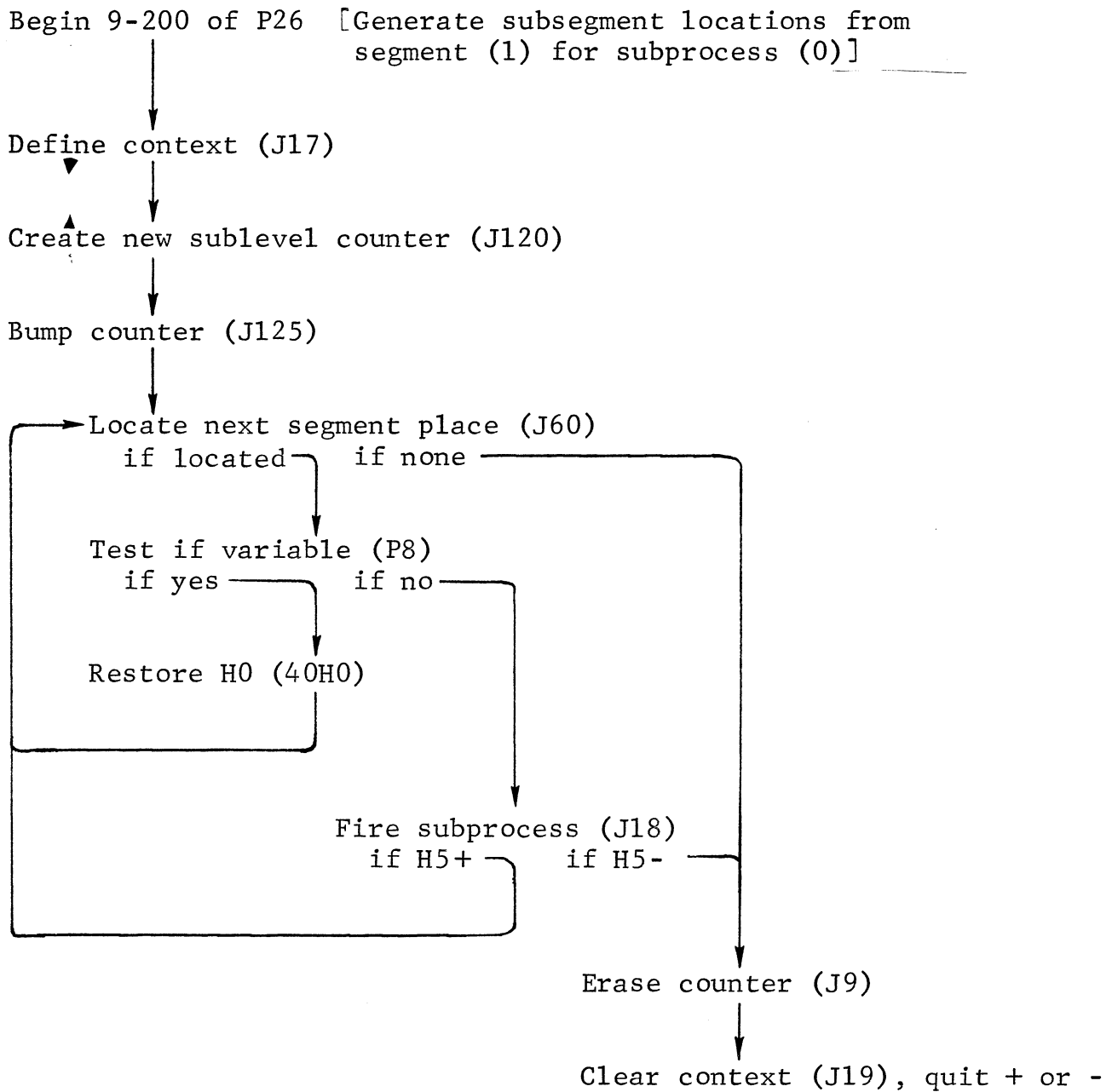


Fig. 32--Generate Subsegment Locations from Segment

P50, P51, and P52 also form an interesting group of routines, They are used to convert expressions from list form to tree form. Previous discussions have dealt only with tree form expressions. LT uses list form expressions to make inputting of expressions easier.

Expressions can be put into LT in any of three different forms. Tree forms and list forms can be directly loaded or M89 may be used to read Hollerith records with the line-read primitives (J180's). M89 will be discussed in the next section.

List form expressions have no tree structure so they are easier to write down on code sheets. The expression is carried as a simple list of character symbols, as shown below. Entire expressions must be parenthesized and redundant parentheses are not allowed.

Head	{*208	9-1	0
	{	(0	
		P0	
List Form	{	I0	
Expression	{	P0	
	{	)0	0
Description	{9-1	0	
List	{	Q7	
	{	9-2	0
External Name	{9-2	21*2.08	

P50 converts expressions from list form to tree form only if they are not yet in tree form. Tree form expressions have attribute Q15, while list forms do not.

P50 is a simple routine which first tests for tree form. If the given expression is not in tree form, P50 replaces delimited external connectives (P51); creates a

tree form from the list form (P52); discards the old list form, saving its head and description list (J75,J71); inserts the new tree form main expression after the old head (J64); and assigns tree form attribute, Q15, to the expression (J11).

P51 replaces delimited connectives such as $\dot{=}$ by scanning the list form for the delimiter symbol. When one is located, a check is made to be sure there is another delimiter symbol on the other side of the delimited symbol. If there is, all three symbols are replaced by the proper internal symbol obtained from the table of delimitable symbols. K7 holds the delimiter symbol and L8 is the name of the table of delimitable symbols.

P52 (Figs. 33, 34) creates a tree form main segment, complete with subsegments, from a given list expression without destroying the latter. Since a parenthesized list

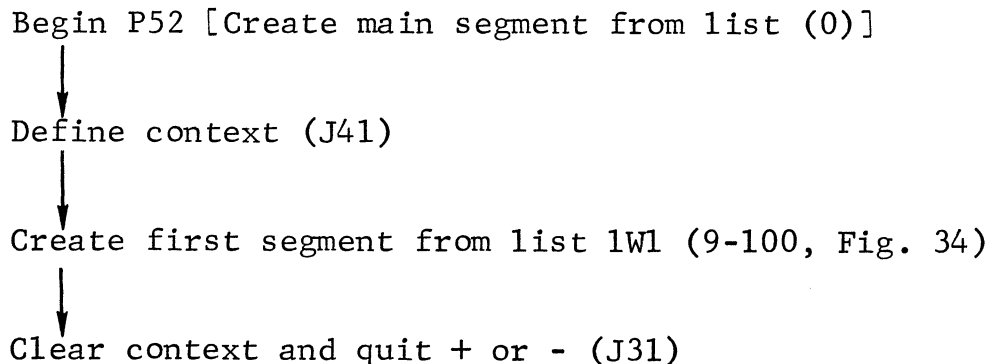


Fig. 33--Create Main Segment from List Expression

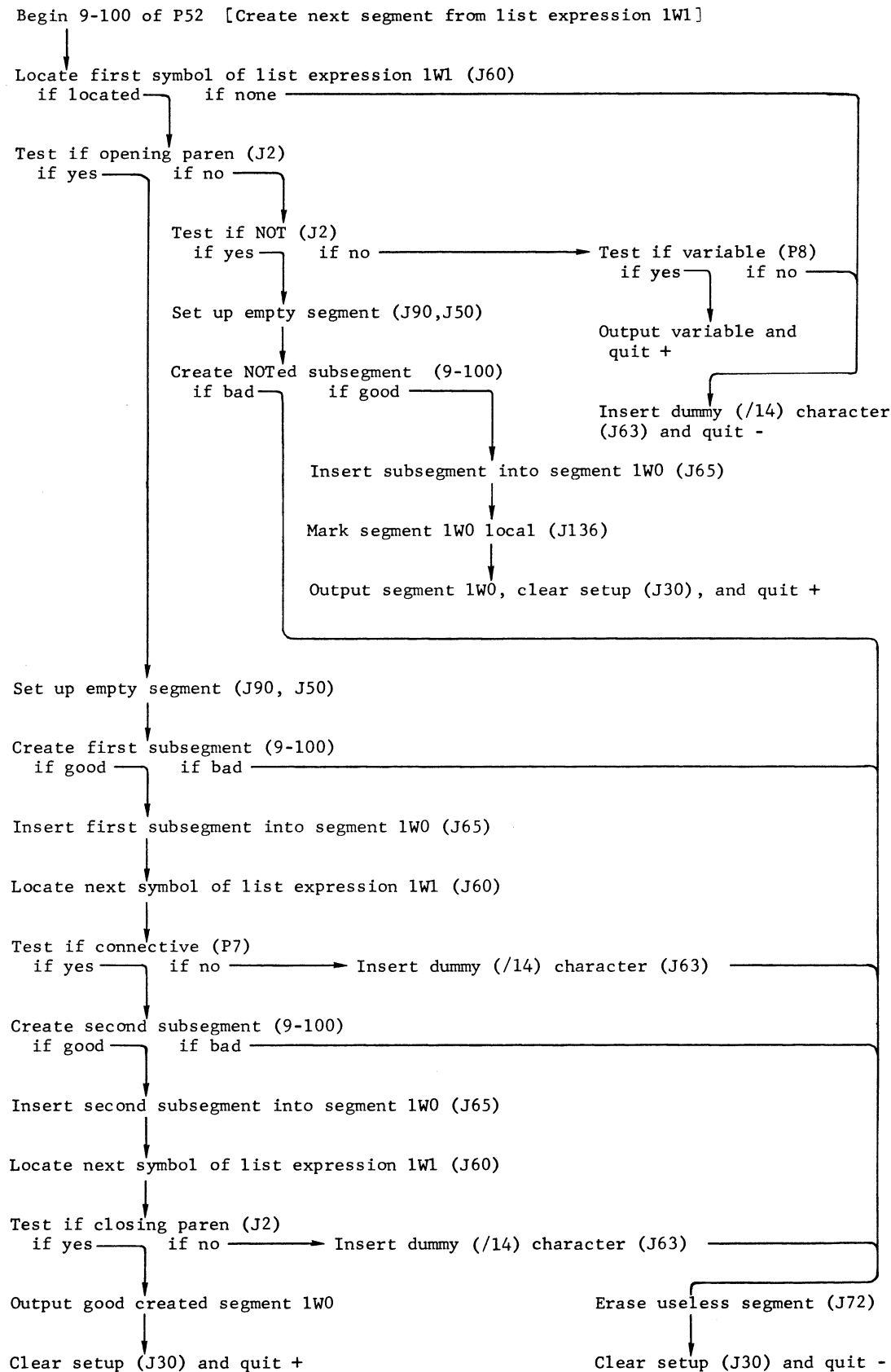


Fig. 34--Create Next Segment from List Expression

expression represents a tree structure, P52 uses a recursive subprocess (9-100) to translate the list expression into its corresponding main expression.

If the list expression is faulty, P52 inserts a dummy character symbol (/14) into the list expression. When the bad list expression is printed by M88, the dummy character appears at the place where trouble was detected.

The remaining lower-level "P" and "Q" routines are relatively trivial and can be studied directly from the code.

X. INPUT-OUTPUT ROUTINES

The input-output routines deal directly with print line primitives (J150's) and read line primitives (J180's).

As mentioned in the preceding section, expressions may be put into LT by loading them with J165 or by reading them with M89. J165 can load tree form expressions or list form expressions, while M89 can read Hollerith records with the following format:

Beginning in column 1: Any number of blanks (including none); followed by a regional symbol expression name; followed by at least 1 blank; followed by an expression string (enclosed in parentheses with no imbedded blanks); followed by at least 1 blank; followed by an optimal suffix (up to 5 characters); followed by at least 1 blank.

Example: *1.01 ((AIB).=(-AVB)) DEF.

M89 (Fig. 35) reads one record from unit 1W18 and then scans it in read line buffer 1W24, using the read line primitives. If the record is an End-of-File (E-O-F) or is totally blank, M89 quits with no output and sets H5-. If the record is faulty in some obvious respects, it is skipped and the next record from read unit 1W18 is tried. This procedure continues until an acceptable expression is found or an empty record (or an E-O-F) is found.

In the first part of the scan procedure, the name of the expression is located and checked to see if it is a regional symbol other than a standard character symbol. This is tested with J130 and P18. If the name is okay,

the main expression string is located and used to build a list form expression of variables, connectives, and parentheses. If a suffix can be found, it is assigned as the data term value of Q18 on the list expressions' description list.

M88 is the only other input-output routine that deals with list form expressions. It is used to print expressions that fail in the conversion process.

Routines M70-M82 are used to print various tree form expressions on unit 1W19. (See the vocabulary, Sec. XIV, for a list of these routines.) M73 is the central routine in this group because it is used by other routines to enter tree form expressions in the print line buffer. M73 works recursively to build a character string in the print line from a tree form expression. The process is similar to, but opposite from, that of P52. If M73 gets in trouble, it enters the external name of /14 to indicate where the expression is faulty and then continues to enter the rest of the given expression.

M71, which prints whole proof sequences, uses the derivation information associated with each problem (attributes Q10 through Q14) to trace out the successful sequence.

Two other output routines are worth mentioning. M76 is a simple routine which enters data term "text lists" by generating from them for subprocess J157.

M79 is more sophisticated. It can be used to enter the name of anything. First it tries to enter the external name (Q7); if it cannot, it tries to enter the problem number (Q8); if neither can be found, it enters

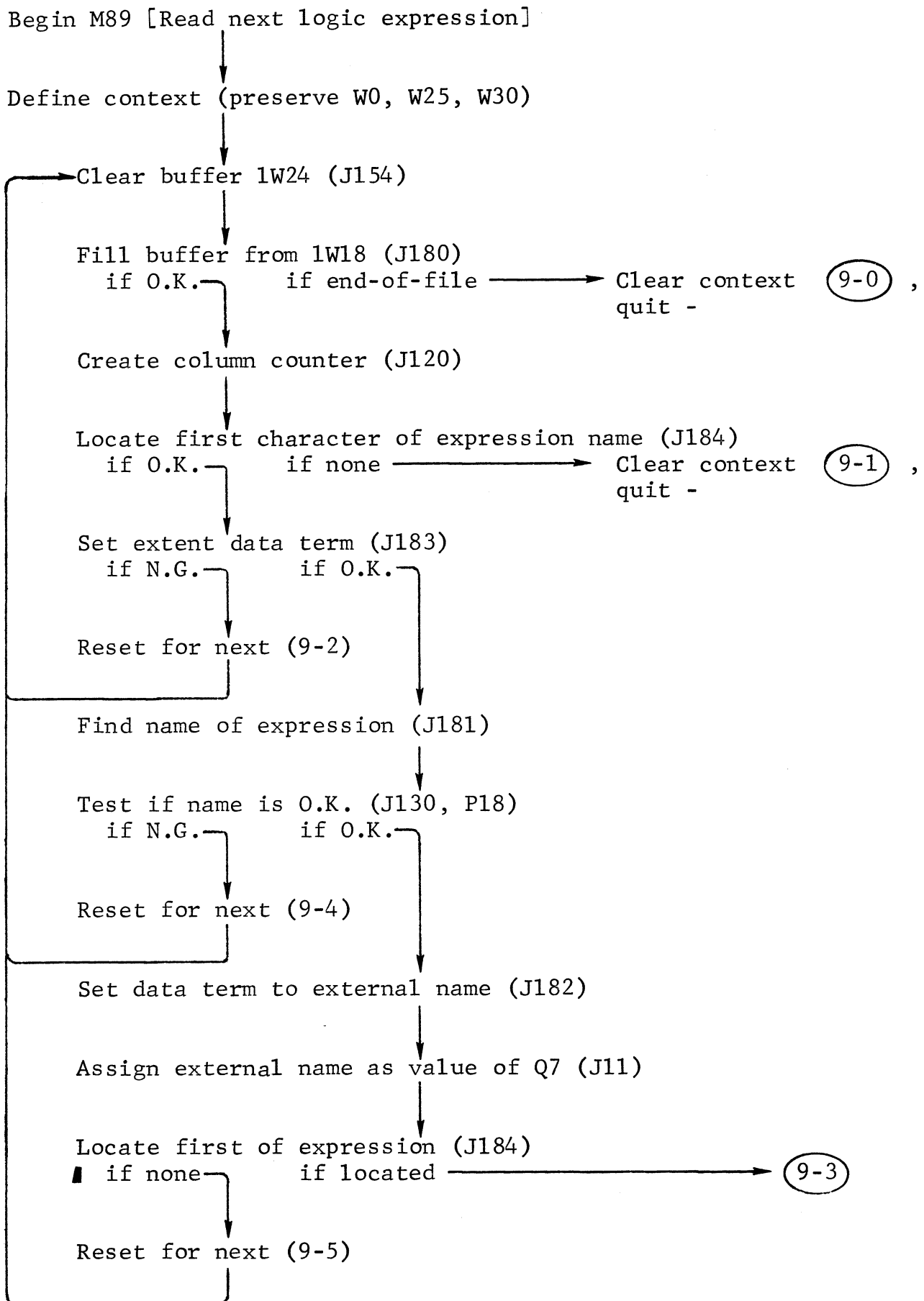


Fig. 35--Read Next Logic Expression

the given symbol with J156 which enters it in regional or internal form.

The remaining input-output routines are fairly simple examples of how to use the print line primitives. The student should be able to decipher them on his own.

XI. RUN EXECUTIVES AND DEBUGGING ROUTINES

There are two run executives. The first is X9 which creates a restart tape holding the main body of LT's routines and data. X9 terminates with J165 to facilitate loading of additional data and/or routines at run time.

The second run executive is X1. It is the highest-level executive for a theorem-proving run. After initializing some debugging devices, which will be discussed later, X1 reads a set of true expressions for the run converting each expression to tree form and adding it to the map of true expressions. The set of true expressions immediately follows the start card for X1 and ends with the first totally blank card. Any expression that fails in its conversion step (P50) is omitted from the expression set and is printed with M88 to report the difficulty.

Next, X1 reads a set of problem expressions. The problems are converted to tree form and added to the list of unproved expressions (L3). Upon encountering the end of the problem set, X1 links to multiple problem executive M2 which feeds the problems, one at a time, to the single problem executive M1 (Fig. 1).

Executive M2 tries for proofs under two different conditions. If K30 holds "R0," proved theorems are to be added to the set of true expressions, thus retaining results of past efforts for future application.[†] If K30 does not hold "R0", proved theorems are to be forgotten and each new problem is to be started from scratch.

[†] See Ref. 12.

This procedure for remembering is not the only possible one. Each subproblem in a successful proof sequence is a true theorem in its own right and could be added to the set of true expressions. Modification of LT to incorporate this other kind of remembering might make a good student project. Many experiments might be designed around this part of LT.

Symbols X10 through X19 are used for debugging routines, and X20 through X29 are used for debugging lists.

When X10 is used as the trap value of attribute H3 on the description list of W26, X10 will be executed when the cycle count in H3 becomes equal to W33. X10 sets 1W31 to full trace mode (1W31 = 1) and forces a monitor point with X19 to invoke the change immediately. The trace mode invoked by X10 can be revoked by executing X11 which pops W31 if it has been pushed down and executes X19 to make the change effective immediately.

X10 and X11 are useful when a full trace is desired of a small part of the program that is executed only after a considerable running time. By initially setting W33 to the desired value, the program can run in no-trace mode until W33 equals H3 and then full trace can be invoked for a short time, after which it can again be revoked.

Other similar routines can be designed to meet special needs when they arise. It is difficult to anticipate exactly what will prove useful in all situations.

Routine X13 is used to "snap" (0) at monitor points by loading X13 into W12 and W13. There is some danger in using J150 this way because (0) is not always guaranteed

to be a proper list structure. For example, (0) is sometimes a generator subprocess.

Routine X14 is used to obtain a restart tape on an operator signal from the computer console by loading X14 into W14. X14 uses the fact that J166 sets H5+ and the restart mechanism sets H5- in order to stop with a post mortem if just saved or to go on with the program if re-starting.

Routine X15 is used to extend the post mortem by loading X15 into W15. The listing in Sec. XV shows that X15 is a simple routine to print L4 (the map of all true expressions).

Routine X19, as mentioned in connection with X10 and X11, is used to force a monitor point in order to make changes in trace mode effective immediately.

The debugging lists (X21-X23) are used by X1 to mark routines for tracing and to set up the trap attribute-value pairs of W26. The technique of marking routines to trace from a given list is simple but effective. The other way to mark for trace is to reload the whole routine with a trace mark ($Q = 3$ or 4) in its head.

The above mentioned debugging aids were the only ones used to check out the program. Other more sophisticated tools might be designed, but LT's bugs have not required them.

XII. A SAMPLE RUN

This section includes a listing of an input deck for a run and a listing of the resultant output.

INPUT DECK

The input deck includes a modified routine, a modified data structure, and a collection of run parameters and lists. Of particular interest are L6 and L7 which control the order of application of the methods. This run has relatively low limits (K20-K22) and calls for printing of rejected problems (K31) and remembering proved problems (K30).

Following the KICK OFF FOR PROVING THEOREMS are three sets of logic expressions. The first set is to be used as true expressions and includes definitions and axioms. Definition *2.33 will fail in conversion because LT cannot handle expressions such as (PVQVR). The second set contains the theorems to be proved (problems) in this run. The third set will be ignored because it follows the second blank card. It was placed here in order to include in this Memorandum all other theorems from *2, *3, *4, and *5 of Principia Mathematica.

JOB	8168,LTNEW1,EAS826,5MIN,0,099,C	STEFFERUD	
ASSIGN	A6=SYSAR2		
ASSIGN	B6=SYSAR3		
IPL			
LOGIC THEORIST TEST	9		
RELOAD FROM TAPE 2	5	4 2	
MODIFIED ROUTINES	5	00	R
	1		R
Q17 FIND LEVEL OF SUBSEGMENT	Q17	40W0	Q017R000
REPLACEMENT IN TEX (0).		60W0	Q017R010
		10Q17	Q017R020
FIND CURRENT LEVEL.		J10	Q017R030
IF NONE,		70	Q017R040
		11W0	Q017R050
FIND NUMBER OF LEVELS,		Q2	Q017R060
IF NONE, QUIT -.		70J30	Q017R065
COPY,		J120	Q017R070
SAVE ONE FOR OUTPUT,		40H0	Q017R080
		11W0	Q017R090
		J6	Q017R100
AND ASSIGN AS CURRENT LEVEL.		10Q17	Q017R110
		30W0	Q017R120
		J11	

MODIFIED DATA	5	01		D
/16 DUMMY EXPRESSION --	/16	9-1		/016D000
'DEFINITIONS'.		9-2	0	/016D010
	9-1	0		/016D020
		Q15		/016D030
		Q15		/016D040
		Q7		/016D050
			0	/016D060
EXTERNAL NAME		21		/016D070
CONNECTIVE 'I'.	9-2	I0		/016D080
		9-10		/016D090
		9-20	0	/016D100
DUMMY VARIABLE 'DEFIN'.	9-10		0	/016D110
		0		/016D120
		Q5		/016D130
		Q5		/016D140
		Q9		/016D150
		Q9		/016D160
		Q7		/016D170
			0	/016D180
EXTERNAL NAME.		21DEFIN		/016D190
DUMMY VARIABLE 'TIONS'.	9-20		0	/016D200
		0		/016D210
		Q5		/016D220
		Q5		/016D230
		Q9		/016D240
		Q9		/016D250
		Q7		/016D260
			0	/016D270
EXTERNAL NAME.		21TIONS		/016D280
RUN DATA HEADER	5	01		D -
LIMIT ON NUMBER OF SUBPROBLEMS	K20	+ 1	50	K020D000
LIMIT ON NUMBER OF SUBSTITUTIONS	K21	+ 1	50	K021D000
LIMIT ON EFFORT	K22	+ 1	20 0000	K022D000
R= ADD PROVED THEOREMS TO THEOREMS	K30	R		K030D000
Y = PRINT REJECTED SUBPROBLEMS.	K31	YES		K031D000
L6 LIST OF METHODS FOR ORIG PROBS	L6	0		L006D000
		M16		L006D010
		M17		L006D020
L7 LIST OF METHODS FOR PROBLEMS.	L7	0		L007D000
REPLACEMENT.		M13		L007D010
DETACHMENT.		M11		L007D010
FORWARD CHAINING.		M14		L007D010
BACKWARD CHAINING.		M15		L007D010
W12 SET-UP ENTRY SNAP ACTION.	W12	X13	0	W012D000
W13 SET-UP EXIT SNAP ACTION.	W13	X13	0	W013D000
	W15	X15	0	W015D000
	X21	0		X021D000
	X22	0		X022D000
DESCRIPTION LIST OF TRAP ACTIONS.	X23	0		X023D000

KICK OFF FOR PROVING THEOREMS. 5

X1

*1.01 ((PIQ).=.(-PVQ)) DEF.
 *2.33 ((PVQVR).=.(PVQ)VR)) DEF.
 *3.01 ((P*Q).=.-(-PV-Q)) DEF.
 4.01 ((P=Q).=.(PIQ)(QIP))) DEF.
 *1.2 ((AVA)IA)
 *1.3 (BI(AVB))
 *1.4 ((AVB)I(BVA))
 *1.5 ((AV(BVC))I(BV(AVC)))
 *1.6 ((BIC)I((AVB)I(AVC)))

X

*2.01 ((PI-P)I-P)
 *2.02 (QI(PIQ))
 *2.03 ((PI-Q)I(QI-P))
 *2.04 ((PI(QIR))I(QI(PIR)))
 *2.05 ((QIR)I((PIQ)I(PIR)))
 *2.06 ((PIQ)I((QIR)I(PIR)))
 *2.07 (PI(PVP))
 *2.08 (PIP)
 *2.10 (-PVP)
 *2.11 (PV-P)
 *2.12 (PI--P)
 *2.13 (PV---P)
 *2.14 (--PIP)
 *2.15 ((-PIQ)I(-QIP))
 *2.20 (PI(PVQ))
 *2.21 (-PI(PIQ))
 *2.24 (PI(-PIQ))
 *3.13 (-(P*Q)I(-PV-Q))
 *3.14 ((-PV-Q)I-(P*Q))
 3.24 -(P-P)
 *4.13 (P--P)
 *4.20 (P=P)
 *4.24 (P=(P*P))
 *4.25 (P=(PVP))

3.02 ((PIQIR).=.(PIQ)(QIR))) DEF.
 4.02 ((P=Q=R).=.(P=Q)(Q=R))) DEF.
 *4.34 ((P*Q*R).=.(P*Q)*R)) DEF.
 *2.16 ((PIQ)I(-QI-P))
 *2.17 ((-QI-P)I(PIQ))
 *2.18 ((-PIP)IP)
 *2.25 (PV((PVQ)IQ))
 *2.26 (-PV((PIQ)IQ))
 *2.27 (PI((PIQ)IQ))
 *2.30 ((PV(QVR))I(PV(RVQ)))
 *2.31 ((PV(QVR))I((PVQ)VR))
 *2.32 (((PVQ)VR)I(PV(QVR)))
 *2.36 ((QIR)I((PVQ)I(RVP)))
 *2.37 ((QIR)I((QVP)I(PVR)))
 *2.38 ((QIR)I((QVP)I(RVP)))

X

X

*2.40	((PV(PVQ))I(PVQ))	
*2.41	((QV(PVQ))I(PVQ))	
*2.42	((-PV(PIQ))I(PIQ))	
*2.43	((PI(PIQ))I(PIQ))	
*2.45	((-PVQ)I-P)	
*2.46	((-PVQ)I-Q)	
*2.47	((-PVQ)I(-PVQ))	
*2.48	((-PVQ)I(PV-Q))	
*2.49	((-PVQ)I(-PV-Q))	
*2.50	((-PIQ)I(-PIQ))	
*2.51	((-PIQ)I(PI-Q))	
*2.52	((-PIQ)I(-PI-Q))	
*2521	((-PIQ)I(QIP))	
*2.53	((PVQ)I(-PIQ))	
*2.54	((-PIQ)I(PVQ))	
*2.55	((-PI((PVQ)IQ))	
*2.56	((-QI((PVQ)IP))	
*2.60	((-PIQ)I((PIQ)IQ))	
*2.61	((PIQ)I((-PIQ)IQ))	
*2.62	((PVQ)I((PIQ)IQ))	
*2621	((PIQ)I((PVQ)IQ))	
*2.63	((PVQ)I((-PVQ)IQ))	
*2.64	((PVQ)I((PV-Q)IP))	
*2.65	((PIQ)I((PI-Q)I-P))	
*2.67	((PVQ)IQ)I(PIQ))	
*2.68	((PIQ)IQ)I(PVQ))	
*2.69	((PIQ)IQ)I((QIP)IP))	
*2.73	((PIQ)I((PVQVR)I(QVR)))	X
*2.74	((QIP)I((PVQVR)I(PVR)))	X
*2.75	((PVQ)I((PV(QIR))I(PVR)))	
*2.76	((PV(QIR))I((PVQ)I(PVR)))	
*2.77	((PI(QIR))I((PIQ)I(PIR)))	
*2.80	((QVR)I((-RVS)I(QVS)))	
*2.81	((QI(RIS))I((PVQ)I((PVR)I(PVS))))	
*2.82	((PVQVR)I((PV-RVS)I(PVQVS)))	X
*2.83	((PI(QIR))I((PI(RIS))I(PI(QIS))))	
*2.85	((PVQ)I(PVR))I(PV(QIR))	
*2.86	((PIQ)I(PIR))I(PI(QIR))	
*3.10	((P*Q)I-(-PV-Q))	
*3.11	((-(-PV-Q)I(P*Q))	
*3.12	((-PV-Q)V(P*Q))	
*3.20	(PI(QI(P*Q)))	
*3.21	(QI(PI(P*Q)))	
*3.22	((P*Q)I(Q*P))	
*3.26	((P*Q)IP)	
*3.27	((P*Q)IQ)	
*3.30	((P*Q)IR)I(PI(QIR))	
*3.31	((PI(QIR))I((P*Q)IR))	
3.33	((PIQ)(QIR))I(PIR))	
3.34	((QIR)(PIQ))I(PIR))	
3.35	((P(PIQ))IQ)	

*3.37 $((P*Q)IR)I((P*-R)I-Q))$
*3.40 $((P*Q)I(PIQ))$
*3.41 $((PIR)I((P*Q)IR))$
*3.42 $((QIR)I((P*Q)IR))$
3.43 $((PIQ)(PIR))I(PI(Q*R))$
3.44 $((QIR)(RIP))I((QVR)IP)$
*3.45 $((PIQ)I((P*R)I(Q*R)))$
3.47 $((PIR)(QIS))I((P*Q)I(R*S))$
3.48 $((PIR)(QIS))I((PVQ)I(RVS))$
*4.10 $((PIQ)=(-QI-P))$
*4.11 $((P=Q)=(-P=-Q))$
*4.12 $((P=-Q)=(Q=-P))$
*4.14 $((P*Q)IR)=((P*-R)I-Q))$
*4.15 $((P*Q)I-R)=((Q*R)I-P))$
*4.21 $((P=Q)=(Q=P))$
4.22 $((P=Q)(Q=R))I(P=R))$
*4.30 $((P*Q)=(Q*P))$
*4.31 $((PVQ)=(QVP))$
*4.32 $((P*Q)*R)=(P*(Q*R))$
*4.33 $((PVQ)VR)=(PV(QVR))$
*4.36 $((P=Q)I((P*R)=(Q*R)))$
*4.37 $((P=Q)I((PVR)=(QVR)))$
4.38 $((P=R)(Q=S))I((P*Q)=(R*S))$
4.39 $((P=R)(Q=S))I((PVQ)=(RVS))$
4.40 $((P(QVR))=((P*Q)V(P*R)))$
*4.41 $((PV(Q*R))=((PVQ)*(PVR)))$
*4.42 $(P=((P*Q)V(P*-Q)))$
4.43 $(P=((PVQ)(PV-Q)))$
*4.44 $(P=(PV(P*Q)))$
4.45 $(P=(P(PVQ)))$
*4.50 $((P*Q)=-(-PV-Q))$
*4.51 $(-(P*Q)=(-PV-Q))$
4.52 $((P-Q)=-(-PVQ))$
4.53 $(-(P-Q)=-(-PVQ))$
*4.54 $((-P*Q)=-(-PV-Q))$
*4.55 $(-(-P*Q)=(PV-Q))$
4.56 $((-P-Q)=-(-PVQ))$
4.57 $(-(-P-Q)=(PVQ))$
*4.60 $((PIP)=(-PVQ))$
4.61 $(-(PIQ)=(P-Q))$
*4.62 $((PI-Q)=(-PV-Q))$
*4.63 $(-(PI-Q)=(P*Q))$
*4.64 $((-PIQ)=(PVQ))$
4.65 $(-(-PIQ)=(-P-Q))$
4.66 $((-P-Q)=(PV-Q))$
*4.67 $(-(-PI-Q)=(-P*Q))$
*4.70 $((PIQ)=(PI(P*Q)))$
*4.71 $((PIQ)=(P=(P*Q)))$
*4.72 $((PIQ)=(Q=(PVQ)))$
*4.73 $(QI(P=(P*Q)))$
*4.74 $(-PI(Q=(PVQ)))$

4.76 $((PIQ)(PIR))=(PI(Q*R))$
 4.77 $((QIP)(RIP))=((QVR)IP)$
 *4.78 $((PIQ)V(PIR))=(PI(QVR))$
 *4.79 $((QIP)V(RIP))=((Q*R)IP)$
 *4.80 $((PI-P)=-P)$
 *4.81 $((-PIP)=P)$
 4.82 $((PIQ)(PI-Q))=-P$
 4.83 $((PIQ)(-PIQ))=Q$
 *4.84 $((P=Q)I((PIR)=(QIR)))$
 *4.85 $((P=Q)I((RIP)=(RIQ)))$
 *4.86 $((P=Q)I((P=R)=(Q=R)))$
 *4.87 $((P*Q)IR)=(PI(QIR))=(QI(PIR)=(Q*P)IR))$ X
 *5.30 $((P*Q)IR)=((P*Q)I(P*R))$
 *5.31 $((R*P)IQ)I(PI(Q*R))$
 *5.32 $((PI(Q=R))=((P*Q)=(P*R)))$
 5.33 $((P(QIR))=(P*((P*Q)IR)))$
 5.35 $((PIQ)(PIR))I(PI(Q=R))$
 5.36 $((P(P=Q))=(Q*(P=Q)))$
 *5.40 $((PI(PIQ))=(PIQ))$
 *5.41 $((PIQ)I(PIR))=(PI(QIR))$
 *5.42 $((PI(QIR))=(PI(QI(P*R))))$
 *5.44 $((PIQ)I((PIR)=(PI(Q*R))))$
 *5.50 $(PI((PIQ)=Q))$
 *5.501 $(PI(Q=(P=Q)))$
 5.53 $((PVQVR)IS)=((PIS)(QIS)*(RIS))$ X
 *5.54 $((P*Q)=P)V((P*Q)=Q)$
 *5.55 $((PVQ)=P)V((PVQ)=Q)$
 5.60 $((P-Q)IR)=(PI(QVR))$
 5.61 $((PVQ)-Q)=(P*-Q)$
 *5.62 $((P*Q)V-Q)=(PV-Q)$
 *5.63 $((PVQ)=(PV(-P*Q)))$
 *5.70 $((PVR)=(QVR))=(RV(P=Q))$
 *5.71 $((QI-R)I((PVQ)*R)=(P*R))$
 *5.74 $((PI(Q=R))=((PIQ)=(PIR)))$
 5.75 $((RI-Q)(P=(QVR))I((P*-Q)=R))$

OUTPUT

The output from the sample run shows that the run commences from a restart. After the loader listings, the true expressions and problems for the run are listed. Note that *2.33 failed and that all true expressions have had their bound variable replaced by free variables.

Next come the proof printouts. First the problem is printed, then the subproblems are printed in the order of generation. Then, if a proof is found, it is printed along with a statement of effort applied. Finally, each proved theorem that is remembered is printed as it appears with its bound variables replaced.

At the end of the run, an IPL-V post mortem printout appears with a printout of the true expressions map as it appeared at the conclusion of the run. At the end of this sample run the map uses approximately 500 cells and holds approximately 30 expressions, which is about 17 cells per expression. With more expressions in the map, it becomes considerably more efficient.

-78-

JOB 8168,LTNEW1,EAS826,5MIN,0,099,C STEFFERUD 1 003650 06/05/63
ASSIGN A6=SYSAR2
ASSIGN B6=SYSAR3
IPL

LOGIC THEORIST TEST 9

RELOAD FROM TAPE 2 5 4 2
MEMORY RELOADED FROM TAPE A6. 3956 0 0 J166 J165

MODIFIED ROUTINES				5	00		R	
671	4	0	24587 3976	1			R	
3976	6	0	24587 3975	4	0	W0	Q17	Q017R000
3975	1	0	671 3977	6	0	W0		Q017R010
3977	0	0	24770 3978	1	0	Q17	Q17	Q017R020
3978	7	0	3979 24790			J10		Q017R030
3979	1	1	24587 3980	7	0		J30	Q017R040
3980	0	0	656 3981	1	1	W0		Q017R050
3981	7	0	24790 3982			Q2		Q017R060
3982	0	0	24880 3983	7	0	J30		Q017R065
3983	4	0	24574 3984			J120		Q017R070
3984	1	1	24587 3985	4	0	H0		Q017R080
3985	0	0	24766 3986	1	1	W0		Q017R090
3986	1	0	671 3987			J6		Q017R100
3987	3	0	24587 24771	1	0	Q17		Q017R110
				3	0	W0	J11	Q017R120

MODIFIED DATA					5	01	D	
1540	0 2	3989	3988	/16		9-1	/16 DUMMY EXPRESSION --	/0160000
3988	0 2	3990	0			9-2	'DEFINITIONS'.	/0160010
3989	0 4	0	3991	9-1		0		/0160020
3991	0 0	669	3992			Q15		/0160030
3992	0 0	669	3993			Q15		/0160040
3993	0 0	661	3994			Q7		/0160050
3994	0 2	3995	0					/0160060
3995	2 1							/0160070
3990	0 0	134	3996	9-2	2 1	10	EXTERNAL NAME	/0160080
3996	0 2	3998	3997			9-10	CONNECTIVE 'I'.	/0160090
3997	0 2	3999	0			9-20		/0160100
3998	0 2	4000	0	9-10		0	DUMMY VARIABLE 'DEFIN'.	/0160110
4000	0 4	0	4001			Q5		/0160120
4001	0 0	659	4002			Q5		/0160130
4002	0 0	659	4003			Q9		/0160140
4003	0 0	663	4004			Q9		/0160150
4004	0 0	663	4005			Q7		/0160160
4005	0 0	661	4006					/0160170
4006	0 2	4007	0					/0160180
4007	2 1		DEFIN		2 1	DEFIN	EXTERNAL NAME.	/0160190
3999	0 2	4008	0	9-20		0	DUMMY VARIABLE 'TIONS'.	/0160200
4008	0 4	0	4009			Q5		/0160210
4009	0 0	659	4010			Q5		/0160220
4010	0 0	659	4011			Q9		/0160230
4011	0 0	663	4012			Q9		/0160240
4012	0 0	663	4013			Q7		/0160250
4013	0 0	661	4014					/0160260
4014	0 2	4015	0					/0160270
4015	2 1		TIONS		2 1	TIONS	EXTERNAL NAME.	/0160280

RUN DATA HEADER				5 01 D -				
164	0 1	50	K20	+	1	50	LIMIT ON NUMBER OF SUBPROBLEMS	K020D000
165	0 1	50	K21	+	1	50	LIMIT ON NUMBER OF SUBSTITUTIONS	K021D000
166	0 1	200000	K22	+	1	20 0000	LIMIT ON EFFORT	K022D000
174	0 0 704	0	K30		R		R= ADD PROVED THEOREMS TO THEOREMS	K030D000
175	0 0 834	0	K31		YES		Y = PRINT REJECTED SUBPROBLEMS.	K031D000
250	0 4 0	4016	L6		0		L6 LIST OF METHODS FOR ORIG PROBS	L006D000
4016	0 0 310	4017			M16			L006D010
4017	0 0 311	0			M17			L006D020
251	0 4 0	4018	L7		0		L7 LIST OF METHODS FOR PROBLEMS.	L007D000
4018	0 0 307	4019			M13		REPLACEMENT.	L007D010
4019	0 0 305	4020			M11		DETACHMENT.	L007D010
4020	0 0 308	4021			M14		FORWARD CHAINING.	L007D010
4021	0 0 309	0			M15		BACKWARD CHAINING.	L007D010
24599	0 0 797	0	W12		X13	0	W12 SET-UP ENTRY SNAP ACTION.	W012D000
24600	0 0 797	0	W13		X13	0	W13 SET-UP EXIT SNAP ACTION.	W013D000
24602	0 0 799	0	W15		X15	0		W015D000
805	0 4 0	0	X21		0			X021D000
806	0 4 0	0	X22		0			X022D000
807	0 4 0	0	X23		0		DESCRIPTION LIST OF TRAP ACTIONS.	X023D000
KICK OFF FOR PROVING THEOREMS.			5		X1			
END OF LOADING. PROGRAM STARTS AT X1 1 1 W26 3921								
NUMBER OF CELLS ON AVAILABLE SPACE=17978								

*1.01 (AIB).=.(-AVB) DEF.
BAD EXPRESSION ((PVQ/UGH/VR).=.(PVQ)VR))
*3.01 (A*B).=.(-(-AV-B)) DEF.
4.01 (A=B).=.(AIB)(BIA)) DEF.
*1.2 (AVA)IA
*1.3 BI(AVB)
*1.4 (AVB)I(BVA)
*1.5 (AV(BVC))I(BV(AVC))
*1.6 (BIC)I((AVB)I(AVC))

*2.01 (PI-P)I-P
*2.02 QI(PIQ)
*2.03 (PI-Q)I(QI-P)
*2.04 (PI(QIR))I(QI(PIR))
*2.05 (QIR)I((PIQ)I(PIR))
*2.06 (PIQ)I((QIR)I(PIR))
*2.07 PI(PVP)
*2.08 PIP
*2.10 -PVP
*2.11 PV-P
*2.12 PI--P
*2.13 PV---P
*2.14 --PIP
*2.15 (-PIQ)I(-QIP)
*2.20 PI(PVQ)
*2.21 -PI(PIQ)
*2.24 PI(-PIQ)
*3.13 (-(P*Q))I(-PV-Q)
*3.14 (-PV-Q)I(-(P*Q))
3.24 -(P-P)
*4.13 P=--P
*4.20 P=P
*4.24 P=(P*P)
*4.25 P=(PVP)

TO PROVE

*2.01 (PI-P)I-P
1. (-PV-P)I-P

, SUBLEVEL REPLACEMENT

PROOF FOUND.

GIVEN
SUBSTITUTION
GIVEN
SUBLEVEL REPLACEMENT
Q.E.D.

*1.2 (AVA)IA
1. (-PV-P)I-P
DEFINITIONS
*2.01 (PI-P)I-P

EFFORT LIMIT 200000
SUBPROBLEMS LIMIT 50
SUBSTITUTIONS LIMIT 50

ACTUAL 8062
ACTUAL 1
ACTUAL 2

REMEMBER PROVED THEOREM

*2.01 (AI-A)I-A

TO PROVE

*2.02 $QI(PIQ)$
1. $QI(\neg PVQ)$

, SUBLEVEL REPLACEMENT

PROOF FOUND.

GIVEN
SUBSTITUTION
GIVEN
SUBLEVEL REPLACEMENT
Q.E.D.

*1.3 $BI(AVB)$
1. $QI(\neg PVQ)$
DEFINITIONS
*2.02 $QI(PIQ)$

EFFORT
SUBPROBLEMS
SUBSTITUTIONS

LIMIT 200000
LIMIT 50
LIMIT 50

ACTUAL 6041
ACTUAL 1
ACTUAL 2

REMEMBER PROVED THEOREM

*2.02 $AI(BIA)$

TO PROVE

*2.03 (PI-Q)I(QI-P)
1. (-PV-Q)I(-QV-P)

, SUBLEVEL REPLACEMENT

PROOF FOUND.

GIVEN
SUBSTITUTION
GIVEN
SUBLEVEL REPLACEMENT
Q.E.D.

*1.4 (AVB)I(BVA)
1. (-PV-Q)I(-QV-P)
DEFINITIONS
*2.03 (PI-Q)I(QI-P)

EFFORT LIMIT 200000
SUBPROBLEMS LIMIT 50
SUBSTITUTIONS LIMIT 50

ACTUAL 11904
ACTUAL 1
ACTUAL 2

REMEMBER PROVED THEOREM

*2.03 (AI-B)I(BI-A)

TO PROVE

*2.04 (PI(QIR))I(QI(PIR))
 1. (PI(-QVR))I(QI(-PVR)) , SUBLEVEL REPLACEMENT
 2. (-PV(QIR))I(-QV(PIR)) , SUBLEVEL REPLACEMENT
 3. (-PV(-QVR))I(-QV(-PVR)) , SUBLEVEL REPLACEMENT

PROOF FOUND.

GIVEN	*1.5	(AV(BVC))I(BV(AVC))
SUBSTITUTION	3.	(-PV(-QVR))I(-QV(-PVR))
GIVEN		DEFINITIONS
SUBLEVEL REPLACEMENT	*2.04	(PI(QIR))I(QI(PIR))
Q.E.D.		

EFFORT	LIMIT 200000	ACTUAL 32262
SUBPROBLEMS	LIMIT 50	ACTUAL 3
SUBSTITUTIONS	LIMIT 50	ACTUAL 4

REMEMBER PROVED THEOREM

*2.04 (AI(BIC))I(BI(AIC))

TC PROVE

*2.05 (QIR)I((PIQ)I(PIR))
1. (QIR)I((-PVQ)I(-PVR)) , SUBLEVEL REPLACEMENT

PROOF FOUND.

GIVEN	*1.6	(BIC)I((AVB)I(AVC))
SUBSTITUTION	1.	(QIR)I((-PVQ)I(-PVR))
GIVEN		DEFINITIONS
SUBLEVEL REPLACEMENT	*2.05	(QIR)I((PIQ)I(PIR))
Q.E.D.		

EFFORT	LIMIT 200000	ACTUAL 13440
SUBPROBLEMS	LIMIT 50	ACTUAL 1
SUBSTITUTIONS	LIMIT 50	ACTUAL 2

REMEMBER PROVED THEOREM

*2.05 (AIB)I((CIA)I(CIB))

TO PROVE

*2.06 (PIQ)I((QIR)I(PIR))
 1. (PIQ)I((-QVR)I(-PVR)) , SUBLEVEL REPLACEMENT
 2. (-PVQ)I((-QIR)V(PIR)) , SUBLEVEL REPLACEMENT
 3. (-PVQ)I((-QVR)V(-PVR)) , SUBLEVEL REPLACEMENT
 4. (-(PIQ)V((QIR)I(PIR))) *1.01, REPLACEMENT
 5. (QIR)I((PIQ)I(PIR)) *2.04, DETACHMENT

PROOF FOUND.

GIVEN	*2.05	(AIB)I((CIA)I(CIP))
SUBSTITUTION	5.	(QIR)I((PIQ)I(PIR))
GIVEN	*2.04	(AI(BIC))I(BI(AIC))
DETACHMENT	*2.06	(PIQ)I((QIR)I(PIR))
Q.E.D.		

EFFORT	LIMIT 200000	ACTUAL 46665
SUBPROBLEMS	LIMIT 50	ACTUAL 5
SUBSTITUTIONS	LIMIT 50	ACTUAL 6

REMEMBER PROVED THEOREM

*2.06 (AIB)I((BIC)I(AIC))

TO PROVE
*2.07 PI(PVP)

PROOF FOUND.

GIVEN
SUBSTITUTION
Q.E.D.

*1.3 BI(AVB)
*2.07 PI(PVP)

EFFORT LIMIT 200000
SUBPROBLEMS LIMIT 50
SUBSTITUTIONS LIMIT 50

ACTUAL 2623
ACTUAL 0
ACTUAL 1

REMEMBER PROVED THEOREM

*2.07 AI(AVA)

TC PROVE

*2.08 PIP

1. -PVP
5088 P
5193 (PIP)V(PIP)
2. (PVP)IP

*1.01, REPLACEMENT
*2.02, DETACHMENT. REJECTED PROBLEM
*1.2, DETACHMENT. REJECTED PROBLEM
*2.07, FORWARD CHAINING

PROOF FOUND.

GIVEN
SUBSTITUTION
GIVEN
FORWARD CHAINING
Q.E.D.

*1.2 (AVA)IA
2. (PVP)IP
*2.07 AI(AVA)
*2.08 PIP

EFFORT LIMIT 200000
SUBPROBLEMS LIMIT 50
SUBSTITUTIONS LIMIT 50

ACTUAL 8998
ACTUAL 2
ACTUAL 3

REMEMBER PROVED THEOREM

*2.08 AIA

TO PROVE

*2.10 -PVP
1. PIP

*1.01, REPLACEMENT

PROOF FOUND.

GIVEN
SUBSTITUTION
GIVEN
REPLACEMENT
Q.E.D.

*2.08 AIA
1. PIP
*1.01 (AIB).=(-AVB) DEF.
*2.10 -PVP

EFFORT LIMIT 200000
SUBPROBLEMS LIMIT 50
SUBSTITUTIONS LIMIT 50

ACTUAL 5150
ACTUAL 1
ACTUAL 2

REMEMBER PROVED THEOREM

*2.10 -AVA

TO PROVE

*2.11 PV-P
1. -PVP

*1.4, DETACHMENT

PROOF FOUND.

GIVEN
SUBSTITUTION
GIVEN
DETACHMENT
Q.E.D.

*2.10 -AVA
1. -PVP
*1.4 (AVB)I(BVA)
*2.11 PV-P

EFFORT
SUBPROBLEMS LIMIT 200000
SUBSTITUTIONS LIMIT 50
SUBSTITUTIONS LIMIT 50

ACTUAL 6190
ACTUAL 1
ACTUAL 2

REMEMBER PROVED THEOREM

*2.11 AV-A

TO PROVE

*2.12 PI--P

1. -PV--P

*1.01, REPLACEMENT

PROOF FOUND.

GIVEN
SUBSTITUTION
GIVEN
REPLACEMENT
Q.E.D.

*2.11 AV-A
1. -PV--P
*1.01 (A1B).=.(-AVB) DEF.
*2.12 PI--P

EFFORT LIMIT 200000
SUBPROBLEMS LIMIT 50
SUBSTITUTIONS LIMIT 50

ACTUAL 8215
ACTUAL 1
ACTUAL 2

REMEMBER PROVED THEOREM

*2.12 A1--A

TO PROVE

*2.13 PV---P
1. ---PVP
5062 ---P
5304 PV---P
5199 (PV---P)V(PV---P)
2. -PI---P

*1.4, DETACHMENT
*1.3, DETACHMENT. REJECTED PROBLEM
*2.08, DETACHMENT. REJECTED PROBLEM
*1.2, DETACHMENT. REJECTED PROBLEM
*2.11, FORWARD CHAINING

PROOF FOUND.

GIVEN
SUBSTITUTION
GIVEN
FORWARD CHAINING
Q.E.D.

*2.12 AI--A
2. -PI---P
*2.11 AV-A
*2.13 PV---P

EFFORT LIMIT 200000
SUBPROBLEMS LIMIT 50
SUBSTITUTIONS LIMIT 50

ACTUAL 18263
ACTUAL 2
ACTUAL 3

REMEMBER PROVED THEOREM

*2.13 AV---A

TO PROVE

*2.14	--PIP	
1.	---PVP	*1.01, REPLACEMENT
5187	P	*2.02, DETACHMENT. REJECTED PROBLEM
5090	--PIP	*2.08, DETACHMENT. REJECTED PROBLEM
5149	((--PIP)V(--PIP))	*1.2, DETACHMENT. REJECTED PROBLEM
2.	----PIP	*2.12, FORWARD CHAINING
5339	--PIP	*2.08, FORWARD CHAINING. REJECTED PROBLEM
3.	((--PV--P)IP	*2.07, FORWARD CHAINING
4.	(BI--P)IP	*2.02, FORWARD CHAINING
5.	(AV--P)IP	*1.3, FORWARD CHAINING
5379	--PIP	*2.08, BACKWARD CHAINING. REJECTED PROBLEM
6.	--PI(PVP)	*1.2, BACKWARD CHAINING
7.	---PV(PVP)	*1.01, REPLACEMENT
5051	PVP	*2.02, DETACHMENT. REJECTED PROBLEM
5245	--PI(PVP)	*2.08, DETACHMENT. REJECTED PROBLEM
5552	((--PI(PVP))V(--PI(PVP)))	*1.2, DETACHMENT. REJECTED PROBLEM
8.	----PI(PVP)	*2.12, FORWARD CHAINING
5612	--PI(PVP)	*2.08, FORWARD CHAINING. REJECTED PROBLEM
9.	((--PV--P)I(PVP)	*2.07, FORWARD CHAINING
10.	(BI--P)I(PVP)	*2.02, FORWARD CHAINING
11.	(AV--P)I(PVP)	*1.3, FORWARD CHAINING
5500	--PIP	*2.07, BACKWARD CHAINING. REJECTED PROBLEM
5737	--PI(PVP)	*1.4, BACKWARD CHAINING. REJECTED PROBLEM
5780	--PIP	*1.3, BACKWARD CHAINING. REJECTED PROBLEM
5653	--PI(PVP)	*2.08, BACKWARD CHAINING. REJECTED PROBLEM
12.	--PI((PVP)V(PVP))	*1.2, BACKWARD CHAINING
12.	--PI((PVP)V(PVP))	
13.	---PV((PVP)V(PVP))	*1.01, REPLACEMENT
5286	(PVP)V(PVP)	*2.02, DETACHMENT. REJECTED PROBLEM
4842	--PI((PVP)V(PVP))	*2.08, DETACHMENT. REJECTED PROBLEM
5854	((--PI((PVP)V(PVP)))V(--PI((PVP)V(PVP))))	*1.2, DETACHMENT. REJECTED PROBLEM
14.	----PI((PVP)V(PVP))	*2.12, FORWARD CHAINING
5846	--PI((PVP)V(PVP))	*2.08, FORWARD CHAINING. REJECTED PROBLEM
15.	((--PV--P)I((PVP)V(PVP))	*2.07, FORWARD CHAINING
16.	(BI--P)I((PVP)V(PVP))	*2.02, FORWARD CHAINING
17.	(AV--P)I((PVP)V(PVP))	*1.3, FORWARD CHAINING
6031	--PI(PVP)	*2.07, BACKWARD CHAINING. REJECTED PROBLEM
18.	--PI(PV((PVP)V(PVP))	*1.5, BACKWARD CHAINING
6059	--PI((PVP)V(PVP))	*1.4, BACKWARD CHAINING. REJECTED PROBLEM
6092	--PI(PVP)	*1.3, BACKWARD CHAINING. REJECTED PROBLEM
5995	--PI((PVP)V(PVP))	*2.08, BACKWARD CHAINING. REJECTED PROBLEM
19.	--PI(((PVP)V(PVP))V((PVP)V(PVP)))	*1.2, BACKWARD CHAINING
1.	---PVP	
6102	--PIP	*1.01, REPLACEMENT. REJECTED PROBLEM
20.	PV---P	*1.4, DETACHMENT

PROOF FOUND.

GIVEN	*2.13	AV---A
SUBSTITUTION	20.	PV---P
GIVEN	*1.4	(AVB)I(BVA)
DETACHMENT	1.	---PVP
GIVEN	*1.01	(AIB).=(-AVB) DFF.
REPLACEMENT	*2.14	--PIP
Q.E.D.		

EFFORT	LIMIT 200000	ACTUAL 181261
SUBPROBLEMS	LIMIT 50	ACTUAL 20
SUBSTITUTIONS	LIMIT 50	ACTUAL 21

REMEMBER PROVED THEOREM

*2.14 --A1A

TO PROVE

```

*2.15  (-PIQ)I(-QIP)
1.      (---PVQ)I(---QVP)          , SUBLEVEL REPLACEMENT
5126    (---PVQ)I(---QVP)          , SUBLEVEL REPLACEMENT. REJECTED PROBLEM
2.      (-(-PIQ))V(-QIP)          *1.01, REPLACEMENT
3.      -QI((-PIQ)IP)              *2.04, DETACHMENT
4.      -QIP                        *2.02, DETACHMENT
5.      -((-((-PIQ)I(-QIP)))      *2.14, DETACHMENT
5539    (-PIQ)I(-QIP)              *2.08, DETACHMENT. REJECTED PROBLEM
5273    ((-PIQ)I(-QIP))V((-PIQ)I(-QIP)) *1.2, DETACHMENT. REJECTED PROBLEM
6.      ((QIC)I(-PIC))I(-QIP)      *2.06, FORWARD CHAINING
7.      ((CI-P)I(CIQ))I(-QIP)      *2.05, FORWARD CHAINING
8.      ((AV-P)I(AVQ))I(-QIP)      *1.6, FORWARD CHAINING
9.      (-((-PIQ))I(-QIP)          *2.12, FORWARD CHAINING
6071    (-PIQ)I(-QIP)              *2.08, FORWARD CHAINING. REJECTED PROBLEM
10.     ((-PIQ)V(-PIQ))I(-QIP)      *2.07, FORWARD CHAINING
11.     (BI(-PIQ))I(-QIP)           *2.02, FORWARD CHAINING
12.     (AV(-PIQ))I(-QIP)           *1.3, FORWARD CHAINING
13.     (-PIQ)IP                    *2.02, BACKWARD CHAINING
14.     (-PIQ)I(-((-QIP)))          *2.14, BACKWARD CHAINING
5757    (-PIQ)I(-QIP)              *2.08, BACKWARD CHAINING. REJECTED PROBLEM
15.     (-PIQ)I((-QIP)V(-QIP))      *1.2, BACKWARD CHAINING

4.      -QIP
16.     --QVP                      *1.01, REPLACEMENT
5882    P                          *2.02, DETACHMENT. REJECTED PROBLEM
17.     -((-(-QIP))                *2.14, DETACHMENT
6171    -QIP                        *2.08, DETACHMENT. REJECTED PROBLEM
5396    (-QIP)V(-QIP)              *1.2, DETACHMENT. REJECTED PROBLEM
18.     ---QIP                     *2.12, FORWARD CHAINING
5396    -QIP                        *2.08, FORWARD CHAINING. REJECTED PROBLEM
19.     (-QV-Q)IP                  *2.07, FORWARD CHAINING
20.     (BI-Q)IP                    *2.02, FORWARD CHAINING
21.     (AV-Q)IP                    *1.3, FORWARD CHAINING
22.     -QI--P                     *2.14, BACKWARD CHAINING
6183    -QIP                        *2.08, BACKWARD CHAINING. REJECTED PROBLEM
23.     -QI(PVP)                   *1.2, BACKWARD CHAINING

23.     -QI(PVP)
24.     --QV(PVP)                  *1.01, REPLACEMENT
5836    PVP                         *2.02, DETACHMENT. REJECTED PROBLEM
25.     -((-(-QI(PVP)))            *2.14, DETACHMENT
6364    -QI(PVP)                    *2.08, DETACHMENT. REJECTED PROBLEM
6420    (-QI(PVP))V(-QI(PVP))      *1.2, DETACHMENT. REJECTED PROBLEM
26.     ---QI(PVP)                 *2.12, FORWARD CHAINING
6459    -QI(PVP)                    *2.08, FORWARD CHAINING. REJECTED PROBLEM
27.     (-QV-Q)I(PVP)              *2.07, FORWARD CHAINING
28.     (BI-Q)I(PVP)                *2.02, FORWARD CHAINING
29.     (AV-Q)I(PVP)                *1.3, FORWARD CHAINING
6580    -QIP                        *2.07, BACKWARD CHAINING. REJECTED PROBLEM
6601    -QI(PVP)                    *1.4, BACKWARD CHAINING. REJECTED PROBLEM
6563    -QIP                        *1.3, BACKWARD CHAINING. REJECTED PROBLEM
30.     -QI((-(-PVP)))              *2.14, BACKWARD CHAINING
6627    -QI(PVP)                    *2.08, BACKWARD CHAINING. REJECTED PROBLEM
31.     -QI((PVP)V(PVP))            *1.2, BACKWARD CHAINING

```

NO PROOF FOUND

EFFORT	LIMIT 200000	ACTUAL 206728
SUBPROBLEMS	LIMIT 50	ACTUAL 31
SUBSTITUTIONS	LIMIT 50	ACTUAL 32

TO PROVE

*2.20 PI(PVQ)

1. -PV(PVQ)
2. PVQ
3. $\neg(\neg(\text{PI(PVQ)}))$
- 5676 PI(PVQ)
- 5675 $(\text{PI(PVQ)})V(\text{PI(PVQ)})$
4. $\neg\text{PI(PVQ)}$
- 6012 PI(PVQ)
5. $(\text{PVP})I(\text{PVQ})$
6. $(\text{BIP})I(\text{PVQ})$
7. $(\text{AVP})I(\text{PVQ})$

- *1.01, REPLACEMENT
- *2.02, DETACHMENT
- *2.14, DETACHMENT
- *2.08, DETACHMENT. REJECTED PROBLEM
- *1.2, DETACHMENT. REJECTED PROBLEM
- *2.12, FORWARD CHAINING
- *2.08, FORWARD CHAINING. REJECTED PROBLEM
- *2.07, FORWARD CHAINING
- *2.02, FORWARD CHAINING
- *1.3, FORWARD CHAINING

PROOF FOUND.

GIVEN
SUBSTITUTION
GIVEN
FORWARD CHAINING
Q.E.D.

- *1.4 $(\text{AVB})I(\text{BVA})$
- 7. $(\text{CVP})I(\text{PVQ})$
- *1.3 BI(AVB)
- *2.20 PI(PVQ)

EFFORT LIMIT 200000
SUBPROBLEMS LIMIT 50
SUBSTITUTIONS LIMIT 50

ACTUAL 38111
ACTUAL 7
ACTUAL 8

REMEMBER PROVED THEOREM

*2.20 AI(AVB)

-99-

TO PROVE

*2.21 $\neg \text{PI}(\text{PIQ})$
1. $\neg \text{PI}(\neg \text{PVQ})$

, SUBLEVEL REPLACEMENT

PROOF FOUND.

GIVEN
SUBSTITUTION
GIVEN
SUBLEVEL REPLACEMENT
Q.E.D.

*2.20 $\text{AI}(\text{AVB})$
1. $\neg \text{PI}(\neg \text{PVQ})$
DEFINITIONS
*2.21 $\neg \text{PI}(\text{PIQ})$

EFFORT
SUBPROBLEMS
SUBSTITUTIONS

LIMIT 200000
LIMIT 50
LIMIT 50

ACTUAL 8021
ACTUAL 1
ACTUAL 2

REMEMBER PROVED THEOREM

*2.21 $\neg \text{AI}(\text{AIB})$

-100-

TO PROVE

*2.24 $\text{PI}(\neg\text{PIQ})$

1. $\text{PI}(\neg\text{PVQ})$
4942 $\text{PI}(\neg\text{PVQ})$
2. $\neg\text{PV}(\neg\text{PIQ})$
5291 $\neg\text{P}$
3. $\neg\text{PI}(\text{PIQ})$

, SUBLEVEL REPLACEMENT
, SUBLEVEL REPLACEMENT. REJECTED PROBLEM
*1.01, REPLACEMENT
*2.21, DETACHMENT. REJECTED PROBLEM
*2.04, DETACHMENT

PROOF FOUND.

GIVEN
SUBSTITUTION
GIVEN
DETACHMENT
Q.E.D.

*2.21 $\neg\text{AI}(\text{AIB})$
3. $\neg\text{PI}(\text{PIQ})$
*2.04 $(\text{AI}(\text{BIC}))\text{I}(\text{BI}(\text{AIC}))$
*2.24 $\text{PI}(\neg\text{PIQ})$

EFFORT LIMIT 200000
SUBPROBLEMS LIMIT 50
SUBSTITUTIONS LIMIT 50

ACTUAL 21521
ACTUAL 3
ACTUAL 4

REMEMBER PROVED THEOREM

*2.24 $\text{AI}(\neg\text{AIB})$

TO PROVE

*3.13 $(\neg(P \cdot Q))I(\neg PV - Q)$
1. $(\neg(\neg(\neg PV - Q)))I(\neg PV - Q)$

SUBLEVEL REPLACEMENT

PROOF FOUND.

GIVEN
SUBSTITUTION
GIVEN
SUBLEVEL REPLACEMENT
Q.E.D.

*2.14 --AIA
1. $(\neg(\neg(\neg PV - Q)))I(\neg PV - Q)$
DEFINITIONS
*3.13 $(\neg(P \cdot Q))I(\neg PV - Q)$

EFFORT
SUBPROBLEMS
SUBSTITUTIONS

LIMIT 200000
LIMIT 50
LIMIT 50

ACTUAL 13293
ACTUAL 1
ACTUAL 2

REMEMBER PROVED THEOREM

*3.13 $(\neg(A \cdot B))I(\neg AV - B)$

TO PROVE

*3.14 $(\neg PV-Q)I(\neg(P*Q))$
 1. $(\neg PV-Q)I(\neg(\neg(\neg PV-Q)))$

SUBLEVEL REPLACEMENT

PROOF FOUND.

GIVEN
 SUBSTITUTION
 GIVEN
 SUBLEVEL REPLACEMENT
 Q.E.D.

*2.12 AI--A
 1. $(\neg PV-Q)I(\neg(\neg(\neg PV-Q)))$
 DEFINITIONS
 *3.14 $(\neg PV-Q)I(\neg(P*Q))$

EFFORT LIMIT 200000
 SUBPROBLEMS LIMIT 50
 SUBSTITUTIONS LIMIT 50

ACTUAL 12753
 ACTUAL 1
 ACTUAL 2

REMEMBER PROVED THEOREM

*3.14 $(\neg AV-B)I(\neg(A*B))$

TO PROVE

*3.24 $-(P \leftrightarrow P)$

1. $-(\neg(\neg PV \leftrightarrow P))$

5417 $-(\neg(\neg PV \leftrightarrow P))$

2. $\neg PV \leftrightarrow P$

, SUBLEVEL REPLACEMENT

, SUBLEVEL REPLACEMENT. REJECTED PROBLEM

*3.14, DETACHMENT

PROOF FOUND.

GIVEN
SUBSTITUTION

GIVEN
DETACHMENT
Q.E.D.

*2.11

AV-A

2.

$\neg PV \leftrightarrow P$

*3.14

$(\neg AV \leftrightarrow B) \vee (\neg(A \leftrightarrow B))$

*3.24

$-(P \leftrightarrow P)$

EFFORT LIMIT 200000
SUBPROBLEMS LIMIT 50
SUBSTITUTIONS LIMIT 50

ACTUAL 14541
ACTUAL 2
ACTUAL 3

REMEMBER PROVED THEOREM

*3.24 $-(A \leftrightarrow A)$

TO PROVE

*4.13 $P \equiv \neg P$

1. $(P \vee \neg P) \wedge (\neg P \vee P)$
2. $\neg(\neg(P \equiv \neg P))$
- 5588 $P \equiv \neg P$
- 5260 $(P \equiv \neg P) \vee (P \equiv \neg P)$
3. PIP

*4.01, REPLACEMENT
*2.14, DETACHMENT
*2.08, DETACHMENT. REJECTED PROBLEM
*1.2, DETACHMENT. REJECTED PROBLEM
*2.12, BACKWARD CHAINING

PROOF FOUND.

GIVEN
SUBSTITUTION
GIVEN
BACKWARD CHAINING
Q.E.D.

*2.08 AIA
3. PIP
*2.12 AI--A
*4.13 $P \equiv \neg P$

EFFORT LIMIT 200000
SUBPROBLEMS LIMIT 50
SUBSTITUTIONS LIMIT 50

ACTUAL 15965
ACTUAL 3
ACTUAL 4

REMEMBER PROVED THEOREM

*4.13 $A \equiv \neg A$

TO PROVE

*4.20 $P=P$

1. $(PIP) * (PIP)$
2. $\neg(\neg(P=P))$
- 6082 $P=P$
- 5765 $(P=P) \vee (P=P)$
3. $\neg PIP$

*4.01, REPLACEMENT

*2.14, DETACHMENT

*2.08, DETACHMENT. REJECTED PROBLEM

*1.2, DETACHMENT. REJECTED PROBLEM

*4.13, FORWARD CHAINING

PROOF FOUND.

GIVEN
SUBSTITUTION
GIVEN
FORWARD CHAINING
Q.E.D.

*2.14 $\neg AIA$
3. $\neg PIP$
*4.13 $A \equiv \neg A$
*4.20 $P=P$

EFFORT LIMIT 200000
SUBPROBLEMS LIMIT 50
SUBSTITUTIONS LIMIT 50

ACTUAL 12404
ACTUAL 3
ACTUAL 4

REMEMBER PROVED THEOREM

*4.20 $A=A$

TO PROVE

*4.24	P=(P*P)		
1.	P=(-(-PV-P))		, SUBLEVEL REPLACEMENT
5377	P=(-(-PV-P))		, SUBLEVEL REPLACEMENT. REJECTED PROBLEM
2.	(PI(P*P))*((P*P)IP)	*4.01,	REPLACEMENT
3.	-(-P=(P*P))	*2.14,	DETACHMENT
5457	P=(P*P)	*2.08,	DETACHMENT. REJECTED PROBLEM
5321	(P=(P*P))V(P=(P*P))	*1.2,	DETACHMENT. REJECTED PROBLEM
4.	PI(P*P)	*4.20,	FORWARD CHAINING
5.	--PI(P*P)	*4.13,	FORWARD CHAINING
6.	PI(-(-P*P))	*2.14,	BACKWARD CHAINING
6563	PI(P*P)	*2.08,	BACKWARD CHAINING. REJECTED PROBLEM
7.	PI((P*P)V(P*P))	*1.2,	BACKWARD CHAINING
4.	PI(P*P)		
8.	-PV(P*P)	*1.01,	REPLACEMENT
6597	-P	*2.21,	DETACHMENT. REJECTED PROBLEM
9.	P*P	*2.02,	DETACHMENT
10.	-(-PI(P*P))	*2.14,	DETACHMENT
5348	PI(P*P)	*2.08,	DETACHMENT. REJECTED PROBLEM
6056	(PI(P*P))V(PI(P*P))	*1.2,	DETACHMENT. REJECTED PROBLEM
11.	(-PIB)I(P*P)	*2.24,	FORWARD CHAINING
12.	(PVB)I(P*P)	*2.20,	FORWARD CHAINING
6514	--PI(P*P)	*2.12,	FORWARD CHAINING. REJECTED PROBLEM
6235	PI(P*P)	*2.08,	FORWARD CHAINING. REJECTED PROBLEM
13.	(PVP)I(P*P)	*2.07,	FORWARD CHAINING
14.	(BIP)I(P*P)	*2.02,	FORWARD CHAINING
15.	(AVP)I(P*P)	*1.3,	FORWARD CHAINING
5907	PI(-(-P*P))	*2.14,	BACKWARD CHAINING. REJECTED PROBLEM
6607	PI(P*P)	*2.08,	BACKWARD CHAINING. REJECTED PROBLEM
5352	PI((P*P)V(P*P))	*1.2,	BACKWARD CHAINING. REJECTED PROBLEM
9.	P*P		
6477	-(-PV-P)	*3.01,	REPLACEMENT. REJECTED PROBLEM
16.	-(-P*P)	*2.14,	DETACHMENT
5352	P*P	*2.08,	DETACHMENT. REJECTED PROBLEM
6484	(P*P)V(P*P)	*1.2,	DETACHMENT. REJECTED PROBLEM
17.	PI--P	*2.14,	BACKWARD CHAINING

PROOF FOUND.

GIVEN	*2.12	AI--A
SUBSTITUTION	17.	PI--P
GIVEN	*2.14	--AIA
BACKWARD CHAINING	9.	P*P
GIVEN	*2.02	AI(BIA)
DETACHMENT	4.	PI(P*P)
GIVEN	*4.20	A=A
FORWARD CHAINING	*4.24	P=(P*P)
Q.E.D.		

EFFORT	LIMIT 200000	ACTUAL 92331
SUBPROBLEMS	LIMIT 50	ACTUAL 17
SUBSTITUTIONS	LIMIT 50	ACTUAL 18

REMEMBER PROVED THEOREM

*4.24 A=(A*A)

TO PROVE

*4.25 $P=(PVP)$

1. $(PI(PVP))*((PVP)IP)$
2. $-(-(P=(PVP)))$
- 5528 $P=(PVP)$
- 5797 $(P=(PVP))V(P=(PVP))$
3. $(P*P)I(PVP)$
4. $PI(PVP)$

- *4.01, REPLACEMENT
- *2.14, DETACHMENT
- *2.08, DETACHMENT. REJECTED PROBLEM
- *1.2, DETACHMENT. REJECTED PROBLEM
- *4.24, FORWARD CHAINING
- *4.20, FORWARD CHAINING

PROOF FOUND.

GIVEN
SUBSTITUTION
GIVEN
FORWARD CHAINING
Q.E.D.

*2.20 $AI(AVB)$
4. $PI(PVP)$
*4.20 $A=A$
*4.25 $P=(PVP)$

EFFORT LIMIT 200000
SUBPROBLEMS LIMIT 50
SUBSTITUTIONS LIMIT 50

ACTUAL 20788
ACTUAL 4
ACTUAL 5

REMEMBER PROVED THEOREM

*4.25 $A=(AVA)$

PROGRAM RAN TO COMPLETION.

IPL-V POST-MORTEM

H0	*1	
	*2	
	*3	
	*4	
H1	J7	
H2	16207 WORDS	
H3		879155
H4	0	
H5	J4	
H6	0	
H7		0
H8	0	
H9	0	
H10	0	
H11	0	
H12	0	
W0	0	
W1	0	
W2	0	
W3	0	
W4	0	
W5	0	
W6	0	
W7	0	
W8	0	
W9	0	
W10	24627	1
W11		0
W12	X13	
W13	X13	
W14	J0	
W15	X15	
W16		
W17		
W18	24628	0
W19	24629	0
W20	28153	0
	24630	0
W21	28088	1
	24631	1
W22	28088	1
	24632	1
W23	28090	
W24	24633	
W25	24754	16
W26	24756	
	0	
	J0	
W27		
W28		
W29		
W30	24755	1
W31	24759	2

W32 0
W33
24153 0
24152 0
24150 0
24151 0
24156 0
24145 0
24154 0
24155 0
24160 0
24157 0
24159 1
24158 1
32531 32548*

*4.25 A=(AVA)

THE FOLLOWING RESULTED FROM EXECUTING 1W15...

L4 0
=0
9-1
-0
9-2
V0
9-3
10
9-4
=1
9-5
5916* 9-1 0
9-6
9-7
4849* 9-2 0
9-8
5095* 9-3 0
9-9
9-10
4247* 9-4 0
9-11
9-12
4054* 9-5 0
9-13
9-14
5707* 9-6 9-15
5719* 9-7 9-16
V0
9-17
*0
9-18
-0
9-19
5398* 9-8 0
*0
9-20
5059* 9-9 9-21
-0
9-22
5138* 9-10 9-23
-0

4239*	9-11	9-24
		9-25
		-0
		9-26
		10
		9-27
		V0
4209*	9-12	9-28
		9-29
		-0
		9-30
		10
		9-31
		V0
4077*	9-13	9-32
		0
		=0
		9-33
		*0
		9-34
		10
4051*	9-14	9-35
		0
		*0
		9-36
		-0
		9-37
		V0
5336*	9-15	9-38
		0
		*425
		*424
		*420
		*413
5037*	9-16	0
		*420
5789*	9-17	0
		9-39
		9-40
5765*	9-18	0
		9-41
		9-42
5454*	9-19	0
		9-43
6035*	9-20	0
		9-44
		9-45
5173*	9-21	0
		*213
		*211
5262*	9-22	0
		9-46
5013*	9-23	0
		*210
5066*	9-24	0
		9-47
4270*	9-25	0
		*224
		*220

		*212
		*208
		*207
		*202
		*13
5109*	9-26	0
		9-48
4368*	9-27	0
		9-49
		9-50
4250*	9-28	0
		9-51
		9-52
4242*	9-29	0
		*214
		*208
		*12
4875*	9-30	0
		9-53
4371*	9-31	0
		9-54
		9-55
4253*	9-32	0
		9-56
		9-57
4181*	9-33	0
		9-58
		9-59
4131*	9-34	0
		9-60
		9-61
4066*	9-35	0
		9-62
		9-63
4204*	9-36	0
		9-64
		9-65
4113*	9-37	0
		9-66
4069*	9-38	0
		9-67
		9-68
5585*	9-39	9-69
5771*	9-40	9-70
6331*	9-41	9-71
6109*	9-42	9-72
5534*	9-43	0
		-0
		9-73
5819*	9-44	9-74
5411*	9-45	0
		-0
		9-75
5010*	9-46	9-76
4983*	9-47	9-77
		-0
		9-78
5217*	9-48	9-79
		*0

		9-80
		-0
		9-81
4300*	9-49	9-82
4367*	9-50	9-83
		10
		9-84
		-0
		9-85
4212*	9-51	9-86
		-0
		9-87
4249*	9-52	9-88
		-0
		9-89
		V0
		9-90
4860*	9-53	9-91
		*0
		9-92
		-0
		9-93
4370*	9-54	9-94
		-0
		9-95
		10
		9-96
		V0
		9-97
4350*	9-55	9-98
		10
		9-99
		-0
		9-100
		V0
		9-101
4278*	9-56	9-102
		-0
		9-103
4276*	9-57	9-104
		-0
		9-105
		V0
		9-106
4202*	9-58	9-107
4165*	9-59	9-108
4133*	9-60	9-109
4112*	9-61	9-110
4061*	9-62	9-111
4046*	9-63	9-112
4116*	9-64	0
		10
		9-113
4178*	9-65	0
		10
		9-114
4109*	9-66	0
		V0
		9-115

4073*	9-67	0
		-0
		9-116
4082*	9-68	9-117
5266*	9-69	0
		*425
5274*	9-70	0
		*425
5609*	9-71	0
		*424
6002*	9-72	0
		*424
6077*	9-73	0
		9-118
6107*	9-74	0
		*324
5417*	9-75	0
		9-119
5136*	9-76	0
		*210
4995*	9-77	0
		*211
5080*	9-78	0
		9-120
5419*	9-79	0
		*221
5608*	9-80	0
		9-121
		9-122
5089*	9-81	0
		9-123
4369*	9-82	0
		*206
		*205
		*204
		*203
		*201
		*16
4257*	9-83	0
		*206
		*205
		*16
5054*	9-84	0
		9-124
		9-125
4956*	9-85	0
		9-126
4254*	9-86	0
		*15
		*14
		*12
5821*	9-87	0
		9-127
4256*	9-88	0
		*14
		*12
4795*	9-89	0
		9-128
4295*	9-90	0

		9-129
		9-130
4871*	9-91	0
		*201
5384*	9-92	0
		9-131
		9-132
5143*	9-93	0
		9-133
4828*	9-94	0
		*221
		*204
		*203
		*202
5950*	9-95	0
		9-134
4979*	9-96	0
		9-135
		9-136
4404*	9-97	0
		9-137
		9-138
4889*	9-98	0
		*224
		*221
		*202
4836*	9-99	0
		9-139
		9-140
4835*	9-100	0
		9-141
4407*	9-101	0
		9-142
		9-143
4283*	9-102	0
		*220
		*207
		*15
		*14
		*13
5549*	9-103	0
		9-144
4285*	9-104	0
		*220
		*207
		*14
		*13
5292*	9-105	0
		9-145
4352*	9-106	0
		9-146
		9-147
4205*	9-107	0
		*401
4176*	9-108	0
		*401
4140*	9-109	0
		*301
4139*	9-110	0

4050*	9-111	*301
		0
		*101
4078*	9-112	0
		*101
4210*	9-113	0
		9-148
		9-149
4215*	9-114	0
		9-150
		9-151
4148*	9-115	0
		9-152
		9-153
4076*	9-116	0
		9-154
4075*	9-117	0
		*101
5854*	9-118	9-155
5579*	9-119	9-156
4910*	9-120	0
		-0
		9-157
5604*	9-121	9-158
5683*	9-122	9-159
6080*	9-123	9-160
4883*	9-124	9-161
4947*	9-125	9-162
4838*	9-126	9-163
6122*	9-127	9-164
5373*	9-128	9-165
4284*	9-129	9-166
4274*	9-130	9-167
5473*	9-131	9-168
5542*	9-132	9-169
5038*	9-133	9-170
6130*	9-134	9-171
5125*	9-135	9-172
5113*	9-136	9-173
4401*	9-137	9-174
4403*	9-138	9-175
5070*	9-139	9-176
4973*	9-140	9-177
4969*	9-141	9-178
4406*	9-142	9-179
4412*	9-143	9-180
5659*	9-144	9-181
5802*	9-145	9-182
4345*	9-146	9-183
4324*	9-147	9-184
4213*	9-148	9-185
4199*	9-149	9-186
4214*	9-150	9-187
4220*	9-151	9-188
4137*	9-152	0
		-0
		9-189
4145*	9-153	0
		-0

4092*	9-154	9-190
6091*	9-155	9-191
		0
		*413
5452*	9-156	0
		*324
5114*	9-157	0
		9-192
5397*	9-158	0
		*313
5428*	9-159	0
		*313
5909*	9-160	0
		*214
4948*	9-161	0
		*204
5058*	9-162	0
		*204
4924*	9-163	0
		*203
		*201
5801*	9-164	0
		*314
6079*	9-165	0
		*314
4323*	9-166	0
		*15
4346*	9-167	0
		*15
5690*	9-168	0
		*314
5599*	9-169	0
		*314
4960*	9-170	0
		*212
6110*	9-171	0
		*224
5166*	9-172	0
		*206
		*205
5133*	9-173	0
		*206
		*205
4408*	9-174	0
		*16
4410*	9-175	0
		*16
5160*	9-176	0
		*206
		*205
		*204
4978*	9-177	0
		*206
		*205
		*204
4903*	9-178	0
		*203
4419*	9-179	0
		*16

4421*	9-180	0
		*16
4807*	9-181	0
		*313
5917*	9-182	0
		*313
4356*	9-183	0
		*15
4358*	9-184	0
		*15
4216*	9-185	0
		*401
4218*	9-186	0
		*401
4227*	9-187	0
		*401
4229*	9-188	0
		*401
4134*	9-189	0
		9-193
4154*	9-190	0
		9-194
4097*	9-191	0
		*101
5207*	9-192	9-195
4158*	9-193	9-196
4163*	9-194	9-197
5261*	9-195	0
		*213
4161*	9-196	0
		*301
4168*	9-197	0
		*301

ACCOUNTING SUMMARY

06/05/63

JOB 8168,LTNEW1,EAS826,5MIN,0,099,C

STEFFERUD

1 003650

PHASE	PROGRAM	TIME
1	IPL	04MIN 30SEC
	TOTAL	04MIN 30SEC

MEDIARY OUTPUT COUNT 10 WORDS
END OF MOCK-DONALD SYSPIT

XIII. LEVELS OF VOCABULARY AND USE OF SYMBOLS

The following list summarizes LT's use of symbols for routines and data.

A0,B0,----,G0	Free variables
P0,Q0,----,T0	Bound variables
I0,V0,-0,*0,=0,=1	Connectives
Other zeroth symbols	Character symbols (except H0,J0,W0)
/1,/2,----,/14	Substitute character symbols
N1,N2,----,N10	Integer data terms 1,2,----,0
K1-K99	Constants and parameters
L1-L49	Control and working lists
T1-T39	Text lists
Q1-Q49	Attributes of terms and expressions
P1-P99	Routines to operate on terms and expressions
M1-M199	Logic theorist routines
M1---M9	Executives, setup
M10--M19	Methods
M40--M49	Utility measures
M50--M59	Information recorders
M60--M69	Information retrievers
M70--M89	Input-output routines
M90--M99	Limit testers
M110-M119	Match processes
X1-X49	Run housekeeping and executives
X1---X9	Executives
X10--X19	Debugging routines
X20--X29	Debugging lists

LIST OF IPL-V BASIC PROCESSES

* Indicates processes which set H5

General Processes

J0 No operation
J1 Execute (0) after restoring H0
*J2 TEST (0) = (1)
*J3 Set H5-
*J4 Set H5+
*J5 Reverse sense of H5
J6 Reverse (0) and (1)
J7 Halt, proceed on G0
J8 Restore H0
J9 ERASE cell (0)

Description Processes

*J10 FIND value of attribute (0) of (1)
J11 Assign (1) as value of attribute (0) of (2)
J12 Add (1) at front of value list of attribute (0) of (2)
J13 Add (1) at end of value list of attribute (0) of (2)
J14 ERASE attribute (0) of (1)
J15 ERASE all attributes of (0)
*J16 FIND attribute of (0) randomly

Generator Housekeeping Processes

J17 Gen set up: context (0), subprocess (1)
*J18 Execute subprocess of Gen
*J19 Gen clean up

Working Storage Processes

J2n MOVE (0)-(n) into WO-Wn
J3n Restore WO-Wn
J4n Preserve WO-Wn
J5n Preserve WO-Wn; MOVE (0)-(n) into WO-Wn

List Processes

*J60 LOCATE next symbol after cell (0)
*J61 LOCATE last symbol on list (0)
*J62 LOCATE (0) on list (1) (1st occurrence)
J63 INSERT (0) before symbol in cell (1)
J64 INSERT (0) after symbol in cell (1)
J65 INSERT (0) at end of list (1)
J66 INSERT (0) at end if not on list (1)
J67 Replace (1) by (0) on list (2) (1st occur.)
*J68 DELETE symbol in cell (0)
*J69 DELETE (0) from list (1) (1st occurrence)
*J70 DELETE last symbol from list (0)
J71 ERASE list (0)
J72 ERASE list structure (0)
J73 COPY list (0)
J74 COPY list structure (0)
J75 Divide list after location (0); name of remainder is output (0)
*J76 INSERT list (0) after (1), locate last symbol
*J77 TEST if (0) is on list (1)
*J78 TEST if list (0) is not empty
*J79 TEST if cell (0) is not empty
*J8n FIND the nth symbol on list (0)
J9n Create list of n symbols, (n-1) to (0)
*J100 Gen symbols on list (1) for (0)
*J101 Gen cells of list structure (1) for (0)
*J102 Gen cells of tree (1) for (0)
*J103 Gen cells of block (1) for (0)
J104

Auxiliary Storage Processes

*J105 MOVE list structure (0) in from auxiliary
J106 File list structure (0) in fast-auxiliary
J107 File list structure (0) in slow-auxiliary
*J108 TEST if list structure (0) is on auxiliary
J109 Compact auxiliary data storage system (0)

Arithmetic Processes

J110 (1) + (2) - (0), leave (0)
J111 (1) - (2) - (0), leave (0)
J112 (1) x (2) - (0), leave (0)
J113 (1) / (2) - (0), leave (0)
*J114 TEST if (0) = (1)
*J115 TEST if (0) > (1)
*J116 TEST if (0) < (1)
*J117 TEST if (0) = 0
*J118 TEST if (0) > 0
*J119 TEST if (0) < 0
J120 COPY (0)
J121 Set (0) identical to (1), leave (0)
J122 Take absolute value of (0), leave (0)
J123 Take negative of (0), leave (0)
J124 Clear (0), leave (0)
J125 Tally 1 in (0), leave (0)
J126 Count list (0)
*J127 TEST if data type (0) = data type (1)
J128 Translate (0) to be data type of (1)
J129 Produce random number between 0 and (0)

Data Prefix Processes

*J130 TEST if (0) is regional symbol
*J131 TEST if (0) names data term
*J132 TEST if (0) is local symbol
*J133 TEST if list (0) has been marked processed
*J134 TEST if (0) is internal symbol
J135
J136 Make (0) local, leave (0)
J137 Mark list (0) processed, leave (0)
J138 Make (0) internal, leave (0)
J139

Read and Write Processes

*J140 Read list structure
*J141 Read symbol from console
J142 Write list structure (0)
J143 Rewind tape (0)
J144 Skip to next tape file
J145 Write end-of-file
J146 Write end-of-set

Monitor System

J147 Mark routine (0) to trace
J148 Mark routine (0) to propagate trace
J149 Mark routine (0) to not trace

Print Processes

J150 Print list structure (0)
J151 Print list (0)
J152 Print symbol (0)
J153 Print data term (0) w/o name or type
J154 Clear print line
J155 Print line
*J156 Enter symbol (0) left-justified
*J157 Enter data term (0) left-justified
*J158 Enter symbol (0) right-justified
*J159 Enter data term (0) right-justified
J160 Tab to column (0)
J161 Increment column by (0)
*J162 Enter (0) according to format W43
J163
J164

In-process Loading

J165 Load routines and data

Save for Restart (\$ 20.0)

*J166 Save on unit (0) for restart
*J167 Skip list structure
J168
J169

Error Trap

J170 Trap on (0)

Block Handling Processes

J171 Return unused regionals to H2
J172 Make block (0) into a list
*J173 Read into block (0)
*J174 Write block (0)
*J175 FIND region control word of regional symbol (0)
J176 Space (0) blocks on unit 1W19
J177
J178
J179

Line Read Processes

*J180 Read line
*J181 Input line symbol
*J182 Input line data term (0)
*J183 Set (0) to next blank
*J184 Set (0) to next non-blank
*J185 Set (1) to next occurrence of character (0)
*J186 Input line character
J187
J188
*J189 Transfer field to line (0)

Partial Word Processes

J190 Input P of cell (0)
J191 Input Q of cell (0)
J192 Input SYMB of cell (0)
J193 Input LINK of cell (0)
J194 Set (1) to be P of cell (0)
J195 Set (1) to be Q of cell (0)
J196 Set (1) to be SYMB of cell (0)
J197 Set (1) to be LINK of cell (0)
J198
J199

Miscellaneous Processes

*J200 LOCATE (0)th symbol on list (1)
J201 ERASE routine (0)
J202 Print post mortem and continue

IPL INSTRUCTION: PQ SYMB LINK

P is operation code
 P = 0 Execute S
 P = 1 Input S (after preserving H0)
 P = 2 Output to S (then restore H0)
 P = 3 Restore (pop up) S
 P = 4 Preserve (push down) S
 P = 5 Replace (0) by S
 P = 6 Copy (0) in S
 P = 7 Branch to S if H5-
 Q is designation code
 Q = 0 S = SYMB
 Q = 1 S = symbol in cell named SYMB
 Q = 2 S = symbol in cell named in cell
 named SYMB
 Q = 3 S = SYMB; start selective trace
 Q = 4 S = SYMB; continue selective
 trace
 Q = 5 Machine language routine
 Q = 6 Routine in fast-aux. storage
 Q = 7 Routine in slow-aux. storage
 SYMB is symbol operated on by Q
 LINK is address of next instruction
 (0 for end of routine)

SYSTEM STORAGE CELLS

H0 Communication cell
 H1 Current instruction address cell
 H2 Available space list
 H3 Tally of interpretation cycles
 H4 Current auxiliary routine cell
 H5 Test cell
 W0-W9 Common working storage
 W10 Random number control cell
 W11 Integer division remainder
 W12 Monitor start cell (Q = 3)
 W13 Monitor end cell (Q = 3)
 W14 External interrupt cell
 W15 Post mortem routine cell
 W16 Input mode cell
 W17 Output mode cell
 W18 Read unit cell
 W19 Write unit cell
 W20 Print unit cell
 W21 Print column cell
 W22 Print spacing cell
 W23 Post mortem list cell
 W24 Print line cell
 W25 Print entry column cell
 W26 Error trap cell
 W27 Trap address cell
 W28 Trap symbol cell
 W29 Monitor point address cell
 W30 Field length cell
 W31 Trace mode cell
 W32 Reserved available space cell
 W33 Cycle count for trap cell
 W34 Current available space cell
 W35 Slow-aux. obsolete structure cell
 W36 Used slow-auxiliary space cell
 W37 Slow-auxiliary storage density cell
 W38 Slow-auxiliary storage compacting
 routine cell
 W39 Fast-aux. obsolete structure cell
 W40 Used fast-auxiliary space cell
 W41 Fast-auxiliary storage density cell
 W42 Fast-auxiliary storage compacting
 routine cell
 W43 Format cell

IPL DATA: PQ SYMB LINK

Q = 0 Standard list cell:
 P is irrelevant
 SYMB is symbol
 LINK is address of next list cell
 (0 for end of list)
 Q = 1 Data term: ±PQ SYMB LINK
 Decimal integer 1 dddd dddd
 Floating point 11 dddd d ±ee
 Alphanumeric 21 aaaaa
 Octal 31 dddd dddd

TYPE CARDS

0 (blank) Routines and data
 1 Comments
 2 Region definition
 NAME = Regional symbol
 SYMB = Origin (if given)
 LINK = Size
 3 Block reservation
 NAME = Block control word (if given)
 SYMB = Origin (if given)
 LINK = Size
 Q = 0 Reserve regional symbols
 Q = 1 Reserve print line
 Q = 2 Reserve block
 Q = 3 Reserve auxiliary buffer
 Q = 4 Specify available space
 4 Listing cards
 5 Main storage header
 6 Fast-auxiliary storage header
 7 Slow-auxiliary storage header
 8 Editing header; inhibits loading
 NAME = Name of storage block
 P = Input mode
 P = 0 IPL standard
 P = 1 IPL compressed
 P = 2 IPL binary
 P = 3 Machine code
 P = 4 Restart mode
 Q = Type of input
 Q = 0 Routines; internals
 symbolic
 Q = 1 Data; internals
 symbolic
 Q = 2 Routines; internals
 symbolic; reset inter-
 nal symbol table
 Q = 3 Data; internals sym-
 bolic; reset internal
 symbol table
 Q = 4 Routines; internals
 absolute
 Q = 5 Data; internals
 absolute
 SYMB = Alternate input unit
 0 (blank) = controlling unit
 1-10 = Internal tapes
 Regional SYMB names first
 routine (terminate loading)
 LINK = Output mode: of form bbbcd
 b = Output unit: blank = unit
 1W19; 1-10 = unit 1-10
 c = 0 (blank) if assembly
 listing
 = 1 or any character if no
 assembly listing
 d = 0 (blank) if no output
 = 1 IPL compressed output
 = 2 IPL binary output
 = 3 Machine code output
 = 9 IPL standard output
 9 First card
 SYMB = Controlling unit (0 or blank
 = normal input unit)

XIV. COMPLETE VOCABULARY LISTING

A complete vocabulary listing, as shown in this section, was kept up to date during conversion of LT into its present form.

The vocabulary is intended to serve as an extension of the List of Basic Processes in the IPL-V system.

A0	FREE VARIABLE -A-.	A000V000
B0	FREE VARIABLE -B-.	B000V000
C0	FREE VARIABLE -C-.	C000V000
D0	FREE VARIABLE -D-.	D000V000
E0	FREE VARIABLE -E-.	E000V000
F0	FREE VARIABLE -F-.	F000V000
G0	FREE VARIABLE -G-.	G000V000
I0	LOGICAL CONNECTIVE -IMPLIES-	I000V000
K0	SYMBOL FOR CHARACTER K.	K000V000
K1	HOLDS -OR-.	K001V000
K2	HOLDS -NOT-.	K002V000
K3	HOLDS -AND-	K003V000
K4	HOLDS -PROVEN EQUIVALENCE-	K004V000
K5	HOLDS -DEFINITIONAL EQUIVALENCE.	K005V000
K6	HOLDS -IMPLIES-	K006V000
K7	HOLDS DELIMIER SYMBOL FOR DELIMITABLE CHARACTERS.	K007V000
K10	PREVIOUS SUBPROBLEM NUMBER. (DATA TERM)	K010V000
K11	SUBSTITUTION COUNT.	K011V000
K12	EFFORT BASE (AND TOTAL).	K012V000
K20	LIMIT ON NO. OF SUBPROBLEMS.	K020V000
K21	LIMIT ON NO. OF SUBSTITUTIONS.	K021V000
K22	LIMIT ON EFFORT.	K022V000
K30	HOLDS R0 IF PROVED THEOREMS ARE TO BE REMEMBERED.	K030V000
K31	HOLDS Y0 IF REJECTED SUBPROBLEMS ARE TO BE PRINTED.	K031V000
K41	D.T. COLUMN FOR PRINTING METHODS IN PROOF SEQUENCES.	K041V000
K42	D.T. COLUMN FOR PRINTING NAMES IN PROOF SEQUENCES.	K042V000
K43	D.T. COLUMN FOR PRINTING EXPRESSIONS IN PROOF SEQUENCES.	K043V000
K44	D.T. COLUMN FOR PRINTING 'LIMIT'	K044V000
K45	D.T. COLUMN FOR PRINTING 'ACTUAL'	K045V000
K46	D.T. COLUMN FOR PRINTING 'REJECTED SUBPROBLEM'.	K046V000
K47	D.T. COLUMN FOR PRINTING NAME OF NEW SUBPROBLEM.	K047V000
K48	D.T. COLUMN FOR PRINTING 'THEOREM, METHOD' OF NEW PROBLEMS.	K048V000
K51	DATA TERM ' ('	K051V000
K52	DATA TERM ')'	K052V000
K53	DATA TERM '.'	K053V000
K54	DATA TERM ','	K054V000
L0	SYMBOL FOR CHARACTER L.	L000V000
L1	LIST OF TRUE EXPRESSIONS. (AXIOMS, DEFINITIONS, THEOREMS)	L001V000
L2	LIST OF FREE VARIABLES.	L002V000
L3	LIST OF UNPROVED EXPRESSIONS. (PROBLEMS)	L003V000
L4	LIST STRUCTURE MAP OF ALL TRUE EXPRESSIONS.	L004V000
L5	HOLDS LIST STRUCTURE MAP OF TRUE EXPRESSIONS.	L005V000
L6	LIST OF SPECIAL METHODS FOR ORIGINAL PROBLEMS.	L006V000
L7	LIST OF REGULAR METHODS FOR ALL PROBLEMS.	L007V000
L8	DESCRIPTION LIST TABLE OF DELIMITABLE CHARACTERS.	L008V000
L9	DESCRIPTION LIST TABLE OF CHARACTER SYMBOLS FOR READING TEXT.	L009V000
L10	LIST OF UNTRIED PROBLEMS.	L010V000
L11	LIST OF FOUND PROBLEMS.	L011V000
M0	SYMBOL FOR CHARACTER M.	M000V000
M1	SINGLE PROBLEM EXECUTIVE FOR PROBLEM (0).	M001V000
M2	MULTIPLE PROBLEM EXECUTIVE FOR LIST L3.	M002V000

M3	SET-UP FOR NEW PROBLEM.	M003V000
M7	APPLY METHODS (1) TO PROBLEM (0), ERASE (1) WHEN THRU.	M007V000
M8	CREATE A LIST OF METHODS FOR PROBLEM (0).	M008V000
M11	DETACHMENT METHOD FOR PROBLEM (0). SETS H5+ AND OUTPUTS SUCCESSFUL SUBPROBLEM IF SOLUTION IS FOUND. SETS H5- IF NO SOLUTION. ADDS GOOD NEW SUBPROBLEMS TO UNTRIED LIST.	M011V000 M011V010 M011V020
M12	SUBSTITUTION METHOD FOR PROBLEM (0). SETS H5+ AND OUTPUTS SUCCESSFUL SUBPROBLEM IF SOLUTION IS FOUND. SETS H5- IF NO OUTPUT.	M012V000 M012V010 M012V020
M13	REPLACEMENT METHOD FOR SUBPROBLEM (0). SETS H5+ AND OUTPUTS SUCCESSFUL SUBPROBLEM IF SOLUTION IS FOUND. SETS H5- IF NO SOLUTION. ADDS GOOD NEW SUBPROBLEMS TO UNTRIED LIST.	M013V000 M013V010 M013V020
M14	FORWARD CHAINING METHOD FOR PROBLEM (0). SETS H5+ AND OUTPUTS SUCCESSFUL SUBPROBLEM IF SOLUTION IS FOUND. SETS H5- IF NO SOLUTION. ADDS GOOD NEW SUBPROBLEMS TO UNTRIED LIST.	M014V000 M014V010 M014V020
M15	BACKWARD CHAINING METHOD FOR PROBLEM (0). SETS H5+ AND OUTPUTS SUCCESSFUL SUBPROBLEM IF SOLUTION IS FOUND. SETS H5- IF NO SOLUTION. ADDS GOOD NEW SUBPROBLEMS TO UNTRIED LIST.	M015V000 M015V010 M015V020
M16	SUBLEVEL REPLACEMENT METHOD FOR PROBLEM (0). SETS H5+ AND OUTPUTS SUCCESSFUL SUBPROBLEM IF SOLUTION IS FOUND. SETS H5- IF NO SOLUTION. ADDS GOOD NEW SUBPROBLEMS TO UNTRIED LIST. M16 TRIES FOR A SUBPROBLEM AT EACH SUBLEVEL OF PROBLEM (0).	M016V000 M016V010 M016V020 M016V030 M016V040
M17	SUBLEVEL REPLACEMENT METHOD FOR PROBLEM (0). SETS H5+ AND OUTPUTS SUCCESSFUL SUBPROBLEM IF SOLUTION IS FOUND. SETS H5- IF NO SOLUTION. ADDS GOOD NEW SUBPROBLEMS TO UNTRIED LIST. M17 TRIES FOR ONLY ONE SUBPROBLEM BY TRYING REPLACEMENT ON ALL SUBLEVELS.	M017V000 M017V010 M017V020 M017V030 M017V040
M19	FINISH BUILDING NEW SUBPROBLEM TEX (3) FROM (2) VIA THEOREM (1) BY METHOD(0). H5- MEANS NEW TEX WAS ERASED DUE TO LOW UTILITY. H5+ MEANS OUTPUT TEX (0) HAS SATISFACTORY UTILITY.	M019V000 M019V010 M019V020 M019V030
M40	TEST MATCH OF TOTAL EXPRESSIONS (0),(1) WITHOUT SUBSTITUTION.	M040V000
M41	TEST MATCH OF SEGMENTS (0),(1) WITHOUT SUBSTITUTION.	M041V000
M42	ADD PROBLEM(0) TO FOUND PROBLEMS LIST IF CAN. H5- MEANS THIS PROBLEM WAS PREVIOUSLY FOUND.	M042V000 M042V010
M43	MEASURE UTILITY OF (0). H5+ IF GOOD, H5- IF NO GOOD.	M043V000
M50	ADD TRUE EXPRESSION (0) TO TRUE EXPRESSIONS LIST AND MAP.	M050V000
M51	ADD PROBLEM (0) TO UNTRIED PROBLEMS LIST.	M051V000
M54	ADD EXPRESSION (0) TO MAP OF TRUE EXPRESSIONS.	M054V000
M60	FIND AND REMOVE NEXT UNTRIED SUBPROBLEM ON UNTRIED LIST. H5- MEANS NO SUBPROBLEM ON LIST AND NO OUTPUT.	M060V000 M060V010
M62	CREATE A LIST OF FEASIBLE MATCHES FOR SEGMENT (0) FROM MAP (1).	M062V000
M63	CREATE A LIST OF FEASIBLE MATCHES FOR TOTAL EXPRESSION (1) FROM MAP (0).	M063V000 M063V010
M70	PRINT EXPRESSION (0).	M070V000
M71	PRINT PROOF SEQUENCE WITH SUCCESSFUL SUBPROBLEM (0).	M071V000
M72	PRINT FAILURE TO FIND PROOF.	M072V000
M73	ENTER SEGMENT (0).	M073V000
M74	ENTER TOTAL EXPRESSION (0).	M074V000
M75	PRINT NEW SUBPROBLEM (0).	M075V000

M76	ENTER LIST OF DATA TERMS (0).	M076V000
M77	PRINT CURRENT STATUS OF LIMITS.	M077V000
M78	PRINT TO PROVE PROBLEM (0).	M078V000
M79	ENTER NAME OF (0). USE EXTERNAL NAME IF CAN.	M079V000
M80	PRINT PROOF LINE FOR METHOD (0) AND TEX (1).	M080V000
M81	PRINT REJECTED PROBLEM.	M081V000
M82	PRINT 'REMEMBER PROVED THEOREM'	M082V000
M88	PRINT LIST FORM EXPRESSION (0).	M088V000
M89	READ NEXT LOGIC EXPRESSION FROM UNIT 1W18.	M089V000
	H5-MEANS NONE THERE.	M089V010
M90	TEST IF A LIMIT HAS BEEN REACHED.	M090V000
M110	MAKE FREE VARIABLES OF TOTAL EXPRESSIONS (0),(1) DISJOINT.	M110V000
	SUBSTITUTES NEW VARIABLES IN (1) AS REQUIRED.	M110V010
M111	MATCH SEGMENTS (0),(1) WITH SUBSTITUTION AS REQUIRED.	M111V000
	H5+ MEANS OUTPUT (0) IS A SUBSTITUTION LIST OF PAIRS,	M111V010
	1ST IS A FREE VARIABLE, 2ND IS ITS SUBSTITUTOR.	M111V020
M112	EXPAND SUBSTITUTION LIST (0).	M112V000
	REPLACE EXPRESSIONS WITH COMPLETELY SUBSTITUTED LOCAL COPIES.	M112V010
M113	MATCH SEGMENTS (0),(1) WITH SUBSTITUTION AS REQUIRED.	M113V000
	H5+ MEANS OUTPUT (0) IS EXPANDED SUBSTITUTION LIST.	M113V010
M114	MATCH SEGMENTS (0),(1) WITH SUBSTITUTION AS REQUIRED.	M114V000
	NO OUTPUT, H5- MEANS THE MATCH FAILED.	M114V010
M115	SUBSTITUTE IN SEGMENT (0) FROM SUBSTITUTION LIST (1).	M115V000
M116	CREATE LIST OF FREE VARIABLES IN TOTAL EXPRESSION (0).	M116V000
	H5- MEANS NO OUTPUT.	M116V010
M117	CREATE LIST OF BOUND VARIABLES IN TOTAL EXPRESSION (0).	M117V000
	H5- MEANS NO OUT PUT.	M117V010
N0	SYMBOL FOR CHARACTER N.	N000V000
N1	INTEGER DATA TERM = 1	N001V000
N2	INTEGER DATA TERM = 2	N002V000
N3	INTEGER DATA TERM = 3	N003V000
N4	INTEGER DATA TERM = 4	N004V000
N5	INTEGER DATA TERM = 5	N005V000
N6	INTEGER DATA TERM = 6	N006V000
N7	INTEGER DATA TERM = 7	N007V000
N8	INTEGER DATA TERM = 8	N008V000
N9	INTEGER DATA TERM = 9	N009V000
N10	INTEGER DATA TERM = 0	N010V000
O0	SYMBOL FOR CHARACTER O.	O000V000
P0	VARIABLE TERM -P-	P000V000
P2	TEST IF (0) IS A BOUND VARIABLE.	P002V000
P3	CLEAR DESCRIPTIONS OF EXPRESSION (0).	P003V000
P4	GO THRU NOTS OF SEGMENT (0). LEAVE FIRST UNNOTTED SEGMENT.	P004V000
	H5- MEANS NO OUTPUT DUE TO FAULTY EXPRESSION.	P004V010
P5	TEST IF MAIN CONNECTIVE OF EXPRESSION (0) IS -IMPLIES--.	P005V000
P6	TEST IF CONNECTIVE (0) IS NON-UNARY.	P006V000
P7	TEST IF (0) IS A CONNECTIVE.	P007V000
P8	TEST IF (0) IS VARIABLE TERM.	P008V000
P9	TEST IF (0) IS FREE VARIABLE.	P009V000
P12	FIND MAIN EXPRESSION OF TOTAL EXPRESSION (0).	P012V000
P13	FIND LEFT SEGMENT OF TOTAL EXPRESSION (0).	P013V000

P14	FIND RIGHT SEGMENT OF TOTAL EXPRESSION (0).	P014V000
P15	TEST IF TOTAL EXPRESSION (0) IS IN TREE FORM.	P015V000
P16	FIND MAIN CONNECTIVE OF TOTAL EXPRESSION (0).	P016V000
P17	COPY SEGMENT (0). IF (0) IS A VARIABLE, OUTPUT THE INPUT.	P017V000
P18	TEST IF (0) IS A CHARACTER SYMBOL.	P018V000
P19	GET APPROPRIATE CHARACTER SYMBOL FOR (0).	P019V000
P20	MAKE LEFT SUBEXPRESSION OF EXPRESSION (0) INTO EXPRESSION, H5- MEANS NO OUTPUT DUE TO FAULTY EXPRESSION.	P020V000 P020V010
P21	MAKE RIGHT SUBEXPRESSION OF EXPRESSION (0) INTO EXPRESSION, H5- MEANS NO OUTPUT DUE TO FAULTY EXPRESSION.	P021V000 P021V010
P22	CREATE NEW PROBLEM WITH SEGMENT (0) AS LEFT SUBEXPRESSION, SEGMENT (1) AS RIGHT SUBEXPRESSION, IMPLIES AS CONNECTIVE.	P022V000 P022V010
P23	ERASE MADE EXPRESSION (0).	P023V000
P24	MAKE SEGMENT (0) INTO A NEW TOTAL EXPRESSION.	P024V000
P25	COPY TEX (0) FOR SUBSTITUTION.	P025V000
P26	GENERATE LOCATIONS OF NON-VARIABLE SEGMENTS FROM TOTAL EXPRESSION (1) AT THE LEVEL OF DATA TERM(2) FOR PROCESS (0).	P026V000 P026V010 P026V020
P27	REPLACE BOUND VARIABLES BY FREE VARIABLES IN TEX (0).	P027V000
P28	GENERATE LOCATIONS OF FREE VARIABLES IN SEGMENT (1) FOR (0).	P028V000
P29	GENERATE LOCATIONS OF BOUND VARIABLES IN SEGMENT (1) FOR (0).	P029V000
P30	CREATE LIST OF FREE VARIABLES IN EXPRESSION (0).	P030V000
P31	CREATE LIST OF BOUND VARIABLES IN EXPRESSION (0).	P031V000
P50	CONVERT LOGIC EXPRESSION (0) TO INTERNAL TREE FORM IF IN EXTERNAL FORM. NO OUTPUT. H5- MEANS NO CONVERSION. ENTIRE EXPRESSION MUST BE PARENTHEZIZED.	P050V000 P050V010 P050V020
P51	REPLACE ALL DELIMITED SYMBOLS IN EXPRESSION (0) IF (0) IS IN LIST FORM.	P051V000 P051V010
P52	CREATE A TREE FORM MAIN SEGMENT FROM LIST FORM EXPRESSION (0). H5- MEANS NO OUTPUT DUE TO A FAULTY INPUT EXPRESSION.	P052V000 P052V010
P55	LOCATE LIST FOLLOWING DATA TERM (0) ON LIST (1). H5+ MEANS OUTPUT (0) IS CELL HOLDING SUBLIST. H5- MEANS OUTPUT (0) IS CELL AFTER WHICH TO INSERT.	P055V000 P055V010 P055V020
Q0	VARIABLE TERM -Q-	Q000V000
Q1	FIND CONNECTIVE OF SEGMENT (0).	Q001V000
Q2	FIND NO. OF LEVELS OF EXPRESSION (0).	Q002V000
Q3	FIND NO. OF DISTINCT VARIABLES OF EXPRESSION (0).	Q003V000
Q4	FIND NO. OF VARIABLE PLACES OF EXPRESSION (0).	Q004V000
Q5	ATTRIBUTE--VARIABLE TERM.	Q005V000
Q6	ATTRIBUTE--FREE VARIABLE.	Q006V000
Q7	ATTRIBUTE--EXTERNAL NAME.	Q007V000
Q8	FIND PROBLEM NO. OF EXPRESSION (0).	Q008V000
Q9	ATTRIBUTE--BOUND VARIABLE.	Q009V000
Q10	FIND PROBLEM EXPRESSION (0) DERIVED FROM.	Q010V000
Q11	FIND METHOD OF DERIVATION FOR EXPRESSION (0).	Q011V000
Q12	FIND THEOREM USED TO DERIVE PROBLEM (0).	Q012V000
Q13	FIND PROVING THEOREM FOR PROBLEM (0).	Q013V000
Q14	FIND TYPE OF CONNECTIVE (0).	Q014V000
Q15	ATTRIBUTE--INTERNAL FORM.	Q015V000
Q16	FIND EXTERNAL NAME OF (0) IN TABLE T10.	Q016V000
Q17	FIND CURRENT LEVEL OF SUBSEGMENT REPLACEMENT OF PROBLEM (0).	Q017V000

Q18	FIND SUFFIX OF EXPRESSION (0).	Q018V000
Q19	FIND APPROPRIATE CHARACTER SYMBOL FOR (0).	Q019V000
R0	VARIABLE TERM -R-	R000V000
S0	VARIABLE TERM -S-	S000V000
T0	VARIABLE TERM -T-	T000V000
T1	TEXT LIST 'GIVEN'	T001V000
T2	TEXT LIST 'PROOF FOUND'	T002V000
T3	TEXT LIST 'SUBSTITUTION'	T003V000
T4	TEXT LIST 'Q.E.D.'	T004V000
T5	TEXT LIST OF 5 BLANK CHARACTERS.	T005V000
T6	TEXT LIST 'NO PROOF FOUND'	T006V000
T7	TEXT LIST 'EFFORT'	T007V000
T8	TEXT LIST 'SUBPROBLEMS'	T008V000
T9	TEXT LIST 'SUBSTITUTIONS'	T009V000
T10	DESCRIPTION LIST TABLE OF NAMES.	T010V000
T12	TEXT LIST 'DETACHMENT'	T012V000
T13	TEXT LIST 'REPLACEMENT'	T013V000
T14	TEXT LIST 'FORWARD CHAINING'	T014V000
T15	TEXT LIST 'BACKWARD CHAINING'	T015V000
T16	TEXT LIST 'SUBLEVEL REPLACEMENT'	T016V000
T19	TEXT LIST 'REJECTED PROBLEM'	T019V000
T20	TEXT LIST 'ACTUAL'	T020V000
T21	TEXT LIST 'LIMIT'	T021V000
T22	TEXT LIST 'TO PROVE'	T022V000
T23	TEXT LIST 'REMEMBER PROVED THEOREM'	T023V000
T24	TEXT LIST 'BAD EXPRESSION'	T024V000
U0	SYMBOL FOR CHARACTER U.	U000V000
V0	LOGICAL CONNECTIVE -OR-	V000V000
X0	SYMBOL FOR CHARACTER X.	X000V000
X1	RUN EXECUTIVE	X001V000
X9	SAVE AND CONTINUE EXECUTIVE.	X009V000
X21	LIST OF ROUTINES TO BE MARKED TO TRACE.	X021V000
X22	LIST OF ROUTINES TO BE MARKED TO PROPAGATE TRACE.	X022V000
X23	DESCRIPTION LIST OF TRAP ACTIONS. (ATTRIBUTE/VALUE PAIRS)	X023V000
Y0	SYMBOL FOR CHARACTER Y.	Y000V000
Z0	SYMBOL FOR CHARACTER Z.	Z000V000
=0	LOGICAL CONNECTIVE -PROVEN EQUIVALENCE-	=000V000
=1	LOGICAL CONNECTIVE -DEFINITIONAL EQUIVALENCE-	=001V000
-0	LOGICAL CONNECTIVE -NOT-	-000V000
*0	LOGICAL CONNECTIVE -AND-	*000V000
'0	SYMBOL FOR QUOTE MARK.	'000V000
.0	SYMBOL FOR PERIOD.	.000V000
)0	SYMBOL FOR RIGHT PAREN.	)000V000
/0	SYMBOL FOR SLASH.	/000V000
/1	SYMBOL FOR DIGIT 1.	/001V000
/2	SYMBOL FOR DIGIT 2.	/002V000
/3	SYMBOL FOR DIGIT 3.	/003V000
/4	SYMBOL FOR DIGIT 4.	/004V000
/5	SYMBOL FOR DIGIT 5.	/005V000
/6	SYMBOL FOR DIGIT 6.	/006V000
/7	SYMBOL FOR DIGIT 7.	/007V000

/8	SYMBOL FOR DIGIT 8.	/008V000
/9	SYMBOL FOR DIGIT 9.	/009V000
/10	SYMBOL FOR DIGIT 0.	/010V000
/11	SYMBOL FOR CHARACTER H.	/011V000
/12	SYMBOL FOR CHARACTER J.	/012V000
/13	SYMBOL FOR CHARACTER W.	/013V000
/14	DUMMY CHARACTER SYMBOL WITH EXTERNAL NAME '/UGH/'	/014V000
/16	DUMMY EXPRESION TO SUPPLY TEXT 'THE DEFINITIONS'	/016V000
,0	SYMBOL FOR COMMA.	,000V000
(0	SYMBOL FOR LEFT PAREN.	(000V000
+0	SYMBOL FOR PLUS SIGN.	+000V000
\$0	SYMBOL FOR DOLLAR SIGN.	\$000V000

XV. COMPLETE PROGRAM LISTING

The program listing has been carefully and extensively documented in the comment fields to enable students to work directly from it.

JOB	8168,LTNEW1,EAS826,5MIN,0,099,C	STEFFERUD	
ASSIGN	A6=SYSAR2		
ASSIGN	B6=SYSAR3		
IPL			
LOGIC THEORIST (IPL-V)	9		-
	2 A	10	0000 000
	2 B	10	0000 010
	2 C	10	0000 020
	2 D	10	0000 030
	2 E	10	0000 040
	2 F	10	0000 050
	2 G	10	0000 060
	2 I	10	0000 070
	2 K	100	0000 080
	2 L	50	0000 090
	2 M	200	0000 100
	2 N	50	0000 110
	2 O	10	0000 120
	2 P	100	0000 130
	2 Q	50	0000 140
	2 R	10	0000 150
	2 S	10	0000 160
	2 T	40	0000 170
	2 U	10	0000 180
	2 V	10	0000 190
	2 X	50	0000 200
	2 Y	10	0000 210
	2 Z	50	0000 220
	2 -	10	0000 230
	2 *	600	0000 240
	2 =	10	0000 260
	2 ,	10	0000 270
	2 /	20	0000 280
	2 +	10	0000 290
	2 .	10	0000 300
	2 (	10	0000 310
	2)	10	0000 320
	2 '	10	0000 330

PROGRAM HEADER	5			R -
M1 SINGLE PROBLEM EXECUTIVE FOR	M1	M3		M001R000
PROBLEM (0). H5 + IF SUCCEEDS.		40W0		M001R010
		60W0	1W0=PROB	M001R020
TEST UTILITY.		M43		M001R030
IF NO GOOD, QUIT.		709-4		M001R040
		11W0		M001R050
PRINT 'TO PROVE' PROBLEM 1W0.		M78		M001R060
		11W0		M001R070
TRY SUBSTITUTION.		M12		M001R080
IF WORKED, PRINT PROOF.		70	9-2	M001R090
	9-1	11W0		M001R100
CREATE LIST OF METHODS FOR PROB.		M8		M001R110
		11W0		M001R120
APPLY METHODS.		M7		M001R130
IF PROOF FOUND, PRINT IT.		70	9-2	M001R140
TEST IF ANY LIMITS EXCEEDED.		M90		M001R150
IF YES, QUIT.		70	9-3	M001R160
FIND NEXT SUBPROBLEM.		M60		M001R170
IF NONE, QUIT.		709-3		M001R180
IF ONE,		60W0		M001R190
PRINT SUBPROBLEM, TRY METHODS.		M70	9-1	M001R200
PRINT PROOF FROM (0).	9-2	M71		M001R210
AND QUIT +.		30W0	J4	M001R220
PRINT FAILURE, QUIT -.	9-3	M72	9-5	M001R230
	9-4	11W0		M001R240
PRINT REJECTED PROBLEM		M81		M001R250
AND QUIT -.	9-5	30W0	J3	M001R260
	1			R
M2 MULTIPLE PROBLEM EXECUTIVE.	M2	10L3		M002R000
		109-100		M002R010
GENERATE PROBLEMS FOR PROOF.		J100	0	M002R020
	9-100	J50		M002R030
		11W0		M002R040
CONVERT TO INTERNAL (TREE) FORM.		P50		M002R050
IF FAILED, TAKE ERROR ACTION.		709-103		M002R055
		11K30		M002R060
		10R		M002R070
TEST IF REMEMBERING.		J2		M002R080
INPUT PROBLEM FOR M1.		11W0		M002R090
		709-101		M002R100
TRY FOR PROOF TO BE REMEMBERED,		M1		M002R110
H5- MEANS NO PROOF FOUND.		709-102		M002R120
		11W0		M002R130
PRINT 'REMEMBER PROVED THEOREM'		M82		M002R135
ADD TO TRUE EXPRESSIONS LIST.		M50	9-102	M002R140
TRY FOR PROOF TO BE FORGOTTEN.	9-101	M1		M002R150
CLEAN UP W0, H5+ FOR GEN.	9-102	30W0	J4	M002R160
	9-103	11W0		M002R170
		M88	9-102	M002R180

M3 SET UP FOR NEW PROBLEM.	1			R
CLEAR UNTRIED PROBLEMS LIST.	M3	10L10		M003R000
		J75		M003R010
		J72		M003R020
		10L11		M003R030
		J75		M003R040
CLEAR FOUND PROBLEMS LIST.		J72		M003R050
		10K10		M003R060
CLEAR PREVIOUS PROBLEM NUMBER.		J124		M003R070
		50K11		M003R080
		J124		M003R090
CLEAR SUBSTITUTIONS COUNT.		50H3		M003R100
		10K12		M003R110
SET EFFORT BASE.		J121	J8	M003R120
	1			R
M7 APPLY METHODS (1) TO PROBLEM	M7	J50		M007R000
(0), ERASE LIST (1) WHEN THRU.		40H0		M007R010
H5+ MEANS OUTPUT (0) IS SOL'N.		109-100		M007R020
H5- MEANS NO OUTPUT, NO SOL'N.		J100		M007R030
		J5		M007R035
		30W0		M007R040
IF NO PROOF, ERASE METHOD LIST.		70J71		M007R045
ELSE SAVE PROOF AND ERASE LIST.		J6	J71	M007R050
9-100 SUBPROCESS, APPLY METHOD (0)	9-100	11W0		M007R060
TO PROBLEM 1W0.		J6		M007R070
REVERSE H5 FOR GENERATOR.		J1	J5	M007R080
	1			R
M8 CREATE METHOD LIST FOR (0).	M8	10L7		M008R000
COPY REGULAR LIST.		J73		M008R010
		J50		M008R020
		J130		M008R030
TEST IF ORIGINAL PROBLEM.		709-1		M008R040
IF NOT, NO SPECIAL METHODS.		11W0		M008R050
IF YES,		10L6		M008R070
		J73		M008R080
COPY SPECIAL LIST,		J76		M008R090
INSERT AFTER HEAD OF REGULAR LIST.		51W0	J30	M008R100
SET UP OUTPUT AND QUIT.	9-1	11W0	J30	M008R110
	1			R
M11 DETACHMENT METHOD FOR PROB (0).	M11	J45		M011R000
ADD NEW SUBPROBLEMS TO UNTRIED		60W1	1W1=PROB	M011R010
LIST IF CAN.		J81		M011R020
H5+ MEANS OUTPUT (0) IS A		70J35		M011R030
SOLUTION.		60W0	1W0=PRBMEX	M011R040
H5- MEANS NO SOLUTION, NO OUTPT		50L5	1L5=L4	M011R050
		11K6	1K6=IMPLY	M011R060
		J10		M011R070
FIND IMPLIES MAPS.		70J35		M011R080
		J82		M011R090
FIND MAP OF RIGHT SIDES OF MAIN		70J35		M011R100
CONNECTIVE IMPLIES.		11W1		M011R110

GET LIST OF FEASIBLE THEOREMS.	J6		M011R120
SAVE LIST FOR ERASURE.	M63		M011R130
	60W5		M011R140
	109-100		M011R150
GENERATE FEASIBLE THEOREMS.	J100		M011R160
	11W5	1W5=THMLSTM	M011R170
CLEAN UP AND	J71		M011R180
REVERSE H5 AFTER GENERATOR.	J5	J35	M011R190
9-100 SUBPROCESS,	9-100 60W2	1W2=THM	M011R200
TRY PROOF WITH THEOREM (0).	11W1		M011R210
	11W2		M011R220
MAKE FREE VARIABLES DISJOINT	M110		M011R230
FIND RIGHT SUBSEGMENT OF THM	P14		M011R240
	70J4		M011R250
	11W0		M011R260
MATCH, MAKE THM RIGHT LIKE PROB.	M113	(1)=THMRT	M011R270
	70J4		M011R280
	60W3	1W3=SUBSTLM	M011R290
	11W2		M011R300
FIND THM LEFT SUBSEGMENT.	P13		M011R310
	709-101	(0)=THMLFTM	M011R320
	P17		M011R340
	P24		M011R345
MAKE NEW TEX FROM COPY OF THMLFT.	60W4	1W4=NEWTEXM	M011R350
	M115		M011R360
SUBSTITUTE INTO NEWTEX FROM (1)	9-101		M011R370
ERASE SUBSTL.	11W4	NEWPROBLEM	M011R390
ASSIGN DERIVATION	11W1	PROBLEM	M011R400
	11W2	THEOREM	M011R410
	10M11		M011R420
COMPLETE NEW SUBPROBLEM	M19		M011R430
DESCRIPTION AND MEASURE UTILITY.	70J4		M011R440
H5- MEANS NEW PROBLEM WAS ERASED.	M12	J5	M011R460
TRY SUBSTITUTION, OUTPUT SOL'N.	9-101 11W3		M011R470
	J72	J4	M011R480
			R
M12 SUBSTITUTION METHOD FOR	1		M012R000
PROBLEM (0). H5+ MEANS	M12		M012R010
OUTPUT (0) IS A SOLUTION	60W1	1W1=PROB	M012R020
H5- MEANS NO OUTPUT.	J81		M012R030
	70J33		M012R040
	60W0	1W0=PR5MEXM	M012R050
	50K11		M012R060
TALLY SUBSTITUTION COUNTER.	J125		M012R070
	51W1		M012R080
	10L4		M012R090
GET A LIST OF FEASIBLE THEOREMS.	M63		M012R100
SAVE THE LIST FOR ERASURE.	60W3	1W3=THMLSTM	M012R110
	109-100		M012R120
GENERATE THEOREMS.	J100		M012R130
	11W3		M012R140
ERASE LIST OF THEOREMS,	J71		

CLEAN-UP AND REVERSE H5.
9-100 SUBPROCESS, TRY PROBLEM MEX
1W0 WITH THEOREM (0).

MAKE FREE VARIABLES DISJOINT.
FIND MAIN EXPRESSION OF THEOREM.

TEST FOR MATCH.
IF H5-, SET + FOR GENERATOR,

OTHERWISE ASSIGN PROOF
AND OUTPUT SUCCESSFUL PROBLEM 1W1
AS SOLUTION. QUIT WITH H5- FOR GEN.

M13 REPLACEMENT METHOD FOR PROB (0)
ADD NEW SUBPROBLEMS TO UNTRIED
LIST IF CAN.
H5+ MEANS OUTPUT (0) IS A
SOLUTION.
H5- MEANS NO SOL'N, NO OUTPUT.

FIND DEFINITIONAL EQUIVALENCE MAPS.

FIND MAP OF LEFT SIDES OF DEFEQ.

GET LIST OF FEASIBLE DEFINITIONS.
SAVE LIST FOR ERASURE.
TRY LEFT SIDES.
GENERATE FEASIBLE DEFINITIONS.

ERASE FEASIBLES LIST.
REVERSE H5 FROM GENERATOR.
QUIT IF PROOF IN H0, OR TRY RIGHTS.

FIND MAP OF RIGHT SIDES OF DEFEQ.

GET LIST OF FEASIBLE DEFINITIONS.
SAVE LIST.
TRY RIGHT SIDES.
GENERATE FEASIBLE DEFINITIONS.

ERASE FEASIBLES LIST.
REVERSE H5, CLEAN UP AND QUIT.
TRY LEFT SIDE OF (0) W/1W0.

	J33	J5		M012R150
9-100	60W2		1W2=THM	M012R160
	11W1			M012R170
	11W2			M012R180
	M110			M012R190
	J81			M012R200
	70J4			M012R210
	11W0			M012R220
	M114			M012R230
	70J4			M012R240
	11W1			M012R250
	11W2			M012R260
	10Q13			M012R270
	J11			M012R280
	11W1	J3		M012R290
			R	
	J46			M013R000
	60W1		1W1=PROB	M013R010
	J81			M013R020
	70J36			M013R030
	60W0		1W0=PRBMEX	M013R040
	50L5		1L5=L4	M013R050
	11K5		1K5=DEFEQ	M013R060
	J10			M013R070
	70J36			M013R080
	60W5		1W5=DEFMPS	M013R090
	J81			M013R100
	709-10			M013R110
	11W1			M013R120
	J6			M013R130
	M63			M013R140
	60W6		1W6=FSBLS	M013R150
	109-100			M013R160
	J100			M013R170
	11W6			M013R180
	J71			M013R190
	J5			M013R200
	709-10	J36		M013R210
9-10	11W5			M013R220
	J82			M013R230
	70J36			M013R240
	11W1			M013R250
	J6			M013R260
	M63			M013R270
	60W6		1W6=FSBLS	M013R280
	109-200			M013R290
	J100			M013R300
	11W6			M013R310
	J71			M013R320
	J5	J36		M013R330
9-100	60W2		1W2=DEF	M013R340

MAKE FREE VARIABLES DISJOINT.	11W1		M013R350
FIND LEFT SUB SEG OF DEF.	11W2		M013R360
	M110		M013R370
	P13		M013R380
	70J4		M013R390
	11W0		M013R400
MATCH, MAKE DEF LIKE PROB IF CAN.	M113		M013R410
	70J4		M013R420
	60W3	1W3=SUBSTL	M013R430
	11W2		M013R440
	P14		M013R445
FIND RIGHT SIDE OF DEFINITION.	70	9-300	M013R450
IF NONE,	J72	J4	M013R455
ERASE SUBST. LIST, QUIT + FOR GEN.	9-200 60W2	1W2=DEF	M013R460
TRY RIGHT SIDE OF (0) W/1W0.	11W1		M013R470
	11W2		M013R480
	M110		M013R490
MAKE FREE VARIABLES DISJOINT.	P14		M013R500
FIND RIGHT SUB SEG OF DEF.	70J4		M013R510
	11W0		M013R520
	M113		M013R530
MATCH, MAKE DEF LIKE PROB IF CAN.	70J4		M013R540
	60W3	1W3=SUBSTL	M013R550
	11W2		M013R560
	P13		M013R565
FIND LEFT SIDE OF DEFINITION.	70	9-300	M013R570
IF NONE,	J72	J4	M013R575
ERASE SUBST. LIST, QUIT + FOR GEN.	9-300 P17		M013R580
MAKE SUBPROB FROM SEGMENT (0)	P24		M013R590
WITH SUBSTITUTION LIST (1)	60W4	1W4=NEWTEX	M013R600
	M115		M013R610
	9-301		M013R620
SUBSTITUTE INTO NEWTEX FROM SUBSTL.	11W4		M013R630
	11W1		M013R640
	11W2		M013R650
	10M13		M013R660
ASSIGN DERIVATION, ADD FOUND LIST,	M19		M013R670
MEASURE UTILITY, ERASE IF NO GOOD.	70J4		M013R680
	M12	J5	M013R700
TRY SUBSTITUTION, H5+ OUTPUT PROOF.	9-301 11W3		M013R710
	J72	J4	M013R720
ERASE SUBSTITUTION LIST.			R
	1		M014R000
M14 FORWARD CHAINING METHOD FOR	M14 J44		M014R010
PROBLEM (0). ADDS NEW SUBPROBS	60W1	1W1=PROB	M014R020
TO UNTRIED LIST IF CAN.	P13		M014R030
H5+ MEANS OUTPUT (0) IS	70J34		M014R040
SOLUTION.	609-1		M014R050
H5- MEANS NO SOLUTION, NO OUTPT	51W1		M014R060
FIND MAIN CONNECTIVE OF PROB.	P16		M014R070
	70J34		M014R080
	10L5		

FIND APPROPRIATE THEOREM MAPS.

FIND MAP OF LEFT SIDES.

INPUT FAKE TEX.

GET FEASIBLE THEOREM LEFT SIDES.

TRY FEASIBLE THM LEFTS WITH
PROBLEM LEFT. GENERATE FSBLs.

ERASE LIST OF FEASIBLES.

FAKE TEX . . . 9-1 HOLDS MEX.
9-100 SUBPROCESS, TRY LEFT SIDES.

MAKE FREE VARIABLES DISJOINT.
FIND LEFT SEGMENT OF THM TEX.

INPUT PROB LEFT
MATCH, OUTPUT LIST OF SUBSTITUTIONS
WILL MAKE THM LIKE PROB IF CAN.

FIND RIGHT SIDE OF PROB

FIND RIGHT SIDE OF THEOREM.

CREATE NEW TEX WITH COPIES.

THM LEFT, PROB RIGHT.

SUBSTITUTE INTO NEW TEX.
ERASE SUBSTL.

ASSIGN DERIVATION, ADD TO FOUND LIST
MEASURE UTILITY, ERASE IF NO, GOOD.
TRY SUBSTITUTION, H5+ OUTPUT PROOF.

ERASE SUBSTITUTION LIST.

M15 BACKWARD CHAINING METHOD FOR
PROBLEM (0). ADDS NEW SUBPROBS
TO UNTRIED LIST IF CAN.

J6				M014R090
J10				M014R100
70J34				M014R110
J81				M014R120
70J34				M014R130
109-0				M014R140
J6				M014R150
M63				M014R160
60W0			1W0=FSBLS	M014R170
109-100				M014R180
J100				M014R190
11W0				M014R200
J71				M014R210
J5		J34		M014R220
9-0	0	9-1		M014R230
9-100	60W2		1W2=THM	M014R240
	11W1		1W1=PROB	M014R250
	11W2			M014R260
	M110			M014R270
	P13			M014R280
70J4				M014R290
119-1				M014R300
M113				M014R310
70J4				M014R320
60W3			1W3=SUBSTL	M014R330
51W1				M014R340
P14				M014R350
709-101				M014R360
11W2				M014R370
P14				M014R380
709-102				M014R390
P22				M014R400
60W4			1W4=NEWTEX	M014R405
11W3				M014R410
J6				M014R420
M115				M014R430
9-101				M014R510
11W4				M014R520
11W1				M014R530
11W2				M014R540
10M14				M014R550
M19				M014R560
70J4				M014R570
M12	J5			M014R590
9-102	30HC			M014R600
9-101	11W3			M014R610
	J72	J4		M014R620
			R	
	J44			M015R000
	60W1		1W1=PROB	M015R010
	P14			M015R020

H5+ MEANS OUTPUT (0) IS A SOLUTION.
H5- MEANS NO SOLUTION, NO OUTPT

FIND APPROPRIATE MAPS.

FIND MAP OF RIGHT SIDES.

GET FEASIBLE THEOREM RIGHT SIDES.

TRY FEASIBLE THM RIGHTS WITH
PROBLEM RIGHT. GENERATE FSBLs.

ERASE LIST OF FEASIBLES.

FAKE TEX . . . 9-1 HOLDS MEX.
9-100 SUBPROCESS, TRY RIGHT SIDES.

MAKE FREE VARIABLES DISJOINT.
FIND RIGHT SEGMENT OF THM TEX.

INPUT PROB RIGHT.
MATCH, OUTPUT LIST OF SUBSTITUTIONS
WILL MAKE THM LIKE PROB IF CAN.

FIND LEFT SIDE OF THEOREM.

FIND LEFT SIDE OF PROBLEM.

CREATE NEW TEX WITH COPIES.

PROB ON LEFT, THM ON RIGHT.

SUBSTITUTE INTO NEW TEX.
ERASE SUBSTL.

ASSIGN DERIVATION, ADD TO FOUND LIST
MEASURE UTILITY, ERASE IF NO GOOD.

TRY SUBSTITUTION, H5+ OUTPUT PROOF.

ERASE SUBSTITUTION LIST.

70J34			MO15R030
609-1	9-0=FAKTEX	MO15R040	
5CL5	L5 IS MAP	MO15R050	
11K6	1K6=IMPLY	MO15R060	
J10		MO15R070	
70J34		MO15R080	
J82		MO15R090	
70J34		MO15R100	
109-0		MO15R110	
J6		MO15R120	
M63		MO15R130	
60W0	1W0=FSBLS	MO15R140	
109-100		MO15R150	
J100		MO15R160	
11W0		MO15R170	
J71		MO15R180	
J5	J34	MO15R190	
0	9-1	MO15R200	
60W2	1W2=THM	MO15R210	
11W1		MO15R220	
11W2		MO15R230	
M110		MO15R240	
P14		MO15R250	
70J4		MO15R260	
119-1		MO15R270	
M113		MO15R280	
70J4		MO15R290	
60W3	1W3=SUBSTL	MO15R300	
51W2		MO15R310	
P13		MO15R320	
709-101		MO15R330	
11W1		MO15R340	
P13		MO15R350	
709-102		MO15R360	
P22		MO15R370	
60W4	1W4=NEWTEX	MO15R375	
11W3		MO15R380	
J6		MO15R390	
M115		MO15R400	
9-101		MO15R480	
11W4		MO15R490	
11W1		MO15R500	
11W2		MO15R510	
10M15		MO15R520	
M19		MO15R530	
70J4		MO15R540	
M12	J5	MO15R560	
30HC		MO15R570	
11W3		MO15R580	
J72	J4	MO15R590	

M16 SUBLEVEL REPLACEMENT METHOD FOR	M16	J48		M016R000
PROBLEM (0). ADD ALL NEW		60W0	1W0=PROB	M016R010
SUBPROBLEMS TO UNTRIED LIST.		50L5		M016R020
H5-MEANS NO SOL'N, NO OUTPUT.		11K5		M016R030
H5+MEANS (0) IS A SOL'N.		J10		M016R040
(M16 TRIES ONE LEVEL AT A TIME)		70J38		M016R050
		60W2		M016R060
FIND MAP OF DEF. LEFT SIDES.		J81		M016R070
IF NONE, QUIT-.		70J38		M016R080
		60W1	1W1=LFTMAP	M016R090
		51W2		M016R100
FIND MAP OF DEF. RIGHT SIDES.		J82		M016R110
IF NONE, QUIT-.		70J38		M016R120
		60W2	1W2=RTMAP	M016R130
		51W0		M016R140
		40H0		M016R142
		10Q17		M016R145
CLEAR LEVEL		J14		M016R148
FIND LOWEST LEVEL IN PROBLEM.		Q17		M016R150
IF NONE, QUIT-. IF YES,		70J38		M016R160
		20W3	1W3=CURLEV	M016R165
BUMP 1W3, TEST IF GREATER THAN 1.	9-3	9-20		M016R170
IF NOT, QUIT-.		70J38		M016R175
IF YES,	9-10	11W0		M016R180
COPY PROBLEM FOR REPLACEMENT.		P25		M016R190
		60W4	1W4=CPROB	M016R200
	9-11	10J3		M016R210
SET 'NEW SUBPROBLEM FLAG' TO NO.		60W5	1W5=FLAG	M016R220
		509-100		M016R230
GENERATE SEGMENT LOCATIONS.		P26		M016R240
EXECUTE 'NEW SUBPROBLEM FLAG'		01W5		M016R250
IF YES, GO FINISH IT UP.		70	9-1	M016R260
BUMP 1W3 AND TEST IF GREATER THAN 1		9-20		M016R270
IF NOT, GO CLEAN UP, QUIT-.		709-2		M016R280
IF GREATER, COPY 1W4 AND		J120		M016R290
		10Q17		M016R300
		11W4		M016R310
ASSIGN COPY TO PROBLEM COPY.		J11		M016R320
		11W3		M016R330
LOOP FOR NEXT LEVEL.		11W4	9-11	M016R340
ALL DONE,	9-2	11W4		M016R350
ERASE LEFTOVER COPY, QUIT.		J72	J38	M016R360
SET UP DERIVATION ASSOCIATIONS,	9-1	11W4		M016R370
		11W0		M016R380
AND		10/16		M016R390
		10M16		M016R400
FINISH BUILDING THE NEW SUBPROBLEM.		M19		M016R410
IF NO GOOD, SET UP TO LOOP.		709-3		M016R420
IF GOOD, TRY SUBSTITUTION.		M12		M016R430
IF PROOF, QUIT+. IF NOT, LOOP.		709-3	J38	M016R440
9-20 SUBPROCESS--BUMP AND TEST	9-20	10N2		M016R470

IF 1W3 GREATER THAN 1.	11W3			MO16R480
TEST IF 1W3 GREATER THAN N2.	J115			MO16R490
IF NO, QUIT SUBPROCESS--.	700		NO OUTPUT	MO16R500
IF YES,	10N1			MO16R510
	11W3			MO16R520
SUBTRACT 1, QUIT SUBPROCESS+.	11W3	J111	OUTPUT 1W3	MO16R530
9-100 SUBPROCESS FOR SUBSEGMENT	60W6		1W6=SEGLOC	MO16R540
REPLACEMENT IN LOCATION (0).	51W1			MO16R550
	12W6			MO16R560
	M62			MO16R570
CREATE LIST OF FEASIBLE DEFS.	40H0			MO16R580
SAVE LIST FOR LATER ARASURE.	109-200			MO16R590
	J100			MO16R600
GENERATE DEFS FOR LEFT SIDE MATCH.	709-101			MO16R610
IF MATCHED, GO CLEAN UP.	J71			MO16R620
IF FAILED, ERASE OLD LIST,	11W2			MO16R630
	12W6			MO16R640
	M62			MO16R650
CREATE NEW LIST OF FEASIBLE DEFS.	40H0			MO16R660
SAVE LIST FOR LATER ERASURE.	109-300			MO16R670
	J100			MO16R680
GENERATE DEFS FOR RIGHT SIDE MATCH.	9-101	J71	J4	MO16R690
ERASE LIST AND QUIR + FOR GEN.	9-200	60W7		1W7=DEF MO16R700
9-200 SUBPROCESS, TRY REPLACEMENT	11W4			1W4=CPROB MO16R710
BY MATCHING SEGMENT TO	11W7			MO16R720
LEFT SIDES.	M110			MO16R730
MAKE FREE VARIABLES DISJOINT.	P13			MO16R740
FIND LEFT SEGMENT OF DEF.	70J4			MO16R750
IF NONE, QUIT + FOR GEN.	12W6			MO16R760
IF FOUND,	M113			MO16R770
MATCH SEGMENT TO LEFT SIDE.	70J4			MO16R780
IF NO MATCH, QUIT + FOR GEN.	60W8			1W8=SUBST MO16R790
IF MATCHED, SAVE LIST,	51W7			MO16R800
	P14	9-301		MO16R810
FIND RIGHT SIDE OF DEF.	9-300	60W7		1W7=DEF MO16R820
9-300 SUBPROCESS, TRY REPLACEMENT	11W4			1W4=CPROB MO16R830
BY MATCHING SEGMENT TO	11W7			MO16R840
RIGHT SIDES.	M110			MO16R850
MAKE FREE VARIABLES DISJOINT	P14			MO16R860
FIND RIGHT SEGMENT OF DEF.	70J4			MO16R870
IF NONE, QUIT + FOR GEN.	12W6			MO16R880
IF FOUND,	M113			MO16R890
MATCH SEGMENT TO RIGHT SIDE.	70J4			MO16R900
IF NO MATCH, QUIT + FOR GEN.	60W8			1W8=SUBST MO16R910
IF MATCHED, SAVE LIST,	51W7			MO16R920
	P13			MO16R930
FIND LEFT SIDE OF DEF.	9-301	709-302		MO16R935
IF NONE, CLEAN UP, QUIT +.	J74			MO16R940
IF FOUND, COPY IT,	12W6			MO16R945
	J72			MO16R950
ERASE OLD SEGMENT, AND	21W6			MO16R955
REPLACE OLD WITH COPY FROM DEF.				

		11W8		M016R960
		11W4		M016R965
SUBSTITUTE IN PROB TEX PER 1W8.		M115		M016R970
		10J4		M016R975
SET 'NEW SUBPROBLEM FLAG' TO ON.		20W5		M016R980
SET H5 TO QUIT-.		J4		M016R985
	9-302	11W8		M016R990
ERASE 1W8, REVERSE H5 FOR GEN.		J72	J5	M016R995
				R
M17 SUBLEVEL REPLACEMENT METHOD FOR	1	J48		M017R000
PROBLEM (0). ADD ALL NEW	M17	60W0	1W0=PROB	M017R010
SUBPROBLEMS TO UNTRIED LIST.		50L5		M017R020
H5-MEANS NO SOL'N, NO OUTPUT.		11K5		M017R030
H5+MEANS (0) IS A SOL'N.		J10		M017R040
(M17 TRIES ALL LEVELS A ONCE)		70J38		M017R050
		60W2		M017R060
FIND MAP OF DEF. LEFT SIDES.		J81		M017R070
IF NONE, QUIT-.		70J38		M017R080
		60W1	1W1=LFTMAP	M017R090
		51W2		M017R100
FIND MAP OF DEF. RIGHT SIDES.		J82		M017R110
IF NONE, QUIT-.		70J38		M017R120
		60W2	1W2=RTMAP	M017R130
		51W0		M017R140
		40H0		M017R142
		10Q17		M017R145
CLEAR LEVEL		J14		M017R148
FIND LOWEST LEVEL IN PROBLEM.		Q17		M017R150
IF NONE, QUIT-. IF YES,		70J38		M017R160
		20W3	1W3=CURLEV	M017R165
BUMP 1W3, TEST IF GREATER THAN 1.	9-3	9-20		M017R170
IF NOT, QUIT-.		70J38		M017R175
IF YES,	9-10	11W0		M017R180
COPY PROBLEM FOR REPLACEMENT.		P25		M017R190
		60W4	1W4=CPROB	M017R200
	9-11	10J3		M017R210
SET 'NEW SUBPROBLEM FLAG' TO NO.		60W5	1W5=FLAG	M017R220
		509-100		M017R230
GENERATE SEGMENT LOCATIONS.		P26		M017R240
BUMP 1W3 AND TEST IF GREATER THAN 1		9-20		M017R250
IF NOT, GO TEST FLAG.		709-2		M017R260
IF GREATER, COPY 1W4 AND		J120		M017R270
		10Q17		M017R280
		11W4		M017R290
ASSIGN COPY TO PROBLEM COPY.		J11		M017R300
		11W3		M017R310
LOOP FOR NEXT LEVEL.		11W4	9-11	M017R320
EXECUTE 'NEW SUBPROBLEM FLAG'	9-2	01W5		M017R330
IF YES, GO FINISH IT UP.		70	9-1	M017R340
ALL DONE,		11W4		M017R350
ERASE LEFTOVER COPY, QUIT.		J72	J38	M017R360

SET UP DERIVATION ASSOCIATIONS,	9-1	11W4			MO17R370
		11W0			MO17R380
AND		10/16			MO17R390
		10M16			MO17R400
FINISH BUILDING THE NEW SUBPROBLEM.		M19			MO17R410
IF NO GOOD, QUIT -.		70J38			MO17R420
IF GOOD, TRY SUBSTITUTION.		M12			MO17R430
IF PROOF, QUIT+. IF NOT, QUIT-		J38	0		MO17R440
9-20 SUBPROCESS--BUMP AND TEST	9-20	10N2			MO17R470
IF 1W3 GREATER THAN 1.		11W3			MO17R480
TEST IF 1W3 GREATER THAN N2.		J115			MO17R490
IF NO, QUIT SUBPROCESS--.		700		NO OUTPUT	MO17R500
IF YES,		10N1			MO17R510
		11W3			MO17R520
SUBTRACT 1, QUIT SUBPROCESS+.		11W3	J111	OUTPUT 1W3	MO17R530
9-100 SUBPROCESS FOR SUBSEGMENT	9-100	60W6		1W6=SEGLOC	MO17R540
REPLACEMENT IN LOCATION (0).		51W1			MO17R550
		12W6			MO17R560
CREATE LIST OF FEASIBLE DEFS.		M62			MO17R570
SAVE LIST FOR LATER ARASURE.		40H0			MO17R580
		109-200			MO17R590
GENERATE DEFS FOR LEFT SIDE MATCH.		J100			MO17R600
IF MATCHED, GO CLEAN UP.		709-101			MO17R610
IF FAILED, ERASE OLD LIST,		J71			MO17R620
		11W2			MO17R630
		12W6			MO17R640
CREATE NEW LIST OF FEASIBLE DEFS.		M62			MO17R650
SAVE LIST FOR LATER ERASURE.		40H0			MO17R660
		109-300			MO17R670
GENERATE DEFS FOR RIGHT SIDE MATCH.		J100			MO17R680
ERASE LIST AND QUIR + FOR GEN.	9-101	J71	J4		MO17R690
9-200 SUBPROCESS, TRY REPLACEMENT	9-200	60W7		1W7=DEF	MO17R700
BY MATCHING SEGMENT TO		11W4		1W4=CPROB	MO17R710
LEFT SIDES.		11W7			MO17R720
MAKE FREE VARIABLES DISJOINT.		M110			MO17R730
FIND LEFT SEGMENT OF DEF.		P13			MO17R740
IF NONE, QUIT + FOR GEN.		70J4			MO17R750
IF FOUND,		12W6			MO17R760
MATCH SEGMENT TO LEFT SIDE.		M113			MO17R770
IF NO MATCH, QUIT + FOR GEN.		70J4			MO17R780
IF MATCHED, SAVE LIST,		60W8		1W8=SUBST	MO17R790
		51W7			MO17R800
FIND RIGHT SIDE OF DEF.		P14	9-301		MO17R810
9-300 SUBPROCESS, TRY REPLACEMENT	9-300	60W7		1W7=DEF	MO17R820
BY MATCHING SEGMENT TO		11W4		1W4=CPROB	MO17R830
RIGHT SIDES.		11W7			MO17R840
MAKE FREE VARIABLES DISJOINT		M110			MO17R850
FIND RIGHT SEGMENT OF DEF.		P14			MO17R860
IF NONE, QUIT + FOR GEN.		70J4			MO17R870
IF FOUND,		12W6			MO17R880
MATCH SEGMENT TO RIGHT SIDE.		M113			MO17R890

IF NO MATCH, QUIT + FOR GEN. IF MATCHED, SAVE LIST,	70J4 60W8 51W7 P13	1W8=SUBST	M017R900 M017R910 M017R920 M017R930
FIND LEFT SIDE OF DEF. IF NONE, CLEAN UP, QUIT +. IF FOUND, COPY IT,	9-301 709-302 J74 12W6 J72 21W6 11W8 11W4 M115 10J4 20W5 J4		M017R935 M017R940 M017R945 M017R950 M017R955 M017R960 M017R965 M017R970 M017R975 M017R980 M017R985
ERASE OLD SEGMENT, AND REPLACE OLD WITH COPY FROM DEF.			M017R990 M017R995
SUBSTITUTE IN PROB TEX PER 1W8.	9-302 11W8 J72	J5	R
SET 'NEW SUBPROBLEM FLAG' TO ON. SET H5 TO QUIT-.			
ERASE 1W8, REVERSE H5 FOR GEN.			
M19 FINISH BUILDING NEW SUBPROBLEM (3) FROM (2) VIA THM(1) BY METHOD (0). MEASURE UTILITY. H5-, NO OUTPUT, TEX ERASED. H5+ MEANS OUTPUT (0) IS OK.	1 M19 J53 11W3 11W2 10Q10 J11 11W3 11W0 10Q11 J11 11W3 11W1 10Q12 J11 11W3 J136 M43 11W3 709-1 M51 11W3 J33	1W3=NEWTEX 1W2=PROBLMM 1W1=THM 1W0=METHOD	M019R000 M019R010 M019R020 M019R030 M019R040 M019R050 M019R060 M019R070 M019R080 M019R090 M019R100 M019R110 M019R120 M019R130 M019R135 M019R140 M019R150 M019R160 M019R170 M019R180 M019R190
FILL QUIT DESCRIPTION.			M019R200 M019R210 M019R220 M019R230 M019R240 M019R250 M019R260 M019R270
MARK LOCAL FOR FOUND LIST. EVALUATE UTILITY.			
IF N.G., REJECT IT. IF O.K., ADD TO UNTRIED LIST.			
QUIT +, OUTPUT NEW PROBLEM.	9-1 11K31 10Y J2 709-2 M81 11W3 J72 J33	J4	
TEST IF PRINTING REJECTS. IF NO, SKIP IT. IF YES, PRINT IT.			
ERASE N. G. SUBPROBLEM. QUIT -.	9-2 J72 J33	J3	
M40 TEST IF TOTAL EXPRESSIONS (0) AND (1) MATCH. FIND MAIN SEGMENT.	1 M40 J51 11W1 J81	1W0=TEX1 1W1=TEX2	M040R000 M040R010 M040R020

		709-0		M040R030
		11W0		M040R040
FIND MAIN SEGMENT.		J81		M040R050
		709-1		M040R060
TEST IF SEGMENTS MATCH.		M41	J31	M040R070
TEST IF OTHER MAIN SEGMENT	9-0	11W0		M040R080
EXISTS.		J81		M040R090
		J5		M040R100
NO, QUIT W/H5+ FOR MATCH		709-1	J31	M040R110
YES, QUIT W/H5- FOR NO MATCH.	9-1	30H0	J31	M040R120
	1			R
M41 TEST IF SEGMENTS (0) AND (1)	M41	J51	1W0=SEG1	M041R000
MATCH.		11W0	1W1=SEG2	M041R010
TEST IF 1ST IS VARIABLE.		P8		M041R020
IF NOT, EXAMINE SUBSEGMENTS.		709-1		M041R030
IF YES,		11W0		M041R040
		11W1		M041R050
TEST IF SAME VARIABLE.		J2		M041R060
IF YES, QUIT, H5+.		70	J31	M041R070
IF NO,		11W1		M041R080
TEST IF 2ND IS FREE VARIABLE.		P9		M041R090
IF NO, QUIT, H5-.		70J31		M041R100
IF YES,		11W0		M041R110
TEST IF 1ST IS FREE VARIABLE.		P9	J31	M041R120
EXAMINE SUBSEGMENTS.	9-1	11W1		M041R130
TEST IF NOT SEGMENT.		P8		M041R140
IF NOT, QUIT, H5-		70	9-2	M041R150
IF YES,		12W0		M041R160
		12W1		M041R170
TEST IF SAME CONNECTIVES.		J2		M041R180
IF NOT, QUIT, H5-		70J31		M041R190
IF YES,		11W0		M041R200
FIND LEFT SUBSEGMENT.		J81		M041R210
IF NONE, CHECK OTHER SIDE.		709-3		M041R220
		11W1		M041R230
FIND OTHER LEFT SUBSEGMENT.		J81		M041R240
IF NONE, QUIT, H5-.		709-7		M041R250
TEST IF LEFT SEGMENTS MATCH.		M41		M041R260
IF NOT, QUIT, H5-		70J31		M041R270
		11W0		M041R280
FIND RIGHT SUBSEGMENT.		J82		M041R290
IF NONE, CHECK OTHER SIDE.		709-4		M041R300
		11W1		M041R310
FIND OTHER RIGHT SUBSEGMENTS.		J82		M041R320
IF NONE, QUIT, H5-		709-7		M041R330
TEST IF RIGHT SEGMENTS MATCH,		M41	J31	AND QUIT. M041R340
QUIT, REVERSE H5.	9-2	J5	J31	M041R350
NO FIRST SEGMENT ON 1W0,	9-3	11W1		M041R360
FIND FIRST SEGMENT ON 1W1.		J81		M041R370
IF NONE, H5+. ELSE H5-.	9-5	709-2	9-6	M041R380
NO SECOND SEGMENT ON 1W0,	9-4	11W1		M041R390

FIND SECOND SEGMENT ON 1W1.		J82	9-5		M041R400
REVERSE H5 AND	9-6	J5			M041R410
QUIT, DISCARD (0).	9-7	30H0	J31		M041R420
	1				R
M42 ADD PROBLEM (0) TO FOUND LIST.	M42	J42			M042R000
IF CANNOT, SET H5 -.		60W0		1W0=PROB	M042R010
		50L11			M042R030
		11W0			M042R040
GET NUMBER OF LEVELS		Q2			M042R050
IF NONE, QUIT -.		709-300			M042R055
GET SUBLIST		9-100			M042R060
		11W0			M042R070
GET NUMBER OF DISTINCT VARIABLES		Q3			M042R080
IF NONE, QUIT -.		709-300			M042R085
GET SUBLIST		9-100			M042R090
		11W0			M042R100
GET NUMBER OF VARIABLE PLACES		Q4			M042R110
IF NONE, QUIT -.		709-300			M042R115
GET SUBLIST		9-100			M042R120
		40H0			M042R130
		109-200			M042R140
GENERATE SUBLIST FOR MATCH		J100			M042R150
		709-1			M042R160
INSERT AT END OF LIST		11W0			M042R170
		J32	J65		M042R180
	9-1	30H0	J32		M042R190
9-200 SUBPROCESS.	9-200	11W0			M042R200
COMPARE EXPRESSIONS		M40	J5		M042R210
9-100 SUBPROCESS, GET SUBLIST.	9-100	64W1		1W1=D.T.	M042R220
LOCATE SUBLIST.		P55			M042R230
		70	J80		M042R240
		40H0			M042R250
CREATE NEW SUBLIST.		J90			M042R260
		J136			M042R270
SAVE SUBLIST FOR OUTPUT.		60W2		1W2=SUBLSTM	M042R280
INSERT NEW SUBLIST.		J64			M042R290
		11W1			M042R300
COPY DATA TERM.		J120			M042R310
MARK LOCAL.		J136			M042R320
INSERT BEFORE NEW SUBLIST.		J64			M042R330
GET SUBLIST AND QUIT.		11W2	0		M042R340
	9-300	30H0	J32		M042R350
	1				R
M43 MEASURE UTILITY SUBPROBLEM (0).	M43	J43			M043R000
SET H5+ IF GOOD, H5- IF N.G.		60W0		1W0=PROB	M043R010
FIND MEX.		J81			M043R020
		70J33			M043R030
		P4			M043R040
GO THRU 'NOTS'		70J33			M043R045
		60W1		1W1=MEX	M043R050
SAVE UNNOTTED MEX.		P8			M043R060
TEST IF VARIABLE					

IF H5--, QUIT (VARIABLE ONLY)	J5		M043R070
	70J33		M043R080
	12W1	2W1=MCONN	M043R090
	11K1	1K1='OR'	M043R100
TEST IF MAIN CONNECTIVE 'OR'	J2		M043R110
IF NOT 'OR', LOOK ON FOUND LIST	709-10		M043R120
	11W1		M043R130
LOCATE RIGHT SIDE	J60		M043R140
	709-0		M043R150
GET RIGHT SIDE	12H0		M043R160
	J6		M043R170
LOCATE LEFT SIDE	J60		M043R180
	709-1		M043R190
GET LEFT SIDE	52HC		M043R200
TEST IF SIDES MATCH.	M114		M043R210
	J5		M043R220
IF SAME QUIT W/H5-	70J33		M043R230
	9-10		M043R240
ADD TO FOUND LIST IF CAN	11WC		M043R260
	M42	J33	M043R270
	9-1		M043R280
	9-0	J33	R
	1		M050R000
M50 ADD TEX(0) TO TRUE EXPRESSIONS	M50		M050R010
LIST AND TRUE EXPRESSIONS MAP.	40H0		M050R020
MAKE ALL VARIABLES FREE.	J50		M050R030
	P27		M050R040
	10L1		M050R050
ADD TO LIST	11WC		M050R060
	J65		M050R070
	10L4		M050R080
ADD TO MAP	11WC		M050R090
	M54		M050R100
PRINT EXPRESSION AND QUIT.	11WC		R
	M70	J30	M051R000
	1		M051R010
M51 PRINT NEW SUBPROBLEM (0) AND	M51		M051R020
ADD TO UNTRIED SUBPROBLEM LIST.	J41	1W0=PROB	M051R030
	60WC		M051R040
	10K10		M051R050
TALLY PREVIOUS SUBPROBLEM NUMBER.	J125		M051R060
	J120		M051R070
	J136		M051R080
	10Q8		M051R110
ASSIGN PROBLEM NO.	J11		M051R120
	10L10		M051R125
	11W0		M051R130
FIND NO. OF LEVELS	Q2	1W1=LEVELS	M051R140
IF NONE, QUIT --.	709-3		M051R150
	60W1		M051R160
LOCATE CORRESPONDING LIST.	P55		M051R170
	709-1		
GET LIST.	52H0		
	11W0		

ADD NEW SUBPROBLEM.		J65	9-2	M051R190
	9-1	40H0		M051R200
		11W0		M051R210
CREATE LIST OF ONE SUBPROBLEM.		J91		M051R223
		J136		M051R226
INSERT NEW LIST.		J64		M051R230
		11W1		M051R240
COPY LEVEL DATA TERM.		J120		M051R250
		J136		M051R260
INSERT BEFORE NEW LIST.		J64	9-2	M051R270
	9-2	11W0		M051R280
		M75	J31	M051R290
PRINT NEW SUBPROBLEM.	9-3	30H0	J31	M051R300
	1			R
M54 ADD TOTAL EXPRESSION (0) TO MAP OF TRUE EXPRESSIONS (1).	M54	40W0		M054R000
		60W0		1W0=THMNAME M054R010
		J81		M054R020
		709-0		(0)=MEX M054R030
		J6		M054R040
ADD MAIN SEGMENT (1) TO MAP (0).		9-100	J30	M054R050
	9-0	30H0	J30	M054R060
9-100 SUBPROCESS, ADD SEGMENT (1) TO MAP (0).	9-100	04J43		1W0=THMNAME M054R070
		20W1		1W1=MAP M054R080
		60W2		1W2=SEGMNT M054R090
TEST IF SIMPLE VARIABLE.		P8		M054R100
IF NO, CONTINUE DOWN MAP.		709-102		M054R110
IF YES, ADD THMNAME.		11W1		M054R120
TEST IF NAME LIST IN MAP HEAD.		J79		M054R130
IF NO, GO MAKE ONE.		709-101		M054R140
IF YES,		12W1		M054R150
		11W0		M054R160
INSERT NAME AND QUIT.		J64	J33	M054R170
	9-101	11W0		M054R180
		J91		M054R190
CREATE LIST OF ONE NAME.		J136		M054R200
		21W1	J33	M054R210
PLACE IN MAP HEAD AND QUIT.		10W1		M054R220
INPUT MAP HOLDER.	9-102	12W2		M054R230
INPUT SEGMENT CONNECTIVE.		J10		M054R240
FIND SUBMAPS LIST.		70	9-110	M054R250
IF FOUND, CONTINUE.		10W1		M054R260
IF NONE,		J90		M054R270
		J136		M054R280
CREATE 1ST LOCAL SUBMAP.		12W2		M054R290
		P6		M054R300
TEST IF 2ND SUBLIST NEEDED.		709-111		M054R310
IF NO, SKIP IT.		40H0		M054R320
IF YES,		J120		M054R330
CREATE 2ND LOCAL SUBMAP.		J92	9-112	M054R340
	9-111	J91		M054R350
CREATE SUBMAP LIST.	9-112	J136		M054R360

ASSIGN AS SUBMAP LIST OF CONNECTIVE	60W3	1W3=MAPLSTM054R370
	12W2	M054R380
	J11	M054R390
	11W3	M054R400
9-110	20W3	1W3=MAPLSTM054R410
	11W2	M054R420
FIND 1ST SUB SEGMENT.	J81	M054R430
IF NONE, QUIT.	70J33	M054R440
	11W3	M054R450
FIND 1ST SUB MAP.	J81	M054R460
IF NONE, QUIT.	709-114	M054R470
ADD SEGMENT (1) TO SUBMAP (0).	9-100	M054R480
	12W2	M054R490
TEST IF MORE SEGMENTS.	P6	M054R500
IF NO, QUIT.	70J33	M054R510
	11W2	M054R520
FIND 2ND SUBSEGMENT.	J82	M054R530
IF NONE, QUIT.	70J33	M054R540
	11W3	M054R550
FIND 2ND SUBMAP.	J82	M054R560
IF NONE, QUIT.	709-114	M054R570
ADD SEGMENT (1) TO SUBMAP (0).	9-100	M054R580
	J33	M054R600
	9-114	R
1	30H0	M060R000
M60 FIND NEXT UNTRIED PROBLEM.	M60	10L10
H5 - MEANS NONE REMAINING.	9-1	J60
LOCATE NEXT SUBLIST OF PROBLEMS.	J60	M060R010
IF NONE, QUIT -.	70J8	M060R020
IF SOME, GET SUBLIST AND	12H0	M060R030
FIND FIRST PROBLEM.	J81	M060R040
IF NONE, LOCATE NEXT LIST.	709-1	M060R050
IF FOUND, GET LOCATION OF LIST,	J6	M060R060
GET NAME OF LIST, AND	52H0	M060R070
LOCATE FIRST PROBLEM.	J60	M060R080
IF NONE, MACHINE ERROR--HALT.	70J7	M060R090
IF LOCATED, DELETE FROM LIST,	J68	M060R100
MARK OUTPUT REGIONAL, QUIT+.	J138	M060R110
	J4	M060R120
1		R
M62 CREATE A LIST OF TRUE	M62	J45
EXPRESSIONS FROM MAP (1) FOR	20W0	M062R000
FEASIBLE MATCH WITH SEGMENT	60W1	1W0=SEG M062R010
(0). OUTPUT MAY BE EMPTY.	52W1	1W1=MAP M062R020
	J73	M062R030
SAVE COPY OF LIST IN MAP HEAD.	60W2	M062R040
	11W0	1W2=THMLSTM062R050
TEST IF SEGMENT IS SIMPLE VAR.	P8	M062R060
IF YES, QUIT WITH OUTPUT.	70	M062R070
IF NO,	51W0	M062R080
LOCATE 1ST SUBSEGMENT.	J60	M062R090
IF NONE, OUTPUT 1W2, QUIT.	709-1	M062R100
	60W3	M062R110
		1W3=SEGLOCM062R120

FIND LIST OF APPROPRIATE SUBMAPS.	50W1		M062R130
IF NONE, OUTPUT 1W2, QUIT.	12W0		M062R140
LOCATE 1ST SUBMAP.	J10		M062R150
IF NONE, OUTPUT 1W2, QUIT.	709-0		M062R160
IF THERE, SAVE LOCATION,	J60		M062R170
	709-1		M062R180
SET UP H0 AND	60W4	1W4=MAPLOC	M062R190
CREATE LIST FROM MAP (1) FOR (0).	52W4		M062R200
	12W3		M062R210
	M62		M062R220
TEST IF CONNECTIVE WAS NON-UNARY.	12W0		M062R230
IF UNARY, FIX OUTPUT, QUIT.	P6		M062R240
IF NON-UNARY, SAVE LIST AND	709-2		M062R250
	60W5	1W5=ANDLST	M062R260
LOCATE NEXT SUBSEGMENT.	9-4 51W3		M062R270
	J60		M062R280
IF NONE, FIX OUTPUT, QUIT.	60W3		M062R290
LOCATE NEXT SUBMAP.	709-3		M062R300
	51W4		M062R310
	J60		M062R320
IF NONE, FIX OUTPUT, QUIT.	60W4		M062R330
	709-3		M062R340
	52W4		M062R350
	12W3		M062R360
CREATE LIST FROM MAP (1) FOR (0),	M62		M062R370
'AND' RESULT WITH LIST 1W5, LOOP.	9-100 9-4		M062R380
	9-3 51W5		M062R390
FIX OUTPUT - - -	9-2 11W2		M062R400
'OR' W5 WITH 1W2,	J6		M062R410
LEAVE RESULT AS 1W2.	J76		M062R420
OUTPUT 1W2, CLEAR CONTEXT.	9-1 51W2 J35		M062R430
	9-0 11W2 J35		M062R440
SUBPROCESS - 'AND' (0) WITH 1W5.	9-100 11W5		M062R450
	109-200		M062R460
GENERATE 1W5 FOR PROCESS MARKING.	J100		M062R470
	40H0		M062R480
	109-300		M062R490
GENERATE '(0)' TO UNMARK MARKED.	J100		M062R500
ERASE '(0)'	J71		M062R510
	11W5		M062R520
LOCATE NEXT OF 1W5.	9-101 J60		M062R530
IF NONE, QUIT SUBPROCESS.	700		M062R540
	9-102 12H0		M062R550
TEST IF EXPRESSION MARKED.	J133		M062R560
IF NO, LOOP TO NEXT.	709-101		M062R570
IF YES, UNMARK IT,	32H0		M062R580
SAVE LOCATION AND DELETE THIS	40H0		M062R590
EXPRESSION DUE NOT ON BOTH LISTS.	J68		M062R600
IF MORE, LOOP WITH NEXT.	700 9-102		M062R610
SUBPROCESS - MARK PROCESSED.	9-200 J137 J8		M062R620
SUBPROCESS - UNMARK IF MARKED.	9-300 40H0		M062R630

		J133		M062R640
		709-201		M062R650
		31H0	J8	M062R660
	9-201	30H0	J4	M062R670
	1			R
M63 CREATE A LIST OF TRUE	M63	J6		M063R000
EXPRESSIONS FROM MAP (0) FOR		J81		M063R010
FEASIBLE MATCH WITH TEX (1).		70	M62	M063R020
OUTPUT MAY BE AN EMPTY LIST.		30H0	J90	M063R030
	1			R
M70 PRINT EXPRESSION (0), WITH OR	M70	J154		M070R000
WITHOUT A SUFFIX.		40H0		M070R010
ENTER NAME.		M79		M070R020
		10N8		M070R030
TAB TO COLUMN (0).		J160		M070R040
		40H0		M070R050
FIND MEX.		J81		M070R060
IF NONE, SKIP IT.		709-1		M070R070
ENTER MEX.		M73		M070R080
FIND SUFFIX.	9-1	Q18		M070R090
IF NONE, PRINT WITHOUT IT.		70J155		M070R100
		10N1		M070R110
BUMP COLUMN.		J161		M070R120
ENTER SUFFIX AND PRINT.		J157	J155	M070R130
	1			R
M71 PRINT PROOF SEQUENCE FROM (0).	M71	J50	1W0=TEX	M071R000
		J154		M071R010
		J155		M071R020
SKIP TWO LINES.		J155		M071R030
		10T2		M071R040
ENTER 'PROOF FOUND' AND		M76		M071R050
PRINT.		J155		M071R060
		J154		M071R070
SKIP ONE LINE.		J155		M071R080
		11W0		M071R090
FIND PROVING THEOREM.		Q13		M071R100
		70	9-1	M071R110
IF NONE, USE DUMMY CHARACTER.		10/14		M071R120
ENTER 'GIVEN'.	9-1	10T1		M071R130
PRINT FIRST LINE OF PROOF.		M80		M071R140
INPUT TEX AND		11W0		M071R150
'SUBSTITUTION'.		10T3		M071R160
PRINT NEXT EVEN LINE.	9-6	M80		M071R170
		11W0		M071R180
FIND THEOREM USED IN DERIVATION.		Q12		M071R190
IF NONE, FINISH WITH Q.E.D.		709-2		M071R200
		10T1		M071R210
PRINT NEXT ODD LINE.		M80		M071R220
		11W0		M071R230
FIND METHOD OF DERIVATION		Q11		M071R240
		709-3		M071R250

FIND EXTERNAL NAME OF METHOD.		Q16		M071R260
		70	9-4	M071R270
IF NONE, USE BLANKS.	9-3	10T5		M071R280
	9-4	11W0		M071R290
FIND PROBLEM USED IN DERIVATION.		Q10		M071R300
		70	9-5	M071R310
IF NONE, USE DUMMY CHARACTER.		10/14		M071R320
	9-5	60W0		M071R330
LOOP TO PRINT NEXT EVEN LINE.		J6	9-6	M071R340
	9-2	J154		M071R350
		10K41		M071R360
TAB TO COLUMN K41.		J160		M071R370
		10T4		M071R380
ENTER 'Q.E.D.' AND		M76		M071R390
PRINT.		J155		M071R400
PRINT LIMITS, CLEAR CONTEXT/QUIT.		M77	J30	M071R410
	1			R
PRINT -NO PROOF FOUND-	M72	J154		M072R000
		10T6		M072R010
ENTER MESSAGE		M76		M072R020
PRINT MESSAGE, LIMITS		J155	M77	M072R030
	1			R
ENTER SEGMENT (0)	M73	40H0		M073R000
TEST IF VARIABLE		P8		M073R010
IF YES, ENTER VARIABLE.		70	M79	M073R020
		12H0		M073R030
		11K2		M073R040
TEST IF CONNECTIVE NOT		J2		M073R050
		709-1		M073R060
		12H0		M073R070
ENTER NOT.		M79		M073R080
		J81		M073R090
		70J7	9-200	M073R095
		700	9-200	M073R100
	9-1	J41		M073R110
		60W0	1W0=SEG	M073R120
LOCAT FIRST SEGMENT.		J60		M073R130
		20W1	1W1=LOC 'N	M073R140
IF NONE, QUIT.		70J31		M073R150
		12W1	2W1=SUBSEG	M073R160
ENTER SEGMENT.		9-200		M073R170
		11W1		M073R180
LOCATE NEXT SUBSEGMENT.	9-2	J60		M073R190
		20W1		M073R195
IF NONE, QUIT.		70J31		M073R200
		12W0	2W0=CONN	M073R205
ENTER CONNECTIVE.		M79		M073R210
		12W1		M073R215
ENTER SEGMENT.		9-200		M073R220
		11W1	9-2	M073R225
	9-200	40H0		M073R230

		P4		M073R235
		709-201		M073R238
TEST IF VARIABLE		P8		M073R240
		70	M73	M073R250
		10K51		M073R260
ENTER LEFT PAREN.		J157		M073R270
ENTER SUBEXPRESSION		M73		M073R280
ENTER RIGHT PAREN.		10K52	J157	M073R290
	9-201	50/14	M79	M073R300
	1			R
M74 ENTER TOTAL EXPRESSION (0), WITH OR WITHOUT SUFFIX.	M74	40H0		M074R000
		J81		M074R010
		70J8		M074R020
ENTER MAIN EXPRESSION.		M73		M074R030
FIND SUFFIX.		Q18		M074R040
IF NONE, QUIT.		700		M074R050
IF ONE,		10N1		M074R060
BUMP COLUMN AND ENTER SUFFIX.		J161	J157	M074R070
	1			R
M75 PRINT NEW SUBPROBLEM (0).	M75	J154		M075R000
		10K47		M075R010
TAB TO COLUMN K47.		J160		M075R020
ENTER SUBPROBLEM NAME (NO.).		40H0	PSV PROB	M075R030
		M79		M075R040
BUMP COLUMN.		10N3		M075R050
		J161		M075R060
ENTER SUBPROBLEM EXPRESSION.		40H0	PSV PROB	M075R070
		M74		M075R080
		11W25		M075R090
		10K48		M075R100
TEST IF EXPRESSION WAS TOO BIG.		J116		M075R110
IF YES, DON'T RESET.		70	9-2	M075R120
IF NO, RESET TO K48.		10K48		M075R130
		J160		M075R140
	9-2	10N2		M075R160
BUMP COLUMN.		J161		M075R170
		40H0		M075R180
FIND THEOREM.		Q12		M075R190
IF NONE, SKIP IT.		709-1		M075R200
		M79		M075R210
ENTER COMMA.	9-1	10K54		M075R220
		J157		M075R230
		10N1		M075R240
BUMP COLUMN.		J161		M075R250
FIND METHOD.		Q11		M075R260
IF NONE, PRINT NOW.		70J155		M075R270
FIND EXTERNAL NAME.		Q16		M075R280
IF NONE, PRINT NOW.		70J155		M075R290
ENTER TEXT AND PRINT.		M76	J155	M075R300
	1			R
M76 ENTER LIST OF DATA TERMS.	M76	10J157		M076R000

		J100	0		M076R010
	1				R
M77 PRINT LIMITS OF PROCF.		M77	J154		M077R000
			J155		M077R010
DOUBLE SPACE.			J155		M077R020
			10K12		M077R030
			10H3		M077R040
			10K12		M077R050
SET K12 TO ACTUAL EFFORT.			J111		M077R060
			10K22		M077R070
INPUT 'EFFORT'			10T7		M077R080
PRINT LINE.			9-100		M077R090
			10K10		M077R100
			10K20		M077R110
INPUT 'SUBPROBLEMS'			10T8		M077R120
PRINT LINE.			9-100		M077R130
			10K11		M077R140
			10K21		M077R150
INPUT 'SUBSTITUTIONS'			10T9	9-100	M077R160
9-100 SUBPROCESS, PRINT LINE.	9-100		J154		M077R170
ENTER MESSAGE.			M76		M077R180
			10K44		M077R190
TAB TO COLUMN K44.			J160		M077R200
			10T21		M077R210
ENTER 'LIMIT'			M76		M077R220
			10N1		M077R230
BUMP COLUMN.			J161		M077R240
ENTER LIMIT.			J157		M077R250
			10K45		M077R260
TAB TO COLUMN K45.			J160		M077R270
			10T20		M077R280
ENTER 'ACTUAL'			M76		M077R290
			10N1		M077R300
BUMP COLUMN.			J161		M077R310
ENTER ACTUAL AND PRINT LINE.			J157	J155	M077R320
					R
M78 PRINT 'TO PROVE' PROBLEM (0).	1	M78	J154	(0)=PROB	M078R000
INPUT TEXT.			10T22		M078R010
ENTER MESSAGE.			M76		M078R020
			40W22		M078R030
			10N3		M078R040
SET UP TO PRINT ON NEW PAGE.			20W22		M078R050
			J155		M078R060
RESTORE SPACING AND PRINT (0).			30W22	M70	M078R070
					R
M79 ENTER NAME OF (0).	1	M79	40H0	PSV (0)	M079R000
FIND EXTERNAL NAME.			Q7		M079R010
IF THERE,			709-1		M079R020
ENTER IT, DISCARD (0).			J157	J8	M079R030
IF NOT THERE,	9-1		40H0	PSV (0)	M079R040
FIND SUBPROBLEM NUMBER.			Q8		M079R050

IF NOT THERE, ENTER INTERNAL.		70J156		M079R060
IF THERE, ENTER NO.		J157		M079R070
AND ENTER PERIOD.		50K53	J157	M079R080
	1			R
M80 PRINT PROOF LINE.	M80	J154		M080R000
INPUT (0) IS METHOD OR 'GIVEN'		10K41		M080R010
INPUT (1) IS TEX		J160		M080R020
ENTER METHOD		M76		M080R030
		10K42		M080R040
		J160		M080R050
		40H0		M080R060
ENTER NAME		M79		M080R070
		10K43		M080R080
		J160		M080R090
ENTER EXPRESSION AND PRINT.		M74	J155	M080R100
	1			R
M81 PRINT REJECTED PROBLEM (0).	M81	J154		M081R000
		10K47		M081R010
TAB TO COLUMN K47.		J160		M081R020
		40H0		M081R030
ENTER NAME.		M79		M081R040
		10N3		M081R050
BUMP COLUMN.		J161		M081R060
		40H0		M081R070
ENTER TEX.		M74		M081R080
		11W25		M081R090
TEST IF TEX TOO LONG.		10K48		M081R100
IF YES, SKIP RESET.		J116		M081R110
IF NO,		70	9-2	M081R120
TAB TO COLUMN K48		10K48		M081R130
		J160		M081R140
	9-2	10N2		M081R150
BUMP COLUMN.		J161		M081R160
		40H0		M081R170
FIND THEOREM.		Q12		M081R180
IF NONE, SKIP IT.		709-1		M081R190
IF THERE, ENTER NAME.		M79		M081R200
		10K54		M081R210
ENTER COMMA, AND		J157		M081R220
		10N1		M081R230
BUMP COLUMN.		J161		M081R240
FIND METHOD.	9-1	Q11		M081R250
IF NONE, SKIP IT.		709-3		M081R260
IF THERE, FIND EXTERNAL NAME.		Q16		M081R270
IF NONE, SKIP IT.		709-3		M081R280
IF THERE, ENTER TEXT,		M76		M081R290
		10K53		M081R300
ENTER PERIOD, AND		J157		M081R310
		10N2		M081R320
BUMP COLUMN.		J161		M081R330
	9-3	10T19		M081R340

ENTER MESSAGE AND PRINT.		M76	J155	M081R350
	1			R
M82 PRINT 'REMEMBER PROVED THEOREM'	M82	J154		M082R000
		J155		M082R010
		J155		M082R020
		10T23		M082R030
ENTER MESSAGE.		M76		M082R040
		J155		M082R050
		J154	J155	M082R060
	1			R
M88 PRINT BAD LIST FORM EXPRESSION.	M88	J154		M088R000
		10T24		M088R010
ENTER 'BAD EXPRESSION'		M76		M088R020
		10N3		M088R030
BUMP COLUMN.		J161		M088R040
		10M79		M088R050
GEN SYMBOLS FOR ENTRY, PRINT.		J100	J155	M088R060
	1			R
M89 READ NEXT LOGIC EXPRESSION	M89	40W0		M089R000
FROM NORMAL INPUT UNIT.		40W25		M089R010
H5- MEANS NONE THERE.		40W30		M089R020
CLEAR AND	9-10	J154		M089R050
FILL BUFFER.		J180		M089R060
IF EOF, QUIT, H5-.		709-0		M089R070
		10N1		M089R080
		J120		M089R090
LOCATE 1ST OF NAME.		J184		M089R110
IF BLANK CARD, QUIT, H5-.		709-1		M089R120
		20W25	1W25=1ST	M089R125
		J90		M089R130
		J124		M089R140
DETERMINE EXTENT.		J183		M089R150
IF REST OF CARD, RESET, GET NEXT		709-2		M089R160
		20W30	1W30=EXTNT	M089R170
INPUT NAME.		J181		M089R180
		40H0		M089R190
TEST IF REGIONAL.		J130		M089R200
IF NOT, RESET, GET NEXT.		709-4		M089R210
		40H0		M089R213
TEST IF NAME IS A CHARACTER SYMBOL.		P18		M089R215
IF YES, RESET, GET NEXT.		709-4		M089R217
		20W0	1W0=EXPR.	M089R220
IF OK, GET EXTERNAL NAME.		11W30		M089R230
		11W25		M089R240
		11W25		M089R250
RESET COLUMN TO 1ST OF NAME.		J111		M089R260
		50K51		M089R265
CREATE PROTOTYPE DATA TERM.		J120		M089R270
MARK LOCAL.		J136		M089R280
SET D.T. TO HOLD NAME.		J182		M089R290
		11W0		M089R300

ASSIGN D.T. AS EXTERNAL NAME.		J6		M089R310
		10Q7		M089R320
		J11		M089R330
		11W25		M089R340
LOCATE 1ST OF EXPRESSION.		J184		M089R350
IF NONE, RESET, GET NEXT.		709-5		M089R360
FIND CHARACTER SYMBOL AT 1W25.	9-3	J186		M089R370
IF NONE, EXPRESSION FINI.		709-6		M089R380
IF FOUND, GET ALTERNATE,		P19		M089R390
		11W0		M089R400
		J6		M089R410
		J65		M089R420
ADD AT END OF LIST EXPRESSION,		J125	9-3	M089R430
TALLY 1W25 AND LOOP FOR NEXT.		J184		M089R440
LOCATE FIRST OF SUFFIX.	9-6	709-7		M089R480
IF NONE, SKIP IT.		51W30		M089R490
		J124		M089R495
		J183		M089R500
DETERMINE EXTENT.		709-7		M089R510
IF NONE, SKIP IT.		50K51		M089R520
		J120		M089R530
CREATE PROTOTYPE.		J182		M089R540
SET D. T. TO SUFFIX.		11W0		M089R550
		J6		M089R560
		10Q18		M089R570
ASSIGN AS SUFFIX OF 1W0.		J11		M089R580
		11W0	9-8	M089R590
ADJUST FOR EXTRA IN H0.	9-7	51W0	9-8	M089R600
SET H5+	9-8	J4		M089R610
		11W25		M089R620
DISCARD COLUMN D.T.		J9		M089R630
		11W30		M089R640
DISCARD EXTENT D.T.	9-1	J9		M089R650
	9-2	30W25		M089R660
		30W30	J30	M089R670
RESET	9-5	11W0		M089R700
AFTER		J15		M089R710
A	9-4	J8		M089R720
BAD		11W30		M089R730
EXPRESSION	9-2	J9		M089R740
AND		11W25		M089R750
GET NEXT CARD.		J9	9-10	M089R760
				R
M90 TEST IF PROBLEM LIMITS REACHED.	1 M90	10K20		M090R000
		10K10		M090R010
TEST NO. OF SUBPROBLEMS SET UP		J116		M090R020
		70J5		M090R030
		10K21		M090R040
		10K11		M090R050
TEST NO. OF SUBSTITUTIONS		J116		M090R060
		70J5		M090R070
		10K12		M090R080

		10H3		M090R090
		J90		M090R100
COMPUTE EFFORT		J111		M090R110
		40H0		M090R120
		10K22		M090R130
		J6		M090R140
TEST EFFORT.		J116		M090R150
		J9	J5	M090R160
				R
M110 MAKE FREE VARIABLES OF TOTAL	1	M110	J47	M110R000
EXPRESSIONS (0) AND (1)			J21	M110R010
DISJOINT. (SUBSTITUTES IN (1).)		11W1		M110R020
CREATE FREE VAR. LIST FOR (1)		M116		M110R030
		70J37		M110R040
		20W3		M110R050
		11W0		M110R060
CREATE FREE VAR. LIST FOR (0)		M116		M110R070
		709-1		M110R080
		20W2		M110R090
CREATE EMPTY SUBSTITUTION LIST.		J90		M110R100
		20W4		M110R110
CREATE EMPTY LIST OF MARKED		J90		M110R114
PROCESSED		20W7		M110R116
		10L2		M110R120
SET LOCATION ON SYSTEM FREE		20W5		M110R130
VARIABLE LIST		11W2		M110R140
		109-100		M110R150
MARK ALL OF (0)-S FREE VARS.		J100		M110R160
		11W3		M110R170
		109-200		M110R180
ADD DUPLICATES TO SUBSTITUTION LIST		J100		M110R190
		11W4		M110R200
TEST IF ANY DUPLICATES.		J78		M110R210
		70	9-6	M110R220
UNMARK ALL MARKED VARIABLES		9-400	9-2	M110R225
	9-6	11W4		M110R230
LOCATE NEXT DUPLICATE	9-5	J60		M110R240
		20W6		M110R250
		709-3		M110R260
	9-4	11W5		M110R270
LOCATE NEXT SYSTEM FREE VAR.		J60		M110R280
		20W5		M110R290
HALT DUE TO NOT ENOUGH FREE VAR.		70J7		M110R300
		12W5		M110R310
TEST IF USED IN EITHER.		J133		M110R320
		70	9-4	M110R330
		11W6		M110R340
		12W5		M110R350
INSERT AS SUBSTITUTOR.		J64		M110R360
		11W6		M110R370
		J60	9-5	M110R380

NUMARK ALL MARKED VARIABLES.

GET MAIN SEGMENT OF (1).

SUBSTITUTE.

MARK PROCESSED

ADD TO LIST 1W7 FOR UNMARKING
ADD THOSE MARKED TO SUBST. LIST.
MARK THOSE NOT MARKED.

UNMARK PROCESSED.

UNMARK PROCESSED ALL VARIABLES
ON LIST 1W7

M111 MATCH SEGMENTS (0) AND (1),
H5+ MEANS OUTPUT (0) IS LIST
OF PAIRS--1ST IS FREE VAR.,
2ND IS SUBSTITUTOR.
H5- MEANS NO MATCH, NO OUTPUT.
9-100 MATCH SUBPROCESS
(EXPECTS FREE VARIABLES DISJOIN
IS (0) A VARIABLE.

IS (0) A FREE VARIABLE.

IS THERE ALREADY A SUBSTITUTOR
FOR (0).
SET SUBSTITUTOR AS (0) AND MATCH.

TEST IF (1) IS A VARIABLE

9-3 9-400

11W4

11W1

J81

70J7

M115

9-2 11W4

J71

11W2

J71

9-1 11W3

J71 J37

9-100 J137

11W7

J6

J64 J4

9-200 40HC

J133

709-100

11W4

J6 J64

9-300 31HC

30HC

J4

9-400 11W7

109-300

J100 0

1

M111 J90

209-10

9-100

119-10

70J71 0

9-100 04J51

9-104 11W0

P8

709-101

11W0

P9

709-102

109-10

11W0

J10

709-103

20W0 9-104

9-103 109-10

11W1

40HC

P8

709-105

40HC

11W0

M110R390

M110R420

M110R430

M110R440

M110R450

M110R460

M110R470

M110R480

M110R490

M110R500

M110R510

M110R520

M110R525

M110R530

M110R535

M110R540

M110R550

M110R560

M110R570

M110R580

M110R590

M110R600

M110R610

M110R620

M110R630

M110R640

R

M111R000

M111R010

M111R020

M111R030

M111R040

M111R050

M111R060

M111R070

M111R080

M111R090

M111R100

M111R110

M111R120

M111R130

M111R140

M111R150

M111R160

M111R170

M111R180

M111R185

M111R190

M111R195

M111R200

M111R205

(1) IS VARIABLE, TEST (1)=(0)
 NO SUBSTITUTOR, QUIT W/H5+
 IF NOT, MAKE EXPRESSION INTERNAL
 ASSIGN (1) AS SUBSTITUTOR FOR (0)

((0) IS VARIABLE, NOT FREE.)
 IS (1) A VARIABLE.

IS (1) A FREE VARIABLE.

IS THERE ALREADY A SUBSTITUTOR
 FOR (1).
 SET SUBSTITUTOR AS (1) AND MATCH.

TEST IF (0) IS A VARIABLE

(0) IS VARIABLE, TEST (0)=(1)
 IF NOT, MAKE EXPRESSION INTERNAL
 ASSIGN (0) AS SUBSTITUTOR FOR (1)
 (BOTH ARE VARIABLES, NOT FREE)

ARE VARIABLES IDENTICAL.
 ((0) IS EXPRESSION.)
 IS (1) A VARIABLE

IS (1) A FREE VARIABLE.

(BOTH ARE EXPRESSIONS.)

ARE CONNECTIVES IDENTICAL

LOCATE NEXT SUBSEGMENT ON (0)

LOCATE NEXT SUBSEGMENT ON (1)

J2
 709-114
 9-116 3CH0
 30H0 J31
 9-105 J138
 9-114 11W0
 9-109 J11
 J31 J4
 9-102 11W1
 P8
 70J31
 11W1
 P9
 709-106
 9-111 109-10
 11W1
 J10
 709-107
 20W1 9-104
 9-107 109-10
 11W0
 4CH0
 P8
 709-108
 40H0
 11W1
 J2
 709-115 9-116
 9-108 J138
 9-115 11W1 9-109
 9-106 11W0
 11W1
 J2 J31
 9-101 11W1
 P8
 709-110
 11W1
 P9
 70J31 9-111
 9-110 12W0
 12W1
 J2
 70J31
 9-113 11W0
 J60
 20W0
 709-112
 11W1
 J60
 20W1
 70J31

M111R210
 M111R215
 M111R220
 M111R225
 M111R230
 M111R235
 M111R240
 M111R250
 M111R260
 M111R270
 M111R280
 M111R290
 M111R300
 M111R310
 M111R320
 M111R330
 M111R340
 M111R350
 M111R360
 M111R370
 M111R380
 M111R390
 M111R400
 M111R405
 M111R410
 M111R415
 M111R420
 M111R425
 M111R430
 M111R435
 M111R440
 M111R450
 M111R460
 M111R470
 M111R480
 M111R490
 M111R500
 M111R510
 M111R520
 M111R530
 M111R540
 M111R550
 M111R560
 M111R570
 M111R580
 M111R590
 M111R600
 M111R610
 M111R620
 M111R630
 M111R640

		12W0		M111R650
		12W1		M111R660
MATCH SUBSEGMENTS		9-100		M111R670
		70J31	9-113	M111R680
	9-112	11W1		M111R690
LOCATE NEXT SUBSEGMENT ON (1)		J60		M111R700
		20W1		M111R710
		J5	J31	M111R720
				R
M112 EXPAND SUBSTITUTION LIST (0).	1	M112 40H0		M112R000
REPLACE EXPRESSIONS WITH		J51		M112R010
COMPLETELY SUBSTITUTED LOCALLY	9-1	11W0		M112R020
NAMED COPIES.		J60		M112R030
LOCATE NEXT SUBSTATION		J60		M112R040
		20W0		M112R050
		70J31		M112R060
		12W0		M112R070
DELINEATE SEGMENT AT THIS STATION		9-100		M112R080
SEGMENT DELINEATION SUBPROCESS.	9-100	21W0	9-1	M112R090
IS INPUT A VARIABLE.		4J50		M112R100
		11W0		M112R110
		P8		M112R120
		11W0		M112R130
IS INPUT A FREE VARIABLE.		709-101		M112R140
		P9		M112R150
		709-102		M112R160
DELINEATE FREE VARIABLE.		11W0		M112R170
		9-200	J30	M112R180
CREATE SUBSTITUTED LOCAL COPY	9-102	11W0	J30	M112R190
	9-101	J74		M112R200
		J136		M112R210
		60W0		M112R220
GENERATE FREE VARIABLE LOCATIONS.		109-300		M112R230
		P28	9-102	M112R240
	9-300	J50		M112R250
DELINEATE FREE VARIABLE		12W0		M112R260
		9-200		M112R270
		21W0		M112R280
		J30	J4	M112R290
	9-200	4J50		M112R300
		10W1		M112R310
		11W0		M112R320
FIND CORRESPONDING SUBSTITUTOR		J10		M112R330
DELINEATE SUBSTITUTOR		709-201		M112R340
OUTPUT FREE VARIABLE		9-100	J30	M112R350
	9-201	11W0	J30	M112R360
				R
MATCH SEGMENTS (0) AND (1) FOR	1	M113 M111		M113R000
SUBSTITUTION. SETS H5, IF + OUTPUT		700		M113R010
(0) IS EXPANDED SUBSTITUTION LIST.		40H0		M113R020
		M112	J4	M113R030

TEST IF SEGMENT (0) MATCHES SEGMENT (1).	1	M114	M111		R
			700	J71	M114R000
					M114R010
M115 SUBSTITUTE IN SEGMENT (0) FROM SUBSTITUTION LIST (1). SUBSTITUTES ONLY FOR VARIABLES. SAMPLE SUBST. LIST. IF EMPTY, QUIT.	1	M115	J42		R
			20W0		M115R000
			60W1		M115R003
			J81		M115R007
			70J32		M115R010
			40HC		M115R013
			P9		M115R017
TEST IF FREE VARIABLE.			709-1		M115R020
IF YES,			51W0		M115R023
GENERATE LOCATIONS OF FREE VARIABLES.			109-100		M115R027
TEST IF BOUND VARIABLE.			P28	J32	M115R030
IF YES,	9-1		P8		M115R033
GENERATE LOCATIONS OF BOUND VARIABLES.			70J32		M115R037
			11W0		M115R040
			109-100		M115R043
			P29	J32	M115R047
9-100 SUBPROCESS, SUBST. IN LOC(0).	9-100		60W2		M115R050
			50W1		M115R055
			12W2		M115R060
FIND SUBSTITUTOR, IF ANY			J10		M115R070
			70J4		M115R080
			40HC		M115R090
TEST IF A VARIABLE			P8		M115R100
IF AN EXPRESSION, COPY IT.			70	9-101	M115R110
			J74		M115R120
			J136		M115R130
STORE IN LOCATION	9-101	21W2	J4		M115R140
					M115R150
CREATE LIST OF FREE VARIABLES IN TEX (0). SETS H5, NO OUTPUT IF -. GET MAIN SEGMENT.	1	M116	J90		R
			J50		M116R000
			J81		M116R010
			709-1		M116R020
			109-100		M116R030
GENERATE LOCATIONS OF FREE VARS.			P28		M116R040
	9-1		11W0		M116R060
			40HC		M116R070
			30W0		M116R080
TEST IF ANY FREE VARS.			J78		M116R090
			70J71	0	M116R100
	9-100	52HC			M116R110
		11W0			M116R120
		J6			M116R130
ADD TO OUTPUT IF NOT ALREADY ON.			J66	J4	M116R140
					M116R150
M117 FIND LIST OF BOUND VARIABLES IN TEX (0). H5- MEANS NO OUTPUT.	1	M117	P31		R
			40HC		M117R000
			J78		M117R010
					M117R020

		70J9	0	M117R030
				R
P2 TEST IF (0) IS A BOUND VARIABLE.	1	P2	Q9	P002R000
			700	P002R010
			J8	R
CLEAR DESCRIPTIONS OF TOTAL	1	P3	40H0	P003R000
EXPRESSION (0)			10Q2	P003R010
			J14	P003R020
			40H0	P003R030
			10Q3	P003R040
			J14	P003R050
			10Q4	P003R060
			J14	R
P4 GO THRU NOTS OF SEGMENT (0),	1	P4	12H0	P004R000
LEAVE 1ST UNNOTTED SEGMENT.			11K2	P004R010
H5- MEANS NO OUTPUT.			J2	P004R020
QUIT, H5+ MEANS NORMAL EXIT.			70J4	P004R030
FIND SUBSEGMENT OF NOT.			J81	P004R040
IF NONE QUIT -, ELSE LOOP.			700	P004R050
			P4	R
P5 TEST IF MAIN CONNECTIVE	1	P5	P16	P005R000
OF TOTAL EXPRESSION (0)			700	P005R010
IS IMPLIES.			11K6	P005R020
			J2	R
P6 TEST IF (0) IS NOT UNARY.	1	P6	Q14	P006R000
			700	P006R010
			J1	R
P7-TEST IF (0) IS CONNECTIVE	1	P7	Q14	P007R000
			700	P007R010
			J8	R
TEST IF (0) IS VARIABLE	1	P8	Q5	P008R000
			700	P008R010
			J8	R
TEST IF (1) IS FREE VARIABLE	1	P9	Q6	P009R000
			700	P009R010
			J8	R
P12 FIND MEX OF TEX (0).	1	P12	J81	P012R000
			0	R
P13 FIND LEFT SIDE OF TEX(0).	1	P13	J81	P013R000
			700	P013R030
			J81	R
P14 FIND RIGHT SIDE OF TEX(0).	1	P14	J81	P014R000
			700	P014R030
			J82	R
P15 TEST IF (0) IS IN	1	P15	Q15	P015R000
INTERNAL (TREE) FORM.			700	P015R010
			J8	R
P16 FIND MAIN CONNECTIVE OF	1	P16	J81	P016R000
TEX (0).			700	P016R010
GO THRU NOTS.			P4	P016R020
			700	P016R030
			Q1	

P17 CREATE COPY OF SEGMENT (0) IF NOT A SIMPLE VARIABLE.	1	P17	40H0 P8 70 J74	0 J136	R P017R000 P017R010 P017R020 P017R030
COPY AND MARK LOCAL.					R
P18 TEST IF (0) IS A CHARACTER SYMBOL.	1	P18	P19 Q7 J5 70J8	0	P018R000 P018R010 P018R020 P018R030
GET APPROPRIATE INTERNAL CHARACTER SYMBOL (0) FOR EXTERNAL CHARACTER SYMBOL (0). IF REPLACED, QUIT +.	1	P19	40H0 Q19 70C J6	J8	R P019R000 P019R010 P019R020 P019R030
P20 MAKE FAKE TEX WITH LEFT SIDE OF TEX (0).	1	P20	P13 70C	P24	R P020R000 P020R010
P21 MAKE FAKE TEX WITH RIGHT SIDE OF TEX (0).	1	P21	P14 70C	P24	R P021R000 P021R010
P22 CREATE NEW SUBPROBLEM WITH SEGMENT (0) ON THE LEFT, SEGMENT (1) ON THE RIGHT, AND IMPLIES AS CONNECTIVE. CREATE MEX.	1	P22	40W0 P17 J6 P17 J92 J136 60W0 11K6 21W0 P24	J30	R P022R000 P022R010 P022R020 P022R030 P022R040 P022R050 P022R060 P022R070 P022R080 P022R090
INSERT CONNECTIVE.					R
CREATE TEX, CLEAN UP, QUIT.	1	P23	40H0 J60 J68	J72	P023R000 P023R010 P023R020
ERASE MADE EXPRESSION (0)					R
DELETE MAIN SEGMENT	1	P24	J91 40H0 10Q15 10Q15	J11	P024R000 P024R010 P024R020 P024R030
P24 MAKE TEX FROM MEX (0).					R
DESCRIBE AS IN TREE FORM.	1	P25	J74 40H0 10Q7 J14 40H0	P3	P025R000 P025R010 P025R020 P025R030 P025R040
P25 COPY TEX (0) FOR SUBSTITUTION.					R
CLEAR EXTERNAL NAME AND CLEAR DESCRIPTIONS.	1	P26	10W1 J17 J81		P026R000 P026R010 P026R020

TEST IF MEX IS A VARIABLE.	709-1		PC26R03C
IF YES, QUIT.	40HC		P026R060
IF NO, REVERSE,	P8		P026R070
	70	9-2	P026R080
	J6		P026R090
	10N1		P026R100
CREATE COUNTER WITH VALUE 1, AND	J120		P026R110
SAVE BOTH LEVEL AND COUNTER.	J21	1W1=LEVEL	P026R120
	109-100	1W0=COUNT	P026R130
GENERATE SUBSEGMENT	9-200		P026R140
LOCATIONS FOR 9-100.	11W0		P026R150
ERASE COUNTER AND QUIT.	J9	J19	P026R160
POP H0,	9-2	30HC	P026R170
POP H0, AND QUIT.	9-1	30HC	P026R180
SUBPROCESS 9-100.	9-100	11W0	P026R190
		11W1	P026R200
TEST IF THIS IS THE LEVEL.	J114		P026R210
IF YES, FIRE J18.	70	J18	P026R220
	52HC		P026R225
IF NO, GENERATE	109-100		P026R230
SUBSEGMENTS FOR 9-100.	9-200	0	P026R240
SUBGENERATOR 9-200.	9-200	14W0	P026R250
		J17	P026R260
		11W0	P026R270
CREATE NEW SUBLEVEL COUNTER.	J120		P026R280
BUMP COUNTER.	J125		P026R290
	20W0		P026R300
	40W0		P026R310
LOCATE NEXT SEGMENT PLACE.	9-201	J60	P026R320
IF NONE, QUIT.		709-203	P026R330
IF FOUND, PRESERVE LOCATION,		40HC	P026R340
		12HC	P026R350
TEST IF SEGMENT IS A VARIABLE.		P8	P026R360
IF NO, FIRE J18.		709-202	P026R370
IF YES, LOOP TO LOCATE.		30HC	P026R380
	9-202	J18	P026R390
IF J18 QUIT+, LOOP TO LOCATE,		70	P026R400
IF J18 QUIT-,	9-203	51W0	P026R410
		30W0	P026R420
ERASE COUNTER AND QUIT.		J9	P026R430
			R
P27 REPLACE BOUND BY FREE IN (0).	1	P27	P027R000
		J43	P027R010
CREATE LIST OF BOUND OF 1W0.		60W0	P027R020
		P31	P027R030
LOCATE FIRST BOUND.		60W1	P027R040
		J60	P027R050
IF NO BOUND, QUIT.		20W2	P027R060
		709-1	P027R070
CREATE LIST OF FREE OF 1W0.		11W0	P027R080
		P30	P027R090
		60W3	1W3=FREE LSP027R090

GEN FREE VAR TO BE MARKED.	10J137		P027R100
	J100		P027R110
	10L2	L2=SYSFREE	P027R120
	109-100		P027R130
GEN SYS FREE VAR. TO REPLACE.	J100		P027R140
H5+ MEANS NOT ENOUGH FREE VAR.	70	J7	P027R150
	11W1	1W1=SUBLST	P027R160
	11W0		P027R170
FIND MAIN SEGMENT.	J81		P027R175
	70	9-2	P027R180
IF NONE, SKIP IT.	J8	9-3	P027R185
REPLACE IN MEX FROM 1W1.	9-2 M115		P027R190
	9-3 11W3		P027R195
	109-200		P027R200
GEN FREE VAR TO BE UNMARKED.	J100		P027R210
	11W3		P027R220
ERASE CREATED FREE LIST.	J71		P027R230
	9-1 11W1		P027R240
ERASE BOUND LIST.	J71	J33	P027R250
9-100 SUBPROCESS, INSERT (0)	9-100 40H0		P027R260
AFTER SYMBOL IN 1W2	J133		P027R270
IF (0) IS UNMARKED.	70	J8	P027R280
THEN ADVANCE TO NEXT	11W2		P027R290
AFTER CELL HOLDING	J6		P027R300
INSERTED SYMBOL.	J64		P027R310
	11W2		P027R320
	J60		P027R330
	J60		P027R340
QUIT, H5- MEANS QUIT GENERATOR.	20W2	0	P027R350
9-200 SUBPROCESS, UNMARK PROCESSED.	9-200 31H0	J8	P027R360
	1		R
GENERATE LOCATIONS OF FREE	P28 10W0		P028R000
VARIABLES WITHIN SEGMENT (1)	J17		P028R010
FOR PROCESS (0).	60W0		P028R020
	P8		P028R030
TEST IF INPUT SEGMENT IS VARIABLE.	70	J19	P028R040
IF SO, QUIT.	9-2 11W0		P028R050
LOCATE NEXT SUBSEGMENT.	J60		P028R060
	20W0		P028R070
	70J19		P028R080
	12W0		P028R090
TEST IF SUBSEGMENT IS FREE VAR.	P9		P028R100
	709-1		P028R110
	11W0		P028R120
IF SO, GENERATE LOCATION.	J18		P028R130
	9-3 70J19	9-2	P028R140
	9-1 12W0		P028R150
TEST IF SUBSEGMENT IS VARIABLE.	P8		P028R160
	70	9-2	P028R170
IF NOT, GENERATE SUBSEGMENT.	12W0		P028R180
	10J18		P028R190

		P28	9-3	P028R200
	1			R
P29 GENERATE LOCATIONS OF BOUND	P29	10W0		P029R000
VARIABLES WITHIN SEGMENT (1)		J17		P029R010
FOR PROCESS (0).		60W0		P029R020
TEST IF INPUT SEGMENT IS VARIABLE.		P8		P029R030
IF YES, QUIT H5+.		70	J19	P029R040
IF NO,	9-2	11W0		P029R050
LOCATE NEXT SEGMENT		J60		P029R060
		20W0		P029R070
IF NONE, QUIT, H5+.		70J19		P029R080
		12W0		P029R090
TEST IF FREE VARIABLE.		P9		P029R100
IF YES, GET NEXT.		70	9-2	P029R110
IF NO,		12W0		P029R120
TEST IF BOUND VARIABLE.		P8		P029R130
IF NO, GENERATE ON SEGMENT.		709-1		P029R140
IF YES, FEED LOCATION		11W0		P029R150
TO PROCESS.		J18		P029R160
IF H5-, SUBPROCESS SAID QUIT.	9-3	70J19	9-2	P029R170
INPUT SEGMENT.	9-1	12W0		P029R180
INPUT PROCESS.		10J18		P029R190
GENERATE LOCATIONS OF BND. VAR.		P29	9-3	P029R200
	1			R
P30 CREATE LIST OF FREE VARIABLES	P30	J90		P030R000
IN TEX (0).		J50	1W0=LIST	P030R010
FIND MAIN SEGMENT		J81		P030R020
IF NONE, CLEAN UP, QUIT.		709-1		P030R030
		109-100		P030R040
GEN. LOCATIONS OF FREE VARIABLES.		P28		P030R050
	9-1	11W0	J30	P030R060
SUBPROCESS, ADD FREE VARIABLE	9-100	52H0		P030R070
ZH0 TO LIST 1W0 IF NOT ON.		11W0		P030R080
		J6		P030R090
QUIT, H5+ FOR GEN.		J66	J4	P030R100
	1			R
P31 CREATE LIST OF BOUND VARIABLES.	P31	J90		P031R000
IN TEX (0).		J50	1W0=LIST	P031R010
FIND MAIN SEGMENT.		J81		P031R020
IF NONE, CLEAN UP, QUIT.		709-1		P031R030
		109-100		P031R040
GENERATE LOCATIONS OF BOUND VAR.		P29		P031R050
	9-1	11W0	J30	P031R060
SUBPROCESS, ADD BOUND VARIABLE	9-100	52H0		P031R070
ZH0 TO LIST 1W0 IF NOT ON.		11W0		P031R080
		J6		P031R090
QUIT, H5+ FOR GEN.		J66	J4	P031R100
	1			R
P50 CONVERT LOGIC EXPRESSION (0) TO	P50	40H0		P050R000
INTERNAL (TREE) FORM IF IN		P15		P050R010
EXTERNAL (LIST) FORM. ENTIRE		70	J8	P050R020

EXPRESSION MUST BE ENCLOSED IN PARENTHESES. NO OUTPUT. H5- MEANS FAILURE.		J41		P050R030
		60W0		P050R040
		P51		P050R050
		11W0		P050R060
CREATE NEW MAIN SEGMENT.		P52		P050R070
IF FAILED, QUIT.		70J31		P050R075
SAVE NEW MEX.		60W1		P050R080
		51W0		P050R090
SAVE OLD HEAD,		J75		P050R100
DISCARD OLD LIST.		J71		P050R110
		11W0		P050R120
		11W1		P050R130
INSERT MEX UNDER OLD HEAD.		J64		P050R140
		11W0		P050R190
		10Q15		P050R200
		10Q15		P050R210
DESCRIBE AS IN INTERNAL FORM.		J11	J31	P050R220
	1			R
P51 REPLACE ALL DELIMITED EXTERNAL CONNECTIVES IN EXPRESSION (0) IF (0) IS IN EXTERNAL LIST FORM.	P51	40H0		P051R000
		P15		P051R005
		70	J8	P051R010
		J42		P051R015
		60W0		P051R020
	9-10	11K7		P051R025
LOCATE FIRST DELIMITER IN LIST.		J62		P051R030
		709-0		P051R040
		60W1		P051R050
LOCATE CONNECTIVE (EXTERNAL FORM).		J60		P051R060
		60W2		P051R070
LOCATE SECOND DELIMITER IN LIST.		J60		P051R080
IF NOT ALL THERE, QUIT.		709-0		P051R090
		12H0		P051R100
TEST IF 2ND IS SAME AS 1ST.		11K7		P051R110
IF NOT, TRY ON REMAINDER.		J2		P051R120
IF YES, DELETE 2ND,		709-10		P051R130
		J68		P051R140
		10L8		P051R170
		12W2		P051R180
FIND INTERNAL FORM,		J10		P051R190
		709-1		P051R200
REPLACE EXTERNAL,		21W2		P051R210
	9-1	11W1		P051R220
DELETE FIRST DELIMITER.		J68		P051R230
RESET AND DO IT AGAIN.		11W0	9-10	P051R240
ALL DONE, CLEAN UP AND QUIT.	9-0	30H0	J32	P051R250
	1			R
P52 CREATE MAIN SEGMENT FROM LIST (0). H5- MEANS NO OUTPUT DUE TO BAD EXPRESSION.	P52	J41		P052R000
		20W1		P052R020
9-100 SUBPROCESS CREATE NEXT SGMNT.	9-100	04JC	J31	P052R030
H5- MEANS NO OUTPUT.		11W1		P052R040
				P052R050

LOCATE FIRST OF EXPRESSION	J60		P052R060
IF NONE, QUIT.	20W1		P052R070
	709-101		P052R080
	12W1		P052R090
	10(		P052R100
TEST IF OPENING PAREN.	J2		P052R110
IF YES, BUILD SEGMENT.	70	9-110	P052R120
	12W1		P052R130
	11K2	1K2= NOT	P052R140
TEST IF NOT.	J2		P052R150
IF YES, BUILD SEGMENT.	70	9-120	P052R160
	12W1		P052R170
TEST IF VARIABLE.	P8		P052R180
IF NO QUIT.	709-101		P052R190
OUTPUT VARIABLE.	12W1	0	P052R200
	9-101	11W1	P052R205
		10/14 J63	P052R210
TAKE ERROR ACTION, THEN	9-103	9-101	P052R220
ERASE USELESS SEGMENT.	9-102	11W0	P052R230
		J72 J30	P052R240
OUTPUT SEGMENT.	9-104	11W0	P052R250
		J136 J30	P052R260
BUILD SEGMENT	9-110	9-200	1W0=NEWSEGP052R270
CREATE 1ST SUBSEGMENT.		9-100	P052R280
IF NONE, CLEAN UP, QUIT.	709-102		P052R290
INSERT 1ST SUBSEGMENT.		9-300	P052R300
		11W1	P052R310
LOCATE NEXT SYMBOL.		J60	P052R320
		60W1	P052R330
		52W1	P052R340
TEST IF CONNECTIVE.		P7	P052R350
IF NOT, CLEAN UP, QUIT.	709-103		P052R360
		12W1	P052R370
INSERT CONNECTIVE.		21W0	P052R380
CREATE 2ND SUBSEGMENT.		9-100	P052R390
IF NONE, CLEAN UP, QUIT.	709-102		P052R400
INSERT 2ND SUBSEGMENT.		9-300	P052R410
		11W1	P052R420
LOCATE NEXT SYMBOL.		J60	P052R430
		60W1	P052R440
		52W1	P052R450
		10)	P052R460
TEST IF CLOSING PAREN.		J2	P052R470
IF NO, CLEANUP, QUIT.	709-103	9-104	P052R480
BUILD NOTTED SEGMENT.	9-120	9-200	P052R490
		12W1	P052R500
NOT THE HEAD.		21W0	P052R510
CREATE SUBSEGMENT.		9-100	P052R520
IF NONE, CLEAN UP AND QUIT.	709-102		P052R530
INSERT		9-300	P052R540
9-200 SUBPROCESS, SET UP EMPTY SEG.	9-200	J90 J50	P052R550

9-300 SUBPROCESS, INSERT SEGMENT.	9-300	11W0		P052R560
		J6	J65	P052R570
	1			R
P55 LOCATE SUBLIST FOLLOWING	P55	J41		P055R000
DATA TERM (0) ON LIST (1))		20W0		P055R010
H5+ MEANS PUTPUT (0) IS	9-3	60W1		P055R020
CELL HOLDING SUBLIST.		J60		P055R030
H5- MEANS OUTPUT (0) IS		70J31		P055R040
CELL AFTER WHICH TO INSERT.		12H0		P055R050
		11W0		P055R060
TEST IF PAST.		J116		P055R070
IF YES, QUIT, H5-.		70	9-1	P055R080
		12H0		P055R090
		11W0		P055R100
TEST IF EQUAL.		J114		P055R110
IF YES, QUIT, H5+.		70	9-2	P055R120
IF NO, MOVE DOWN THE LIST.		J60		P055R130
		70J7	9-3	P055R140
SET UP CELL TO INSERT AFTER.	9-1	51W1		P055R150
		J3	J31	P055R160
SET UP CELL HOLDING SUBLIST.	9-2	J60		P055R170
		70J7	J31	P055R180
	1			R
Q1 FIND CONNECTIVE OF SEGMENT (0).	Q1	J80		Q001R000
		700		Q001R010
		40H0		Q001R020
TEST IF IT IS A CONNECTIVE.		P7		Q001R030
		70J8	0	Q001R040
	1			R
Q2 FIND NO. OF LEVELS OF TEX (0).	Q2	40H0		Q002R000
H5- MEANS DEFECTIVE EXPRESSION.		10Q2		Q002R010
FIND VALUE ON DESCRIPTION LIST.		J10		Q002R020
IF NONE, GO COUNT LEVELS.		709-0		Q002R030
IF THERE, CLEAN UP, QUIT +.		J6	J8	Q002R040
	9-0	10N10		Q002R050
		J120		Q002R060
CREATE LEVEL DATA TERM = ZERO.		J136		Q002R070
		40H0		Q002R080
CREATE COUNTER.		J120		Q002R090
SAVE COUNTER, LEVEL, TEX.		J52		Q002R100
		11W2		Q002R110
FIND MEX.		J81		Q002R120
IF NONE, CLEANUP, QUIT-.		709-1		Q002R130
IF THERE, COUNT LEVELS.		9-100		Q002R140
	9-1	11W0		Q002R150
ERASE COUNTER.		J9		Q002R160
INPUT LEVEL,		11W1		Q002R170
IF H5-, ERASE LEVEL, QUIT-.		709-2		Q002R180
IF H5+,		11W2		Q002R190
ASSIGN LEVEL 1W1 AS VALUE		11W1		Q002R200
OF Q2 OF TEX 1W2.		10Q2		Q002R210

1W0=COUNTR
1W1=LEVEL
1W2=TEX

QUIT +, OUTPUT (0) IS LEVEL.		J11	J32	Q002R220
	9-2	J9	J32	Q002R230
9-100 SUBPROCESS. COUNT SUBLEVELS.	9-100	11W0		Q002R240
		J120		Q002R250
CREATE COUNTER EQUAL TO THIS LEVEL.		J125		Q002R260
PRESERVE PREVIOUS COUNTER.		J50		Q002R270
		40H0		Q002R300
TEST IF SIMPLE VARIABLE.		P8		Q002R310
IF YES, UPDATE LEVEL.		70	9-102	Q002R320
IF NO, COUNT SUBLEVELS,		109-100		Q002R330
THEN QUIT + OR-.		J100	9-101	Q002R340
UPDATE LEVEL.	9-102	51W1		Q002R350
		11W0		Q002R360
TEST IF COUNTER GREATER THAN LEVEL.		J115		Q002R370
IF NO, QUIT +.		709-103		Q002R380
		11W0		Q002R390
IF YES,		11W1		Q002R400
SET LEVEL SAME AS COUNTER.		J121		Q002R410
		30H0		Q002R420
SET H5+.	9-103	J4		Q002R430
ERASE COUNTER OF THIS LEVEL.	9-101	11W0		Q002R440
RESTORE PREVIOUS COUNTER, QUIT.		30W0	J9	Q002R450
	1			R
Q3 FIND NO. OF DISTINCT VARIABLES	Q3	40H0		Q003R000
IN TOTAL EXPRESSION (0).		10Q3		Q003R010
		J10		Q003R020
FIND AS VALUE IN DESC. LIST.		709-0		Q003R030
IF THERE, CLEAN UP, QUIT +.		J6	J8	Q003R040
IF NONE, COUNT VARIABLES.	9-0	J42		Q003R050
		60W0	1W0=TEX	Q003R060
CREATE FREE LIST,		P30		Q003R070
SAVE IT,		60W1	1W1=FREE	Q003R080
COUNT IT,		J126		Q003R090
MARK COUNT LOCAL,		J136		Q003R100
AND SAVE FOR OUTPUT.		60W2	1W2=OUTPUT	Q003R110
		51W1		Q003R120
ERASE FREE LIST.		J71		Q003R130
		11W0		Q003R140
CREATE BOUND LIST,		P31		Q003R150
SAVE IT,		60W1	1W1=BOUND	Q003R160
COUNT IT,		J126		Q003R170
		40H0		Q003R180
		11W2		Q003R190
		11W2		Q003R200
ADD IT TO OUTPUT DATA TERM.		J110		Q003R210
		51W1		Q003R220
ERASE BOUND LIST.		J71		Q003R230
ERASE BOUND COUNT.		J9		Q003R240
		11W0		Q003R250
		11W2		Q003R260
		10Q3		Q003R270

ASSIGN AS VALUE OF Q3.		J11		Q003R280
		11W2		Q003R290
CLEAN UP AND QUIT.		J32	J4	Q003R300
	1			R
Q4 FIND NO. OF VARIABLE PLACES	Q4	40H0		Q004R000
IN TEX (0).		10Q4		Q004R010
FIND AS VALUE OF DESC. LIST.		J10		Q004R020
IF NONE, GO COUNT PLACES.		709-0		Q004R030
IF THERE, CLEAN UP, QUIT, H5+.		J6	J8	Q004R040
	9-0	40H0		Q004R050
SET UP THREE COPIES OF TEX NAME.		40H0		Q004R060
		J90		Q004R070
		J136		Q004R080
CREATE D.T. WITH VALUE = 0.		J124		Q004R090
		J6		Q004R100
		109-100		Q004R110
GENERATE FREE LOCATIONS FOR TALLY.		P28		Q004R120
		J6		Q004R130
		109-100		Q004R140
GENERATE BOUND LOCATIONS FOR TALLY.		P29		Q004R150
		40W0		Q004R160
SAVE OUTPUT D.T.		60W0	1W0=OUTPUT	Q004R170
		10Q4		Q004R180
ASSIGN AS VALUE OF Q4 OF TEX.		J11		Q004R190
		11W0	J30	Q004R200
DISCARD (0), TALLY (1).	9-100	30H0	J125	Q004R210
	1			R
ATTRIBUTE--VARIABLE	Q5	10Q5	J10	Q005R000
	1			R
ATTRIBUTE--FREE VARIABLE	Q6	10Q6	J10	Q006R000
	1			R
ATTRIBUTE--EXTERNAL NAME	Q7	10Q7	J10	Q007R000
	1			R
FIND PROBLEM NUMBER OF (0)	Q8	10Q8	J10	Q008R000
	1			R
Q9 ATTRIBUTE--BOUND VARIABLE.	Q9	10Q9	J10	Q009R000
	1			R
FIND PROBLEM (0) DERIVED FROM	Q10	10Q10	J10	Q010R000
	1			R
FIND METHOD OF DERIVATION FOR (0)	Q11	10Q11	J10	Q011R000
	1			R
FIND THEOREM USED FOR (0)	Q12	10Q12	J10	Q012R000
	1			R
FIND PROVING THEOREM FOR (0)	Q13	10Q13	J10	Q013R000
	1			R
Q14 FIND TYPE OF CONNECTIVE (0).	Q14	10Q14	J10	Q014R000
	1			R
Q15 ATTRIBUTE -- INTERNAL FORM.	Q15	10Q15	J10	Q015R000
	1			R
Q16 FIND EXTERNAL NAME OF (0)	Q16	10T10		Q016R000
IN TABLE T10.		J6	J10	Q016R010

Q17 FIND LEVEL OF SUBSEGMENT REPLACEMENT IN TEX (0).	1	Q17	40W0		R	Q017R000
			60W0			Q017R010
			10Q17			Q017R020
FIND CURRENT LEVEL.			J10			Q017R030
IF NONE,			70	J30		Q017R040
			11W0			Q017R050
FIND NUMBER OF LEVELS,			Q2			Q017R060
IF NONE, QUIT -.			70J30			Q017R065
COPY,			J120			Q017R070
SAVE ONE FOR OUTPUT,			40HC			Q017R080
			11W0			Q017R090
			J6			Q017R100
AND ASSIGN AS CURRENT LEVEL.			10Q17			Q017R110
			30W0	J11		Q017R120
	1				R	
Q18 FIND SUFFIX OF EXPRESSION (0).	1	Q18	10Q18	J10		Q018R000
	1				R	
Q19 FIND CHARACTER SYMBOL FOR '0).	1	Q19	10L9			Q019R000
			J6	J10		Q019R010

	5	1		D -
DATA HEADER				
FREE VARIABLE A	A		0	A000D000
		C		A000D010
		Q5		A000D020
		Q5		A000D030
		Q6		A000D040
		Q6		A000D050
		Q7		A000D060
			0	A000D070
		+21A		A000D080
FREE VARIABLE B	B		0	B000D000
		0		B000D010
		Q5		B000D020
		Q5		B000D030
		Q6		B000D040
		Q6		B000D050
		Q7		B000D060
			0	B000D070
		+21B		B000D080
FREE VARIABLE C	C		0	C000D000
		0		C000D010
		Q5		C000D020
		Q5		C000D030
		Q6		C000D040
		Q6		C000D050
		Q7		C000D060
			0	C000D070
		+21C		C000D080
FREE VARIABLE D	D		0	D000D000
		0		D000D010
		Q5		D000D020
		Q5		D000D030
		Q6		D000D040
		Q6		D000D050
		Q7		D000D060
			0	D000D070
		+21D		D000D080
FREE VARIABLE E	E		0	E000D000
		0		E000D010
		Q5		E000D020
		Q5		E000D030
		Q6		E000D040
		Q6		E000D050
		Q7		E000D060
			0	E000D070
		+21E		E000D080
FREE VARIABLE F	F		0	F000D000
		0		F000D010
		Q5		F000D020
		Q5		F000D030

		Q6		F000D040
		Q6		F000D050
		Q7		F000D060
			0	F000D070
		+21F		F000D080
FREE VARIABLE G	G		0	G000D000
		0		G000D010
		Q5		G000D020
		Q5		G000D030
		Q6		G000D040
		Q6		G000D050
		Q7		G000D060
			0	G000D070
		+21G		G000D080
H0 LUBRICATION.	H0	*1		H000D000
		*2		H000D010
		*3		H000D020
		*4	0	H000D030
IMPLIES	I		0	I000D000
		0		I000D010
		Q14		I000D020
		J4		I000D030
		Q7		I000D040
			0	I000D050
		+21I		I000D060
K0 SYMBOL FOR CHARACTER K.	K		0	K000D000
		0		K000D010
		Q7		K000D020
			0	K000D030
		+21K		K000D040
HOLDS 'OR'	K1	V0	0	K001D000
HOLDS 'NOT'	K2	-0	0	K002D000
HOLDS 'AND'	K3	*0	0	K003D000
HOLDS 'PROVEN EQUIVALENCE'	K4	=0	0	K004D000
HOLDS 'DEFINITIONAL EQUIVALENCE'	K5	=1	0	K005D000
HOLDS 'IMPLIES'	K6	10	0	K006D000
K7 HOLDS CONNECTIVE DELIMITER.	K7	.	0	K007D000
K10 PREVIOUS PROBLEM NUMBER.	K10	01	0	K010D000
SUBSTITUTION COUNT	K11	+ 1	0	K011D000
EFFORT BASE (AND TOTAL).	K12	+ 1	0	K012D000
LIMIT ON NO. OF SUBPROBLEMS	K20	+ 1	100	K020D000
LIMIT ON NO. OF SUBSTITUTIONS	K21	+ 1	100	K021D000
LIMIT ON EFFORT	K22	+ 1 100	0000	K022D000
	K30	F		K030D000
K31 DON'T PRINT REJECTS IF HOLDS NO	K31	NO		K031D000
K41 VALUE = METHOD COLUMN.	K41	01	10	K041D000
K42 VALUE = NAME COLUMN.	K42	01	40	K042D000
K43 VALUE = EXPRESSION COLUMN.	K43	01	50	K043D000
K44 VALUE = 'LIMIT' COLUMN.	K44	01	20	K044D000
K45 VALUE = 'ACTUAL' COLUMN.	K45	01	40	K045D000
K46 VALUE = 'REJECTED' COLUMN.	K46	01	20	K046D000

K47 VALUE = NAME OF NEW SUBPROBLEM	K47	01	11 COLUMN.	K047D000
K48 VALUE = THM, METHOD COLUMN.	K48	01	43	K048D000
K51 DATA TERM '()'	K51	21(		K051D000
K52 DATA TERM '))'	K52	21)		K052D000
K53 DATA TERM '.,'	K53	21.		K053D000
K54 DATA TERM ',,'	K54	21,		K054D000
L0 SYMBOL FOR CHARACTER L.	L		0	L000D000
		0		L000D010
		Q7		L000D020
			0	L000D030
		+21L		L000D040
L1 TRUE THEOREMS AXIOMS DEFINITIONS	L1	0	0	L001D000
LIST OF FREE VARIABLES	L2	0		L002D000
		A0		L002D010
		B0		L002D020
		C0		L002D030
		D0		L002D040
		E0		L002D050
		F0		L002D060
		G0	0	L002D080
L3 PROBLEM LIST FOR MULTI PROB EXEC	L3	0	0 (M2)	L003D000
TRUE EXPRESSIONS MAPS	L4	0	0	L004D000
LIST DESCRIBED BY L4	L5	L4	0	L005D000
L6 LIST OF METHODS FOR ORIG PROBS	L6	0		L006D000
		M16		L006D010
		M17		L006D020
L7 LIST OF METHODS FOR PROBLEMS.	L7	0		L007D000
DETACHMENT.		M11		L007D010
REPLACEMENT.		M13		L007D010
FORWARD CHAINING.		M14		L007D010
BACKWARD CHAINING.		M15		L007D010
L8 DESCRIPTION LIST TABLE OF	L8		0	L008D000
DELIMITABLE EXTERNL CONNECTIVES		0		L008D010
		=		L008D020
		=1		L008D030
L9 DESCRIPTION LIST TABLE OF	L9		0	L009D000
CHARACTER SYMBOLS FOR		0		L009D010
READING TEXT.		0		L009D020
		/10		L009D025
		1		L009D030
		/1		L009D040
		2		L009D050
		/2		L009D060
		3		L009D070
		/3		L009D080
		4		L009D090
		/4		L009D100
		5		L009D110
		/5		L009D120
		6		L009D130
		/6		L009D140

		7		L009D150
		/7		L009D160
		8		L009D170
		/8		L009D180
		9		L009D190
		/9		L009D200
		H		L009D210
		/11		L009D220
		J		L009D230
		/12		L009D240
		W		L009D250
		/13		L009D260
UNTRIED PROBLEMS LIST	L10	0	0	L010D000
L11 FOUND PROBLEMS LIST.	L11	0	0	L011D000
M0 SYMBOL FOR CHARACTER M.	M		0	M000D000
		0		M000D010
		Q7		M000D020
			0	M000D030
		+21M		M000D040
NO SYMBOL FOR CHARACTER N.	N		0	N000D000
		0		N000D010
		Q7		N000D020
			0	N000D030
		+21N		N000D040
1 INTEGER CONSTANTS.	N1	+01	1	N001D000
	N2	+01	2	N002D000
	N3	+01	3	N003D000
	N4	+01	4	N004D000
	N5	+01	5	N005D000
	N6	+01	6	N006D000
	N7	+01	7	N007D000
	N8	+01	8	N008D000
	N9	+01	9	N009D000
	N10	+01	0	N010D000
0	O		0	O000D000
O0 SYMBOL FOR CHARACTER O.		0		O000D010
		Q7		O000D020
			0	O000D030
		+21O		O000D040
VARIABLE P	P		0	P000D000
		0		P000D010
		Q5		P000D020
		Q5		P000D030
		Q9		P000D033
		Q9		P000D037
		Q7		P000D040
			0	P000D050
		+21P		P000D060
VARIABLE Q	Q		0	Q000D000
		0		Q000D010
		Q5		Q000D020

		Q5		Q000D030
		Q9		Q000D033
		Q9		Q000D037
		Q7		Q000D040
			0	Q000D050
		+21Q		Q000D060
VARIABLE R	R		0	R000D000
		0		R000D010
		Q5		R000D020
		Q5		R000D030
		Q9		R000D033
		Q9		R000D037
		Q7		R000D040
			0	R000D050
		+21R		R000D060
VARIABLE S	S		0	S000D000
		0		S000D010
		Q5		S000D020
		Q5		S000D030
		Q9		S000D033
		Q9		S000D037
		Q7		S000D040
			0	S000D050
		+21S		S000D060
VARIABLE T	T		0	T000D000
		0		T000D010
		Q5		T000D020
		Q5		T000D030
		Q9		T000D033
		Q9		T000D037
		Q7		T000D040
			0	T000D050
		+21T		T000D060
T1 'GIVEN'	T1	0		T001D000
			0	T001D010
		21GIVEN		T001D020
T2 'PROOF FOUND.'	T2	0		T002D000
		9-1		T002D010
		9-2		T002D020
		9-3	0	T002D030
	9-1	21 PROO		T002D040
	9-2	21F FOU		T002D050
	9-3	21ND.		T002D060
T3 'SUBSTITUTION'	T3	0		T003D000
		9-1		T003D010
		9-2		T003D020
		9-3	0	T003D030
	9-1	21SUBST		T003D040
	9-2	21ITUTI		T003D050
	9-3	21ON		T003D060
T4 'Q.E.D.'	T4	0		T004D000

		9-1		T004D010
		9-2	0	T004D020
	9-1	21Q.E.D		T004D030
	9-2	21.		T004D040
T5 LIST OF ONE BLANK D.T.	T5	0		T005D000
			0	T005D010
		21		T005D020
T6 'NO PROOF FOUND'	T6	0		T006D000
		9-1		T006D010
		9-2		T006D020
		9-3	0	T006D030
	9-1	21NO PR		T006D040
	9-2	21OOF F		T006D050
	9-3	21OUND		T006D060
T7 'EFFORT'	T7	0		T007D000
		9-1		T007D010
		9-2	0	T007D020
	9-1	21EFFOR		T007D030
	9-2	21T		T007D040
T8 'SUBPROBLEMS'	T8	0		T008D000
		9-1		T008D010
		9-2		T008D020
		9-3	0	T008D030
	9-1	21SUBPR		T008D040
	9-2	21OBLEM		T008D050
	9-3	21S		T008D060
T9 'SUBSTITUTIONS'	T9	0		T009D000
		9-1		T009D010
		9-2		T009D020
		9-3	0	T009D030
	9-1	21SUBST		T009D040
	9-2	21ITUTI		T009D050
	9-3	21ONS		T009D060
T10 DESC. LIST TABLE OF NAMES.	T10	9-0	0	T010D000
	9-0	0		T010D010
		M11		T010D020
		T12		T010D030
		M12		T010D040
		T3		T010D050
		M13		T010D060
		T13		T010D070
		M14		T010D080
		T14		T010D090
		M15		T010D100
		T15		T010D110
		M16		T010D120
		T16	0	T010D130
T12 'DETACHMENT'	T12	0		T012D000
		9-1		T012D010
		9-2	0	T012D020
	9-1	21DETAC		T012D030

T13 'REPLACEMENT'	9-2	21HMENT	T012D040
	T13	0	T013D000
		9-1	T013D010
		9-2	T013D020
		9-3	T013D030
		0	T013D040
	9-1	21REPLA	T013D050
	9-2	21CEMEN	T013D060
	9-3	21T	T014D000
T14 'FORWARD CHAINING'	T14	0	T014D010
		9-1	T014D020
		9-2	T014D030
		9-3	T014D040
		9-4	T014D050
		0	T014D060
	9-1	21FORWA	T014D070
	9-2	21RD CH	T014D080
	9-3	21AININ	T015D000
	9-4	21G	T015D010
T15 'BACKWARD CHAINING'	T15	0	T015D020
		9-1	T015D030
		9-2	T015D040
		9-3	T015D050
		9-4	T015D060
		0	T015D070
	9-1	21BACKW	T015D080
	9-2	21ARD C	T016D000
	9-3	21HAINI	T016D010
	9-4	21NG	T016D020
T16 'SUBLEVEL REPLACEMENT'	T16	0	T016D030
		9-1	T016D040
		9-2	T016D050
		9-3	T016D060
		9-4	T016D070
		0	T016D080
	9-1	21SUBLE	T019D000
	9-2	21VEL R	T019D010
	9-3	21EPLAC	T019D020
	9-4	21EMENT	T019D030
T19 'REJECTED PROBLEM'	T19	0	T019D040
		9-1	T019D050
		9-2	T019D060
		9-3	T019D070
		9-4	T019D080
		0	T020D000
	9-1	21REJEC	T020D010
	9-2	21TED P	T020D020
	9-3	21ROBLE	T020D030
	9-4	21M	T020D040
T20 'ACTUAL'	T20	0	T021D000
		9-1	T021D010
		9-2	T021D020
		0	T021D030
	9-1	21ACTUA	T021D040
	9-2	21L	T021D050
T21 'LIMIT'	T21	0	T021D060
		0	T021D070

T22 'TO PROVE'	T22	21LIMIT		T021D020
		0		T022D000
		9-1		T022D010
		9-2	0	T022D020
	9-1	21TO PR		T022D030
	9-2	21OVE		T022D040
T23 'REMEMBER PROVED THEOREM'	T23	0		T023D000
		9-1		T023D010
		9-2		T023D020
		9-3		T023D030
		9-4		T023D040
		9-5		T023D050
		9-6	0	T023D060
	9-1	21 RE		T023D070
	9-2	21MEMBE		T023D080
	9-3	21R PRO		T023D090
	9-4	21VED T		T023D100
	9-5	21HEORE		T023D110
	9-6	21M		T023D120
T24 'BAD EXPRESSION'	T24	0		T024D000
		9-1		T024D010
		9-2		T024D020
		9-3	0	T024D030
	9-1	21BAD E		T024D040
	9-2	21XPRES		T024D050
	9-3	21SION		T024D060
U0 SYMBOL FOR CHARACTER U.	U	0	0	U000D000
		Q7		U000D010
			0	U000D020
		+21U		U000D030
			0	U000D040
OR	V	0	0	V000D000
		Q14		V000D010
		J4		V000D020
		Q7		V000D030
			0	V000D040
		+21V		V000D050
			0	V000D060
X0 SYMBOL FOR CHARACTER X.	X	0	0	X000D000
		Q7		X000D010
			0	X000D020
		+21X		X000D030
			0	X000D040
Y0 SYMBOL FOR CHARACTER Y.	Y	0	0	Y000D000
		Q7		Y000D010
			0	Y000D020
		+21Y		Y000D030
			0	Y000D040
Z0 SYMBOL FOR CHARACTER Z.	Z	0	0	Z000D000
		Q7		Z000D010
				Z000D020

			0	Z000D030
		+21Z		Z000D040
NOT	-		0	-000D000
		0		-000D010
		Q14		-000D020
		J3		-000D030
		Q7		-000D040
			0	-000D050
		+21-		-000D060
AND	*		0	*000D000
		0		*000D010
		Q14		*000D020
		J4		*000D030
		Q7		*000D050
			0	*000D050
		+21*		*000D060
PROVEN EQUIVALENCE	=		0	=000D000
		0		=000D010
		Q14		=000D020
		J4		=000D030
		Q7		=000D040
			0	=000D050
		+21=		=000D060
=1 DEFINITIONAL EQUIVALENCE	=1		0	=001D000
		0		=001D010
		Q14		=001D020
		J4		=001D030
		Q7		=001D040
			0	=001D050
		+21.=.		=001D060
+0 SYMBOL FOR PLUS SIGN.	+		0	+000D000
		0		+000D010
		Q7		+000D020
			0	+000D030
		+21+		+000D040
/0 SYMBOL FOR SLASH.	/		0	/000D000
		0		/000D010
		Q7		/000D020
			0	/000D030
		+21/		/000D040
/1 SYMBOL FOR DIGIT 1.	/1		0	/001D000
		0		/001D010
		Q7		/001D020
			0	/001D030
		+211		/001D040
/2 SYMBOL FOR DIGIT 2.	/2		0	/002D000
		0		/002D010
		Q7		/002D020
			0	/002D030
		+212		/002D040
/3 SYMBOL FOR DIGIT 3.	/3		0	/003D000

		0	/003D010
		Q7	/003D020
		0	/003D030
		+213	/003D040
/4 SYMBOL FOR DIGIT 4.	/4	0	/004D000
		Q7	/004D010
		0	/004D020
		+214	/004D030
/5 SYMBOL FOR DIGIT 5.	/5	0	/004D040
		Q7	/005D000
		0	/005D010
		+215	/005D020
/6 SYMBOL FOR DIGIT 6.	/6	0	/005D030
		Q7	/005D040
		0	/006D000
		+216	/006D010
/7 SYMBOL FOR DIGIT 7.	/7	0	/006D020
		Q7	/006D030
		0	/006D040
		+217	/007D000
/8 SYMBOL FOR DIGIT 8.	/8	0	/007D010
		Q7	/007D020
		0	/007D030
		+218	/007D040
/9 SYMBOL FOR DIGIT 9.	/9	0	/008D000
		Q7	/008D010
		0	/008D020
		+219	/008D030
/10 SYMBOL FOR DIGIT 0.	/10	0	/008D040
		Q7	/009D000
		0	/009D010
		+210	/009D020
/11 SYMBOL FOR CHARACTER J.	/11	0	/009D030
		Q7	/009D040
		0	/010D000
		+211	/010D010
/12 SYMBOL FOR CHARACTER W.	/12	0	/010D020
		Q7	/010D030
		0	/010D040
		+212	/011D000
/13 SYMBOL FOR CHARACTER H.	/13	0	/011D010
		Q7	/011D020
		0	/011D030
		+213	/011D040
		0	/012D000
		Q7	/012D010
		0	/012D020
		+214	/012D030
		0	/012D040
		Q7	/013D000
		0	/013D010

		Q7		/013D020
			0	/013D030
		+21W		/013D040
/14 DUMMY CHARACTER SYMBOL	/14		0	/014D000
		0		/014D010
		Q7		/014D020
			0	/014D030
		+21/UGH/		/014D040
/16 DUMMY EXPRESSION --	/16	9-1		/016D000
'DEFINITIONS'.		9-2	0	/016D010
	9-1	0		/016D020
		Q15		/016D030
		Q15		/016D040
		Q7		/016D050
			0	/016D060
EXTERNAL NAME		21		/016D070
CONNECTIVE 'I'.	9-2	10		/016D080
		9-10		/016D090
		9-20	0	/016D100
DUMMY VARIABLE 'DEFIN'.	9-10		0	/016D110
		0		/016D120
		Q5		/016D130
		Q5		/016D140
		Q9		/016D150
		Q9		/016D160
		Q7		/016D170
			0	/016D180
EXTERNAL NAME.		21DEFIN		/016D190
DUMMY VARIABLE 'TIONS'.	9-20		0	/016D200
		0		/016D210
		Q5		/016D220
		Q5		/016D230
		Q9		/016D240
		Q9		/016D250
		Q7		/016D260
			0	/016D270
EXTERNAL NAME.		21TIONS		/016D280
(0 SYMBOL FOR LEFT PAREN.	(		0	(000D000
		0		(000D010
		Q7		(000D020
		K51	0	(000D030
'0 SYMBOL FOR QUOTE MARK.	'		0	'000D000
		0		'000D010
		Q7		'000D020
			0	'000D030
		+21'		'000D040
)0 SYMBOL FOR RIGHT PAREN.	)		0	)000D000
		0		)000D010
		Q7		)000D020
		K52	0	)000D030
,0 SYMBOL FOR COMMA.	,		0	,000D000

		0		,000D010
		Q7		,000D020
.0 SYMBOL FOR PERIOD	.	K54	0	,000D030
			0	,000D000
		0		,000D010
		Q7		,000D020
		K53	0	,000D030
\$0 SYMBOL FOR DOLLAR SIGN.	\$		0	\$000D000
		0		\$000D010
		Q7		\$000D020
			0	\$000D030
		+21\$		\$000D040

EXECUTIVE HEADER	5			R -
	1			R
	X1	11W26		X001R000
		10X23		X001R010
		J73		X001R020
SET UP TRAPS.		J76		X001R030
		50X21		X001R040
		10J147		X001R050
MARK TO TRACE.		J100		X001R060
		10X22		X001R070
		10J148		X001R080
MARK TO PROPAGATE TRACE.		J100		X001R090
GET NEXT TRUE TEX FROM	9-1	M89		X001R100
INPUT UNIT IF ANY LEFT.		709-10		X001R110
		40HC		X001R120
CONVERT TO TREE FORM.		P50		X001R130
		709-2		X001R140
ADD TO SET OF TRUE EXPRESSION.		M50	9-1	X001R150
TAKE ACTION, TRY FOR ANOTHER	9-2	9-100	9-1	X001R160
	9-10	J154		X001R170
SKIP TWO LINES.		J155		X001R180
		J155		X001R190
GET NEXT PROBLEM TEX.	9-11	M89		X001R200
IF NC MORE, GO TRY PROOFS.		70M2		X001R210
		40HC		X001R215
CONVERT TO INTERNAL.		P50		X001R220
		709-12		X001R230
		40HC		X001R240
PRINT EXPRESSION.		M70		X001R250
		10L3		X001R260
		J6		X001R270
ADD TO LIST OF PROBLEMS.		J65	9-11	X001R280
	9-12	9-100	9-11	X001R290
BAD INPUT ACTION.	9-100	40HC		X001R300
		M88		X001R310
		40HC		X001R320
		J15		X001R330
		J75	J72	X001R340
	1			R
	X9	10N2		X009R000
		J166	J165	X009R010
	1			R
X10 INVOKE FULL TRACE.	X10	10N1		X010R000
		J120		X010R010
		40W31		X010R020
		20W31		X010R030
		X19	0	X010R040
	1			R
X11 REVOKE CURRENT TRACE MODE	X11	40H5		X011R000
IF ANOTHER EXISTS.		10W31		X011R010

X14 SAVE FOR RESTART ON INTERRUPT
USING UNIT 3.

SAVE AND SET H5+.
IF H5+, HALT.
IF H5-, CONTINUE.

X19 MONITOR POINT FORCER.
SAVE FOR RESTART

		J78		X011R020
		709-0		X011R030
		11W31		X011R040
		30W31		X011R050
		X19		X011R060
		30H5	J9	X011R070
	9-0	30H5	0	X011R080
1				R
	X13	40H0	J150	X013R000
1				R
	X14	40H5		X014R000
		10N3		X014R010
		J3		X014R020
		J166		X014R020
		70	J7	X014R030
		30H5	0	X014R040
1				R
	X15	10L4	J150	X015R000
1				R
	X19	03J0	0	X019R000
5		X9		

REFERENCES

1. Newell, Allen, ed., Information Processing Language-V Manual, Prentice-Hall, Inc., Englewood Cliffs, New Jersey, 1961.
2. Newell, A., and H. A. Simon, The Logic Theory Machine: A Complex Information Processing System, The RAND Corporation, P-868, July 12, 1956. Published in the IRE Transactions on Information Theory, Vol. IF-2, September 1956.
3. Newell, A., J. C. Shaw, and H. A. Simon, Empirical Explorations of the Logic Theory Machine: A Case Study in Heuristics, The RAND Corporation, P-951, March 14, 1957. Published in the Proceedings of the Western Joint Computer Conference, IRE, February 1957.
4. Newell, A., and J. C. Shaw, Programming the Logic Theory Machine, The RAND Corporation, P-954, February 28, 1957. Published in the Proceedings of the Western Joint Computer Conference, IRE, February 1957.
5. Whitehead, Alfred North, and Bertrand Russell, Principia Mathematica, Vol. 1, 2nd ed., Part I, "Mathematical Logic," University Press, Cambridge, England, 1927, Sec. A., "The Theory of Deduction," pp. 90-126. Also published in soft-cover edition (\$1.95) by University Press, Cambridge, England (to *56).
6. Wang, H., "Toward Mechanical Mathematics," IBM J. Res. & Develop., Vol. 4, No. 1, January 1960, pp. 2-22.
7. Wang, H., "Proving Theorems by Pattern Recognition, I," Communications of the ACM, Vol. 3, No. 4, April 1960.
8. Minsky, M., "Steps Toward Artificial Intelligence," Proceedings of the IRE, January 1961, pp. 21-23.
9. Newell, A., J. C. Shaw, and H. A. Simon, The Process of Creative Thinking, The RAND Corporation, P-1320, January 28, 1959, pp. 21-49.

10. Minsky, op. cit., pp. 9-10.
11. Newell, Allen, Some Problems of Basic Organization In Problem-Solving Programs, The RAND Corporation, RM-3283-PR, December 1962.
12. Newell, Allen, Learning, Generality, and Problem-Solving, The RAND Corporation, RM-3285-1-PR, February 1963.

IPL-V CODING SHEET

Problem No. _____ Programmer _____ Date _____ Page _____ of _____

[illegible]